

人脸识别 Android 离线 SDK

用户接入文档

文档名称	人脸识别 Android 离线 SDK 用户接入文档
所属平台	Android 5.0+
适用版本	JDK 1.8+ & Gradle 3.0+
开发语言	Java
文档提交日期	2025.08.27

百度在线网络技术（北京）有限公司
(版权所有, 翻版必究)

更新记录

No	版本号	修改内容简介	修改日期
1	V1.1	初版，包括离线人脸采集、离线活体检测、离线对比识别、离线人脸库管理等功能	2018.06.29
2	v1.0.1	1、修复设备指纹发生变化 bug 2、替换近红外活体模型，优化效果 3、修复批量注册人脸到人脸库，失败问题 4、修复注册、图片人脸检测、视频返回图片抽取特征失败，经常出现检测不到人脸问题	2018.08.03

3	v1.1.0	1、增加离线证件照特征抽取模型，可有效处理芯片照、证件照比对需求 2、增加离线人脸属性模型，支持性别、年龄等属性分析 3、增加多线程支持 4、增加离线激活支持，可导入授权文件无网激活 5、增加对奥比中光 Astra Pro 深度图镜头模组支持 6、增加对华捷艾米深度镜头模组支持 7、其他细节优化及已知 bug 修复	2018.09.03
4	v2.0.0	1、优化生活照模型精度及速度 2、优化检测模型及策略 3、优化接口设计 4、优化活体检测速度	2019.01.10
5	v2.0.2	1、全新人脸检测模型，检测追踪更流畅； 2、全新证件照模型：体积更小，速度更快； 3、部分接口细节优化	2019.04.01
6	v2.0.3	1、更新硬件指纹获取模块，优化特殊环境下指纹变更的问题	2019.06.14
7	v3.0.0	1、新版检测模型，人脸检测及跟踪速度大幅提升； 2、新版 RGB、NIR、DEPTH 活体模型，优化活体检测的有效人脸大小尺寸范围，活体通过率大幅提升。增加更多攻击类型样本数据，非活体拒绝率大幅提升； 3、新版生活照模型，特征抽取速度大幅提升； 4、新版接口实现，功能逻辑精简优化，更易集成； 5、新版示例工程，优化功能逻辑，新增大量自定义设置项，效果调节更简单； 6、新增「按应用批量鉴权」，应用初始化时自动联网激活。	2019.07.25
8	v3.1.0	1、优化人脸检测性能； 2、新增离线人脸属性模型，包括年龄、种族、性别、7 种细分情绪和 3 种分类情绪、佩戴眼镜等属性	2019.08.13
9	v3.2.0	1、 1、底层支持多线程； 2、增加睁闭眼、张闭嘴模型； 3、优化 NIR 界面适配效果；	2019.12.10

		4、修复部分已知 Bug	
10	v4.2.1	1、增加场景化示例工程，针对「闸机通行」、「考勤打卡」、「金融支付」、「属性检测」、「人证核验」、「驾驶员行为分析」和「注意力检测」7 大应用场景提供定制化的交互流程和交互界面； 2、优化示例工程中的注册激活功能，提供快速、便捷的「在线激活」、「离线激活」和「批量激活」功能； 3、优化示例工程中人脸库管理功能和设置功能； 4、解决部分已知 Bug 注：4.2.1 版本主要针对示例工程进行了全面重构和优化，功能和接口与 4.1.1 版本保持一致	2020.07.07
11	v5.0	1、升级了人脸检测模型，重点优化了复杂光线场景下的人脸检测能力； 2、升级了 RGB、NIR 和 DEPTH 三种模态的活体检测模型，优化后 2D 照片攻击防御能力和复杂光线场景下的真人活体检测能力提升明显； 3、升级了最新的人脸特征抽取与比对模型（人脸识别模型），升级后识别模型对老人、儿童等极端年龄群体的泛化行提升明显； 4、在 RK3288 主板上，端到端全流程耗时缩短到<300ms； 5、增加了最优图像帧功能，提升识别准确率； 6、增加多线程的示例工程，支持边注册、边识别和多模态同时检测功能； 7、解决了部分已知 Bug。 注：由于该版本的 SDK 更换了特征抽取与比对的模型，因此从旧版本升级到该版本的用户需要重新注册人脸底库（刷库）	2020.10.16
12	v5.1	1、新增『金融活检』示例工程，该示例工程集成了多帧检测等活体检测策略，提升活体检测准确率； 2、新增『RGB+NIR』混合模态人脸识别示例工程，解决暗光人脸识别问题，提升弱光人脸识别准确率； 3、优化示例工程的功能和代码结构，更有利于直接使用和进行二	2021.01.01

		次开发； 2、4、修复部分已知 Bug	
13	v6.0	1、更新人脸检测模型，提升在强光、暗光、逆光、阴阳光等复杂光线场景下人脸检测的召回率； 2、更新 RGB、NIR 和 DEPTH 三种模态的活体检测模型，提升在复杂光线场景下真人活体检测通过率，同步提升高清 2D 和高质量 3D 道具的活体攻击防御效果； 3、更新人脸识别模型，提升复杂光线场景下人脸识别的准确率； 4、新增暗光恢复功能，对暗光场景下的图片进行亮度提升，提高检测和识别的准确率； 5、区分『通用版』和『口罩版』，通用版适用于非戴口罩场景下的人脸识别，口罩版适用于在戴口罩场景下完成人脸识别； 6、修复部分已知 Bug	2021.02.26
14	v7.0	1、更新人脸检测模型，解决部分场景下非人脸误检测的问题； 2、全面更新多模态活体检测模型，提升对新材质活体攻击道具的防御能力； 3、更新人脸识别模型，实现单模型同时支持戴口罩人脸识别和外国人识别； 4、增加业务层二次开发接口，降低 SDK 集成难度，保障端到端算法效果； 5、完善示例工程功能，优化示例工程交互； 6、修复部分已知 Bug	2021.08.24
15	v8.0	1、检测、活体、识别更新至 8.0 最新算法版本； 2、解决旧版本偶发崩溃问题，支持边注册、边识别； 3、隐藏情绪识别功能，优化 Demo 中人脸追踪框的效果	2022.04.20
16	v8.1	1、修复日志信息错误 bug； 2、去除应用层的网络判断逻辑，解决每次设备断网重启重新激活的问题； 3、添加安卓 11 系统兼容性配置	2022.11.18

17	V8.1.2	1、接入动态底库策略，解决由于底库质量差带来的误识别问题； 2、优化口罩检测、安全帽检测模型； 3、添加活体多帧策略，配合阈值设备同解决多镜头泛化问题； 4、添加加密芯片授权支持，软硬授权并线支持； 5、支持多款 3D 结构光模组兼容适配	2023.03.02
18	V8.2.0	1、优化了人脸特征提取模型； 2、考勤模块下新增了多人脸识别功能并支持 UI 横屏识别	2024.06.07
	V8.5.0	1、修改 demo UI，去掉部分模块减少 demo 工程复杂度，新增 1:1 比对示例； 2、新增调用云端特征提取接口示例代码； 3、升级支持安卓 13、14 高阶版本； 4、人证核验增加图片 1:1 比较功能； 5、优化部分已知问题	2024.06.07
20	V8.6.0	1、批量激活优化，工程添加静默激活 2、部分设备人脸跟踪卡顿问题优化 3、支持横屏效果展示 4、添加应用层日志打印 5、人脸特征多次注册内存溢出问题优化	

百度在线网络技术（北京）有限公司	1
(版权所有, 翻版必究)	1
1 设计背景	9
1.1 场景特点	9
1.2 用户特点	10
1.3 核心功能	10
1.4 使用场景	10
2 SDK 简介	10
2.1 功能架构	10
2.2 人脸识别流程	11
3 SDK 及 Demo 工程结构说明	12
3.1 官网工程包结构说明	12
3.2 工程结构说明	13
3.3 SDK aar 文件和模型文件说明	14
4、Demo 工程的编译和使用	16
4.1 授权激活	19
4.1.1 离线激活（适应于无网或弱网环境）	19
4.1.2 在线激活(需要设备能连网)	19
4.1.3 批量激活(需要设备能联网)	20
4.2 SDK Demo 使用说明（可直接编译 Demo 生成 apk 体验）	21
4.2.1 模型加载及镜头角度设置	21
在设置界面中找到镜头设置	21
4.2.2 图片及视频流注册	22
4.2.3 1:N 识别和 1:1 比对	24
5、SDK 以及项目的集成	25
5.1 集成说明	25
5.2 项目集成流程说明	26
5.3 工程核心类和方法说明	30
5.3.1 FaceUtils 类方法说明	31
5.3.2 FaceApi 类方法说明	31
5.3.4 CameraPreviewManager 类方法说明	35
5.3.6 FaceModel 类方法说明	37
5.4 工程数据库 db 文件路径自定义说明	39
5.5 在线特征提取接口调用方式	39
5.5.1 接口调用	39
5.5.2 特征字符串转换	40
5.6 摄像头模组集成方法说明	41
5.6.1 RGB 摄像头模组集成说明	41
5.6.2 RGB+NIR 双目摄像头模组集成说明	42
5.6.3 RGB+DEPTH 摄像头模组集成说明	42
5.7 多人脸识别说明	48
5.8 Log 日志说明	48

6 SDK 功能接口说明	49
6.1 FaceAuth 鉴权接口	49
6.1.1 在线授权	49
6.1.2 离线授权	49
6.1.3 批量授权	50
6.1.4 底层 log 日志接口	50
6.1.5 设置 cpu 核数	51
6.1.6 获取 deviceId 设备指纹	52
6.2 BdfFaceInstance 线程实例	52
6.2.1 开启 BdfFaceInstance 实例	52
6.3 FaceDetect 检测对象	52
6.3.1 模型加载接口	53
6.3.2 质量检测模型加载接口	53
6.3.3 最优人脸模型加载接口	54
6.3.4 配置信息加载接口	54
6.3.5 人脸检测接口	54
6.3.6 人脸跟踪接口	56
6.3.7 检测模型卸载接口	56
6.4 FaceLive 活体检测对象	56
6.4.1 活体模型加载接口	57
6.4.2 人脸静默活体检测接口	57
6.4.3 活体模型卸载接口	58
6.5 FaceFeature 特征提取对象	58
6.5.1 特征提取模型加载接口	59
6.5.2 特征提取接口	59
6.5.3 特征提取卸载接口	60
6.6 FaceSearch 识别对象	60
6.6.1 SDK 注册接口	60
6.6.2 识别多帧检测接口	61
6.6.3 1:N 识别接口	63
6.6.4 1:1 比对接口	63
6.7 BdfFaceImageInstance 图片对象	64
6.7.1 获取图片信息接口	64
6.7.2 图片销毁接口	65
6.8 FaceGaze 注意力检测对象	65
6.8.1 注意力模型加载接口	65
6.8.2 注意力信息获取接口	65
6.8.3 注意力卸载接口	66
6.9 FaceCrop 抠图对象	66
6.9.1 抠图能力加载接口	66
6.9.2 人脸关键点抠图接口	67
6.10 FaceMouthMask 口罩检测接口	67
6.10.1 口罩检测模型加载接口	67
6.10.2 口罩检测结果获取接口	68

6.10.3 口罩检测卸载接口	68
6.11 FaceSafetyHat 安全帽检测	68
6.11.1 安全帽模型加载接口	69
6.11.2 安全帽检测结果获取接口	69
6.11.3 安全帽检测卸载接口	69
7 SDK 实体类说明	70
7.1 FaceInfo 实体类	70
7.2 BDFaceOcclusion 实体类	71
7.3 BDFaceGazeInfo 实体类	71
7.4 Feature 实体类	72
8 升级指导	72
8.1 3.v - 4.v	72
8.2 4.v - 7.v	73
8.3 7.v - 8.v	73
9 激活和模型初始化错误码说明	75
9.1 在线和离线初始化错误码说明	75
9.2 模型初始化错误码说明	76
9.3 图片特征批量提取错误码说明	77
10 常见问题	77
10.1 误识别问题	77
10.1.1 误识底图与识别图片得分过高导致误识别	77
10.1.2 多人脸情况下误识别到后方人脸	79
10.1.3 没有注册底图的人被误识别到	80
10.1.4 全量下发人脸(特征注册)同时进行识别, 出现误识别如何解决	80
10.2 识别效果问题	81
10.2.1 识别速度慢, 识别率低, 识别得分低如何优化?	81
10.2.2 注册成功后无法正常识别到	82
10.2.3 数据库查询不到识别到的人员 id	83
10.2.4 戴口罩如何识别?	83
10.2.5 是否支持远距离如何识别和人脸检测	84
10.2.6 老版特征 SDK 更新后识别不到人脸	84
10.2.7 检测不到人脸, 结果返回为空	84
10.2.8 检测角度和回显角度是歪的如何解决	85
10.2.9 人脸框左右人脸镜像	86
10.2.10 RGB 活体检测被 2D 图片攻破	86
10.2.11 CPU 占用高 使用卡顿 如何解决	87
10.2.12 活体得分低如何排查	88
10.2.13 图片批量注册不成功 特征提取失败如何解决?	88
10.2.14 特征注册接口 pushPersonById 注册失败如何解决?	89
10.2.15 同一时间反复初始化数据库卡死如何解决?	89
10.2.16 接口崩溃如何解决?	90
10.2.17 多线程人脸检测(图片注册+识别)崩溃如果解决	90
10.2.18 如何判断图片中是否存在多人脸?	91
10.2.19 如何只检测画面中指定区域的人脸?	91

10.2.20 SDK 支持多远距离下识别？能否设置远距离人脸识别？	92
10.2.21 人脸特征是否支持保存到后台下发到其他设备？	92
10.2.22 通用版 SDK 与 3568 专版 SDK 有什么区别？ SDK 支持哪个版本的 npu?... ..	93
10.2.23 SDK 是否支持外国人识别？有没有专版 SDK	93
10.3 激活问题	93
10.3.1 设备指纹获取失败，在线激活报错 no enough params 如何解决	93
10.3.2 每次启动都需要激活吗？	93
10.3.3 相同设备不同应用能否使用相同激活码？	94
10.3.4 激活时填写了不属于硬件指纹的号码或设备指纹改变如何解决？	94
10.3.5 在线或批量激活可以不联网激活吗？	94
10.3.6 激活失败怎么办？	94
10.3.7 设备指纹绑定那些信息？	95
10.4 摄像头问题	95
10.4.1 SDK 支持哪些摄像头？	95
10.4.2 相机采集到的图像如果转成图片？	95
10.4.3 摄像头是黑白的，摄像头想调整为前（后）置摄像头如何调整	96
10.4.4 单目 NIR 能否进行人脸识别	96
10.4.5 更改分辨率后预览崩溃如何解决	96
10.5 demo 工程相关问题	97
10.5.1 人脸注册的图片存在 SD 卡 Success-Import 文件下，能否删除？	97
10.5.2 人脸 SDK 与 openCV 库冲突如何解决	97
10.5.2 demo 是否支持保存相机注册时的图片？	97
10.6 其他问题	98

1 设计背景

1.1 场景特点

- **网络**：无网、局域网等情况均可使用。如政府单位、金融保险、教育机构等。
- **安全**：行业特点所带来的人脸数据敏感性，许多客户不希望将人脸数据传入百度服务器，即使连接公网也不会请求。
- **速度**：由于各地网络线路、机房部署等诸多原因，网络请求速度往往耗时较高，容易影响业务正常运转，离线 SDK 不会存在网络延迟问题。
- **稳定**：离线 SDK 不会受网络抖动、机房故障、线上连带 bug 等影响，进一步控制可用性影响因素
- **海外人群**：支持海外人脸识别，适用于以海外人群为主的通行、考勤、人证核验等场景下的 1:1 及 1:N 人脸比对和搜索

1.2 用户特点

- **1:N-小型人脸库检索**: 多为通道通行、固定区域人群验证等需求, 如写字楼闸机门禁、企业考勤打卡等, 人脸库范围较小, 且不易经常变动。
- **1:1-自有数据源对比**: 有效应用于人证比对、身份核验等场景, 将当前采集的人脸, 与其他数据源中的人脸进行对比, 如身份证芯片照、教务系统图片、档案图片等, 进行快速的 1:1 对比验证。

1.3 核心功能

- **离线人脸检测**: 可在设备端, 离线实时检测视频流中的人脸。并对当前检测到的人脸进行持续跟踪、捕获、质量校验等, 动态定位人脸轮廓, 稳定贴合人脸。
- **RGB+NIR 活体检测**: 可判断可见光 RGB 成像、NIR 近红外成像是否为活体, 可有效防止屏幕二次翻拍等作弊攻击
- **DEPTH 深度图像(3D 结构光)活体检测**: 通过 3D 建模判断目标对象是否为活体, 基于 3D 结构光成像原理, 可强效防御图片、视频、屏幕、模具等攻击。
- **本地特征提取**: 所有在 SDK 中运行的人脸图片, 都可以完成本地特征抽取, 以便进行对比或识别操作。
- **1:1 对比**: 支持两张图片的相似度对比, 可直接传入图片, 获取图片中人脸特征进行比对、适应于人证核验等场景;
- **1:N 比对**: 可在指定人脸集合中, 找到最相似的人脸, 并返回相似度最高的人脸信息及对应分值。人脸库可存储于本地数据库, 并支持通过视频流实时采集的人脸特征值比对实现人脸的 1:N 识别。

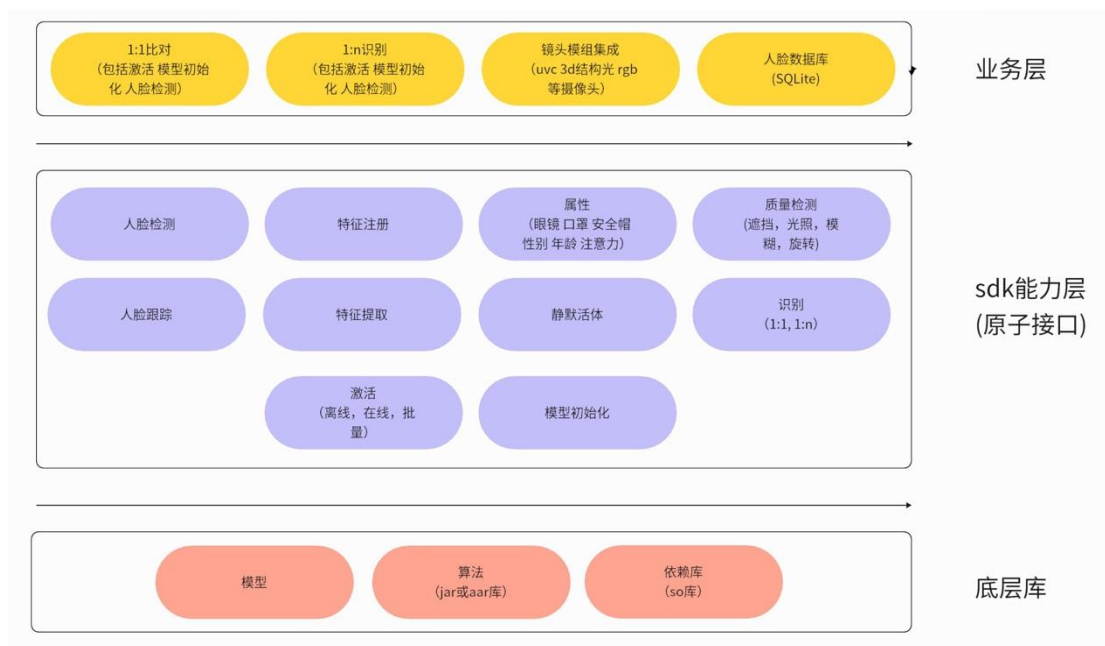
1.4 使用场景

- **通用版 SDK**: 适用于大部分 Android5.0 以上系统的手机和面板机, 由于不同设备、摄像头和 cpu 等的差异, 识别效果可能会存在细微差距。SDK 专门针对如 RK3399、RK3288 主板和迪威泰、视派尔等摄像头进行了模型和识别效果上的优化
- **3568 专版 SDK**: 适用于 3568、3566 的 npu 主板, 有 npu 算法加持, 相对通用版 SDK 专版在处理和识别速度上有明显提升, 提升范围在 1.5 到 3 倍左右。

2 SDK 简介

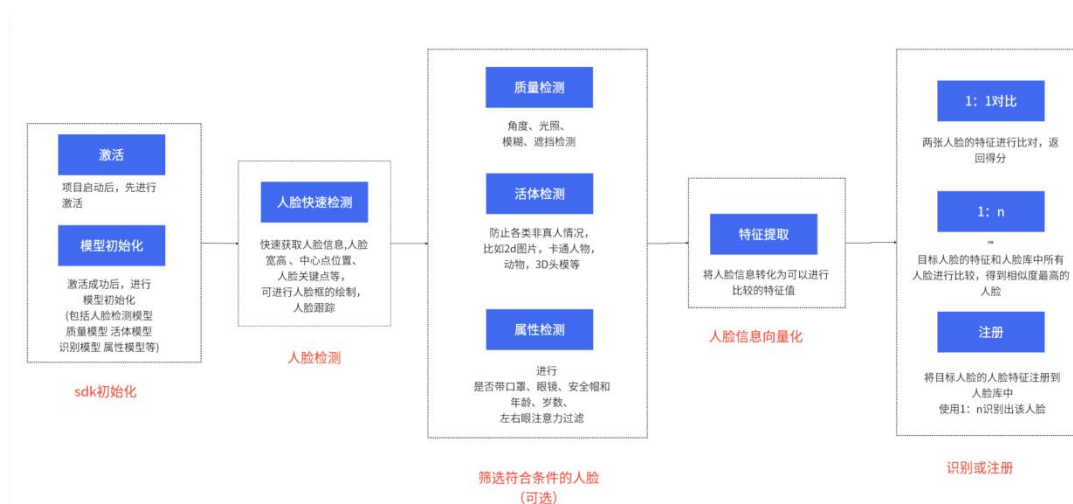
2.1 功能架构

SDK 具有人脸检测、跟踪、质量检测、特征提取、RGB、NIR 以及 3D 结构光静默活体、1:1 比对、1:N 识别、年龄、性别、是否戴口罩、戴眼镜和佩戴安全帽、左右眼球注意力等功能, 架构图如下:



2.2 人脸识别流程

人脸识别流程如下图所示，通过 SDK 的授权激活且模型初始化成功后，根据输入的图片或视频帧进行人脸检测，同时对人脸进行质量、活体和属性等进行过滤判断（通常情况下质量合格的图在 1:N 人脸识别时候识别率更高），通过对质量通过后的高质量图片进行人脸特征提取后、可根据人脸特征进行人脸 1:1 的比对和 1:N 的识别（1:N 识别前需要先进行人脸注册入本地库），通常以人脸特征的比对或识别阈值大于 80 分为比对成功或识别通过。



3 SDK 及 Demo 工程结构说明

3.1 官网工程包结构说明

目前百度 AI 官网有集成 SDK 摄像头、人脸数据库统一封装的效果展示的 SDK Demo 工程，官网的下载地址为：

<https://console.bce.baidu.com/ai-engine/old/#/ai/face/offline/index>



该工程中分为 Andorid 通用项目、DEPTH 3D 结构光项目、SDK、文档，四个板块，主要功能为：

文件夹名	说明
SDK	SDK 核心库，包含 SDK 的 aar 以及模型文件
FaceSDKAndroid	Android 通用项目，适用于大部分摄像头和安卓设备，支持 RGB+NIR 双目摄像头识别，集成方式可参考后述 5、SDK 以及项目的集成
FaceSDKAndroid_depth	DEPTH 3D 结构光项目，使用 RGB+DEPTH 的 3D 结构光摄像头场景，对 3D 头模的活体防御有明显提升，改项目仅支持对华捷 和 奥比 两款摄像头进行测试，集成方式可参考后述 5、SDK 以及项目的集成
开发文档	项目开发文档

3.2 工程结构说明

Demo 示例工程针对百度离线人脸识别 SDK 调用，并且对 sqlite 人脸数据库和相机等功能进行封装，FaceSDKAndroid Demo 工程结构如下：

工程模块名	说明
app	工程主模块，主要进行欢迎页，激活页，首页的展示。并对 SDK 进行激活和模型初始化接口的调用，sqlite 数据库初始化的工作
attributelibrary	属性模块，主要对识别的到人员性别、年龄、是否佩戴安全帽子 眼镜 口罩和左右眼注意力的效果展示
datalibrary	数据模块，对 SDK 原子接口(人脸检测，注册，识别，活体，质量等)、sqlite 数据库、摄像头展示、人脸绘制功能进行封装，依赖 faceaar
faceaar	SDK 核心库，添加了 SDK aar 包依赖
facelibrary	SDK 核心库，依赖 faceaar, 并添加 SDK 模型文件
gatelibrary	1:N 识别模块，展示界面有 闸机模式、考勤模式、支付模式，主要对人脸检测活体检测 质量检测 1:N 识别和耗时的效果展示, 依赖 datalibrary 模块
identifylibrary	1:1 识别模块，展示界面有 人证核验，主要针对 视频流和图片的 1:1 和 图片与图片的 1:1 效果展示，依赖 datalibrary 模块
registerlibrary	注册模块，主要针对视频流数据和图片数据信息特征提取和注册，注册成功后可在 gatelibrary 模块中识别出人员信息

settinglibrary	阈值设置界面，对活体检测、质量检测、人脸大小等阈值参数设置，及对摄像头角度、检测调度及镜像的设置
----------------	--

3.3 SDK aar 文件和模型文件说明

SDK 分为 aar(历史版本为 jar so)和模型资源文件等几个部分、SDK aar 库文件位于项目中的 faceaar/FaceSDK_8.5_20241220-release.aar 目录下

模型文件储存在项目 facelibrary/assets/face-sdk-models 文件夹中，各模型文件说明如下：

模型文件名称	说明	是否可删	删减说明
align_rgb-customized-pa- fast.model.float32-0.7.5.5	人脸快速对其模型	否	使用人脸跟踪能力可快速进行人脸检测
align_rgb-customized-pa- 80.model.float32-6.4.14.4	人脸对其模型	否	相比快速对其模型，通常与人脸检测模型同步使用。 再检测过程中更加稳定的模型，且后续的活体检测和识别都需要依赖人脸检测的输出结果，不建议删除
attribute-customized-pa- mobile.model.float32- 1.0.9.5	人脸属性模型	是	若不使用人脸属性检测功能，该文件夹可删除，
best_image-mobilenet-pa- dcqe449_live_e5l_relu_128. model.float32-1.0.3.1	最佳人脸	否	最佳人脸通常是质量判断的一个环节、不建议删除
blur-customized-pa- addcloud_quant_e19.model.f loat32-3.0.13.3	人脸质量模糊度检测	否	过滤模糊的图片，选择清晰的图片，可提高识别通过率
dark_enhance-customized-	暗光恢复模型	是	用于暗光情况图像优

pa-zero_depthwise.model.float32-1.0.2.2			化，若不使用该功能，该文件夹可删除
detect_rgb-customized-pa-192.model.float32-0.0.18.1	人脸检测模型	否	通常与人脸对其模型同时加载，可检测出人脸宽高位置及人脸关键点信息，且后续人脸特征人脸识别 人脸活体检测中需要用到该模型输出结果，不建议删除
driver_monitor_nir-customized-pa-DMS_rgb_nir_detect.model.float32-1.0.1.2	驾驶员行为模型	是	用于喝水 抽烟 打电话吃东西的行为检测，不需要该功能可删除
feature_live-mnasnet-pa-attention_v4.model.int8-2.0.239.1	特征提取模型	是	人脸特征提取模型，需要进行 1:N 识别或 1:1 比对的情况下不建议删除
gaze-customized-pa-mobile.model.float32-1.0.3.4	注意力检测模型	是	眼球上下左右注意力检测模型，不需要该功能可删除
mouth_mask-customized-pa-faceocc_3classes.model.float32-1.0.9.2	口罩检测模型	是	检测是否戴口罩，不需要该功能可删除
occlusion-customized-pa-paddle.model.float32-2.0.7.3	质量检测遮挡模型	否	面部遮挡模型，人脸识别时候，建议通过遮挡模型判断
attribute-customized-pa-anquanmao2023_v1.model.float32-1.0.73.1	安全帽检测模型	是	安全帽检测模型，不需要可删除
liveness_rgb-customized-	RGB 活体检测模型	是	用于 RGB 可见光单目静

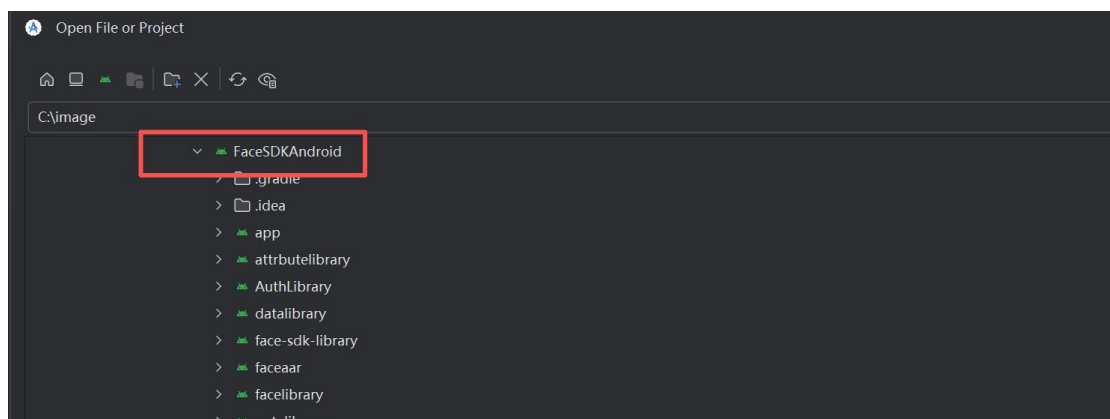
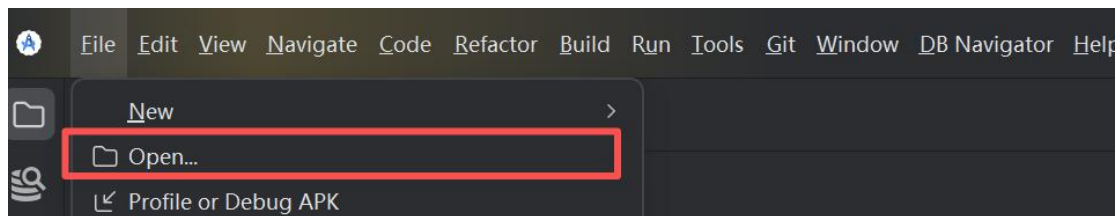
pa-DCQsdk80.model.float32-1.1.82.1			默活体，若只使用 rgb 静默活体，则该文件夹下除该文件外均可删除
liveness_nir-customized-pa-DCQ_80.model.float32-1.1.78.1	NIR 活体检测模型	是	用于 NIR 可见光单目静默活体，若没有 NIR 摄像头或不使用 NIR 活体可删除
liveness_depth-customized-pa-paddle_60.model.float32-1.1.13.2	DEPTH 活体检测模型	是	用于 3D 结构光静默活体检测，不使用 3D 结构光摄像头情况下可删除
liveness_rgb-customized-pa-DCQsdk80.model.float32-1.1.82.1	RGB 活体多音子检测模型	是	用于增强 RGB 可见光静默活体防攻击效果的模型，更好的防御电子设备中人脸图片的攻破效果，不需要可删除
liveness_rgb-customized-pa-hand_sdk_224.model.float32-1.1.69.1			
liveness_rgb-customized-pa-model_freeze_2dmask_20211210_sdk_224_epoch7.model.float32-1.1.80.1			

4、Demo 工程的编译和使用

SDK Demo 工程下载解压后，FaceSDKAndroid 工程如下：

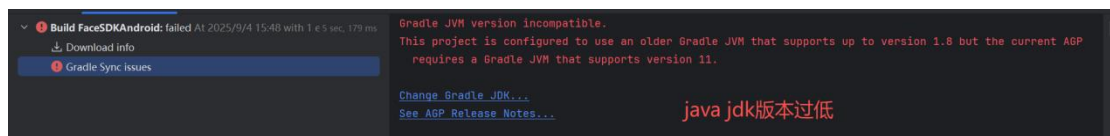


使用 android studio 打开 FaceSDKAndroid 项目

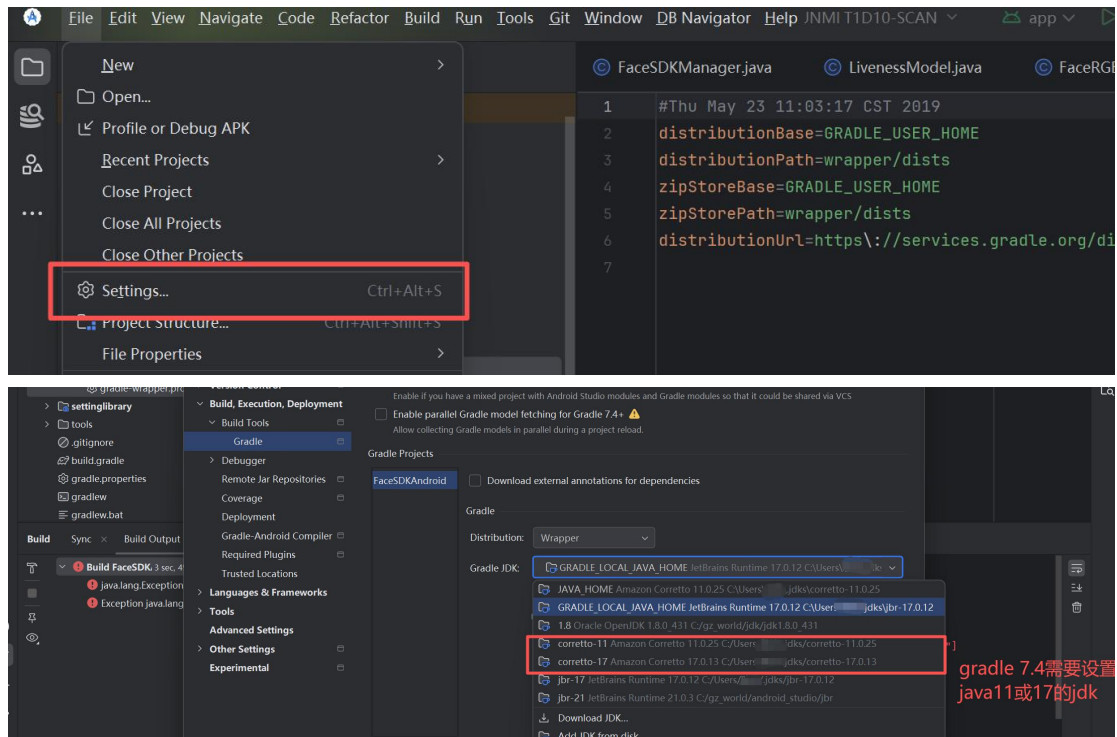


注:部分版本的 android studio 在编译的时候会出现错误,或者提示需要升级 gradle8.0 以上版本,该问题为 java jdk 不适配导致,目前 demo 的 gradle 版本为 7.4,可在 android studio 中设置 11 或 17 版本的 java jdk,截图如下:

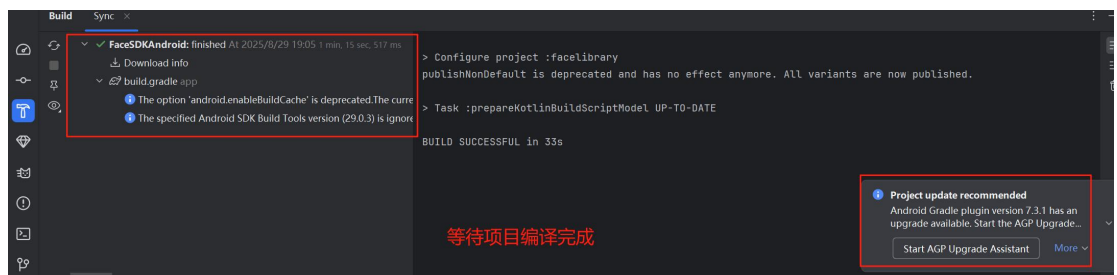
相关报错



解决方案



Jdk 设置完成后，等待项目编译完成，运行项目



注：android 手机需开启开发者模式和 usb 调试才可以正常运行，通常在手机设置-关于手机-找到系统版本连点 7 下即可开启开发者模式，不同型号手机位置不同



demo 主要运行流程为，其中激活须设备能连接网络(离线激活不需要)，激活用到的序列号可在百度 AI 官网申请，官网具体位置和激活流程请参考 [4.1 授权激活](#)，demo 主要流程如下：

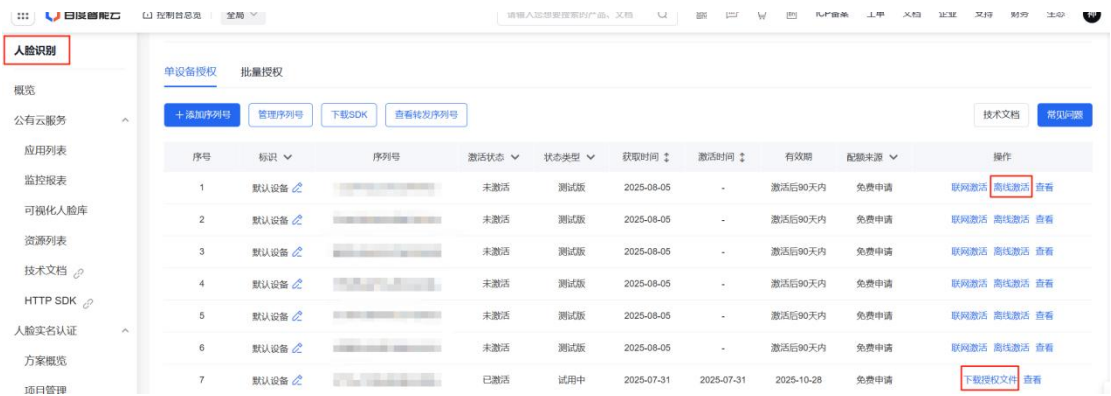


4.1 授权激活

SDK 分为离线激活、在线激活、批量激活三种激活方式(可使用其中任何一种进行激活)，SDK 需要通过授权激活后才能正常使用，三种激活方式可从官网申请激活文件或激活序列号(三种激活方式均会对通过指纹对设备进行绑定，如果存在系统更新、硬件更新、固件刷新的情况会导致设备指纹改变从而激活失败)，官网申请激活序列号以及离线激活地址如下：
https://console.bce.baidu.com/ai/?_from=1670415860750&fromai=1#/ai/face/offline/index

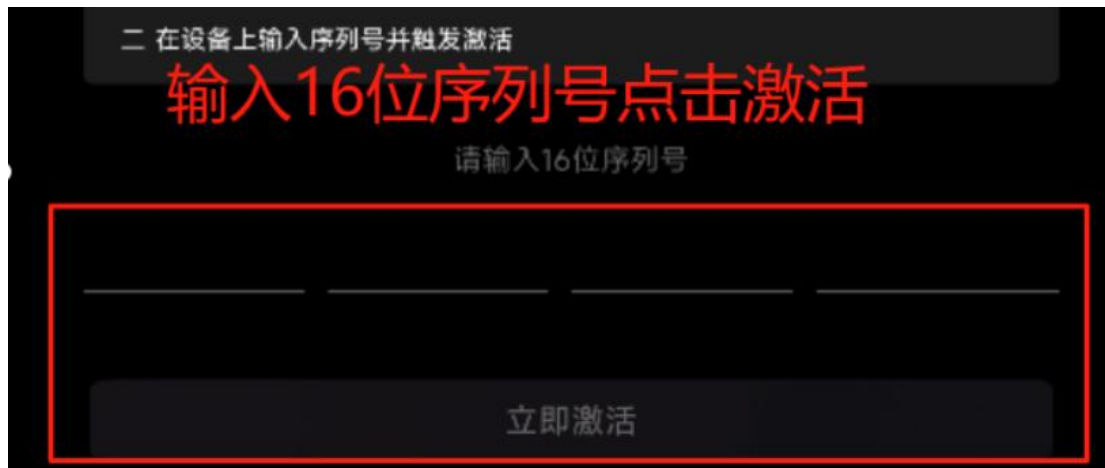
4.1.1 离线激活（适应于无网或弱网环境）

SDK Demo 运行后，首先会进入激活界面，在离线激活选项中可找到当前设备的设备指纹，在官网填入指纹信息，可获取授权文件（License.zip）无需解压，将授权文件放到设备sdcard(/sdcard/License.zip)目录下，进入离线激活界面，点击激活，即可完成激活，百度官网的离线激活页面如下：



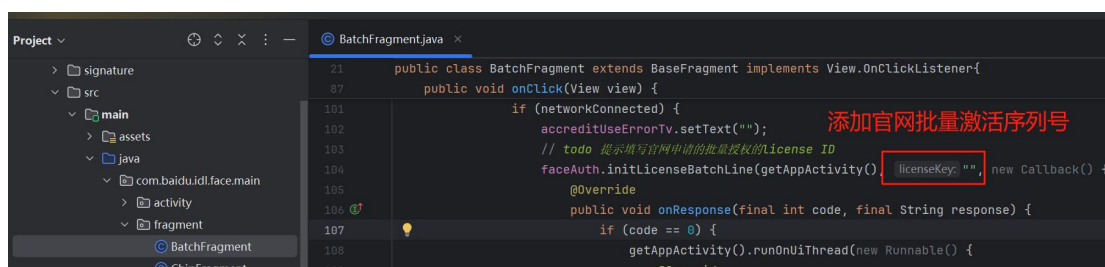
4.1.2 在线激活(需要设备能联网)

SDK Demo 运行后(首次需联网)，首先会进入激活界面，点击在线激活，在激活选项中输入官网的 16 位序列号，点击激活，即可完成在线激活，激活授权序列号可参考如下页面（百度 AI 官网登录账号购买后可显示）：



4. 1. 3 批量激活(需要设备能联网)

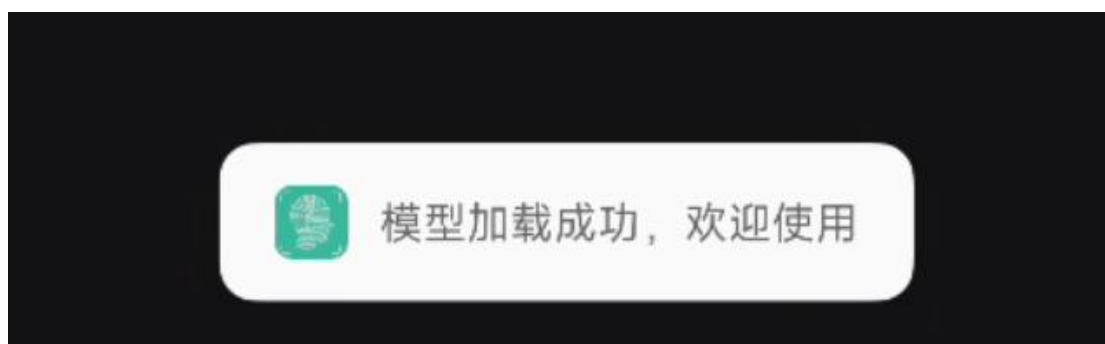
SDK Demo 运行后(首次需联网)，首先会进入激活界面，点击批量激活，该激活方法需要在代码中添加批量激活序列号，在官网-批量授权选项中可申请批量激活授权序列号，批量激活序列号可参考如下页面在百度 AI 官网获取、获取后填写到如下页面代码中（红色标记处）：



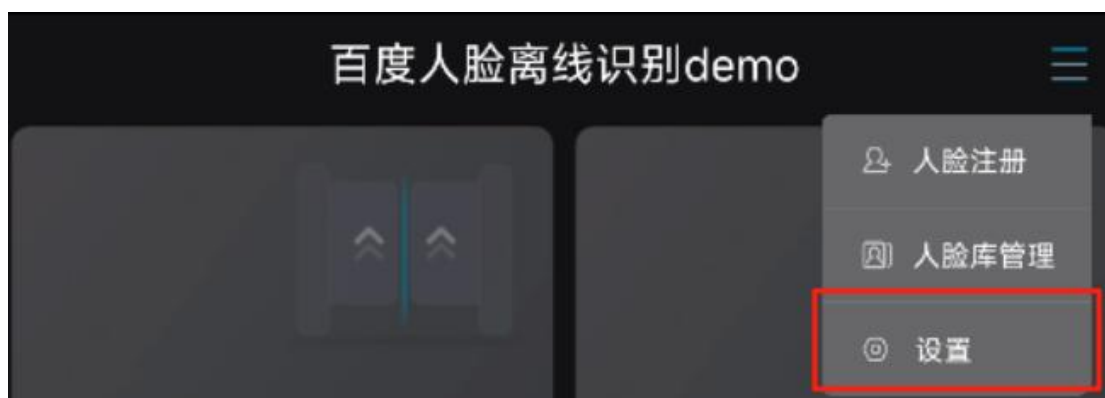
4.2 SDK Demo 使用说明（可直接编译 Demo 生成 apk 体验）

4.2.1 模型加载及镜头角度设置

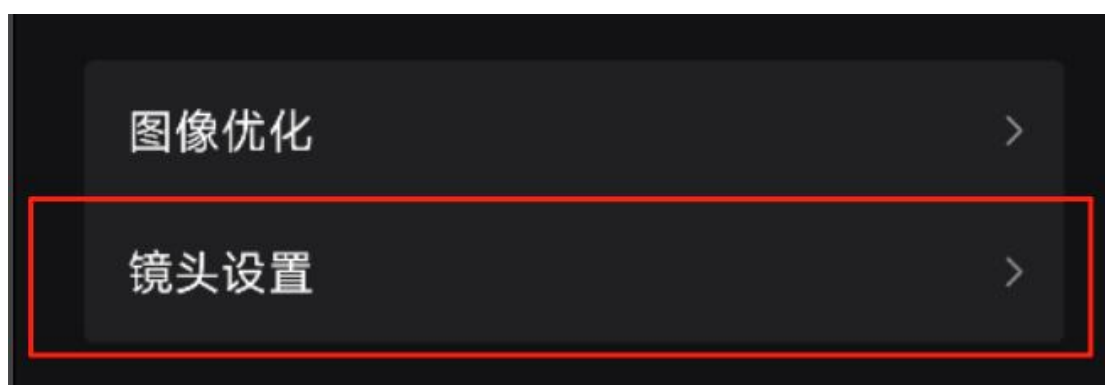
SDK 激活成功后(激活可参考 [4.1 授权激活](#))会进入 demo 首页，此时会自动进行模型初始化加载操作，等待首页提示模型初始化成功后，再进行后续操作，模型初始化接口及弹窗提示效果如下：



模型初始化成功后，部分设备还不能进行注册和人脸识别，需在设置界面中调整人脸检测和摄像头的角度（须人脸正向），角度正确后可进行注册和人脸识别操作，界面如下：



在设置界面中找到镜头设置



镜头设置中分为:人脸检测图像设置为 SDK 对视频流进行解码的角度(识别角度), 视频流回显图像设置为摄像头在屏幕中显示的角度(显示角度), 具体界面如下(如人脸不是正向的时候, 可点击旋转按钮把人脸摆正):

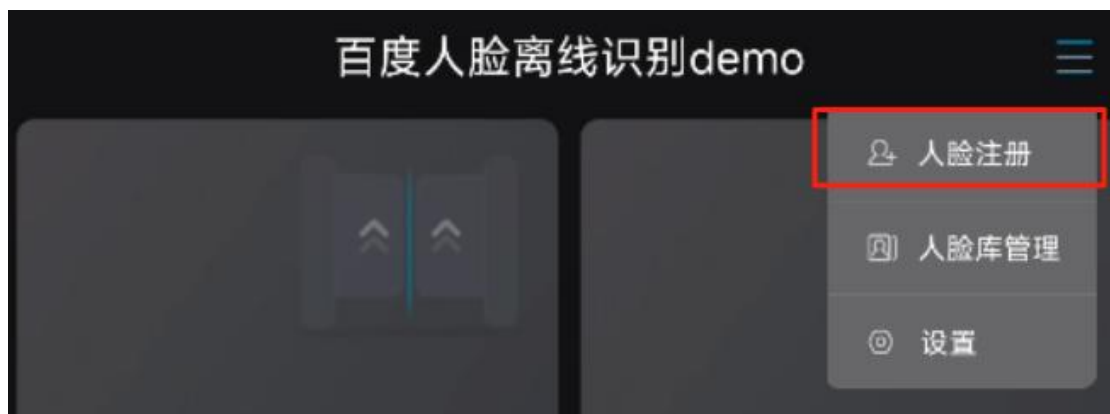


4.2.2 图片及视频流注册

设置好摄像头角度后, 返回首页, 在右上角按钮中可找到人脸注册和人脸库管理选项 (进行人脸 1:N 识别, 需要先进行人脸注册入库, 1:N 识别时候是实时检测的视频帧和人脸库里的已经注册入库的 N 个人进行比对识别), 其中人脸注册功能支持将摄像头中检测的人脸数据注册到数据库 (也支持图片批量导入注册入库), 人脸库管理中可查看注册的人脸信息并且也可以对图片进行人脸注册, 整体效果截图如下:

视频流注册:

选择人脸注册



检测到人脸后，输入用户信息，点击确认即可完成注册



图片注册：

图片注册需要手动拷贝将需要注册的人脸图片放到设备/sdcard/Face-Imort/文件夹中，在人脸库管理-批量导入中方可进行图片注册，效果如下：



在人脸库管理中还可以看到目前数据库中的人脸数据进行查询和删除的操作，点击右上角批量导入，方可进行图片注册（注：目前 demo 对图片名进行控制，不得超过 5 个字、另外可能会有部分不符合质量要求的人脸注册入库时候失败，具体可根据调试信息查看）



4.2.3 1:N 识别和 1:1 比对

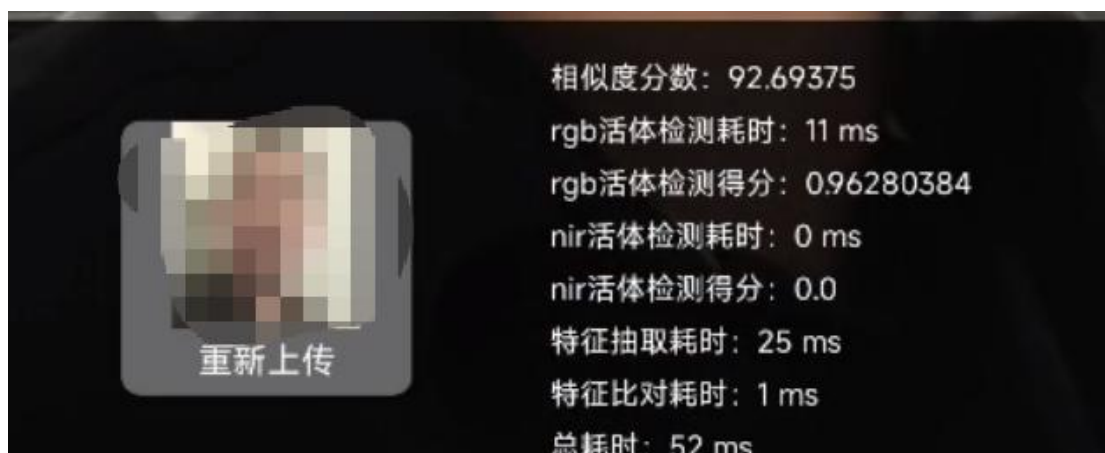
1:N 识别:

SDK Demo 提供了闸机模式、支付模式、考勤模式，这几种场景化示例 Demo 均为 1:N 人脸识别功能，除界面不同外识别效果和接口基本相同，开发者可根据项目需求选择不同界面进行集成，通过操作可将人脸注册入库，并且在识别时候对视频流中的人脸进行人脸框绘制、活体、质量检测的判断、提取摄像头中人脸特征、最后跟前述已经注册入库的人脸特征值进行识别比对，返回相似度最高的人员信息和各个 SDK 原子接口的识别耗时以及总耗时，具体页面展示效果如下：



1:1 比对:

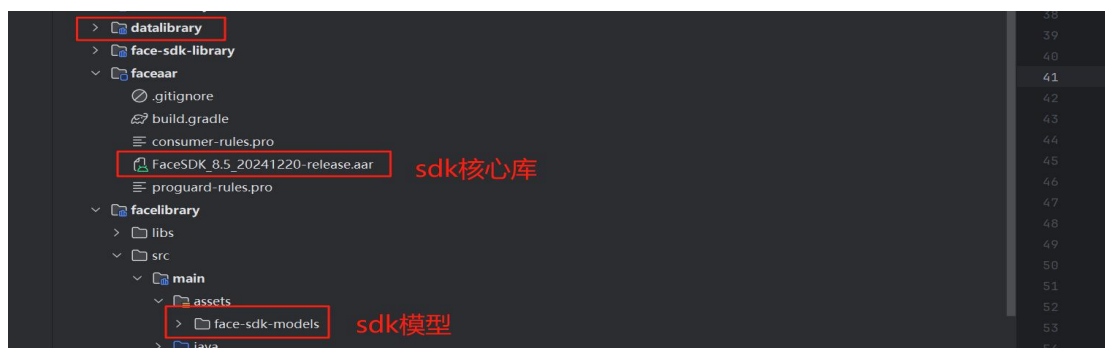
SDK Demo 中的人证核验模块为 1:1 比对展示界面，点击底部上传证件照按钮，进入相册选择证件照图片方可进行视频流 1:1 比对，比对展示效果如下：



5、SDK 以及项目的集成

5.1 集成说明

项目集成分为业务层(安卓原生)和 SDK 能力层(原子接口)的集成，集中需要将 SDK 核心库、SDK 模型和 dataLibrary 模块集成到项目中。（其中 dataLibrary 模块为安卓原生业务层代码，内容可以自行修改，该模块包含了摄像头启动、sqlite 数据库封装、SDK 原子接口封装，可以更高效率的进行 SDK 的接口集成）具体位置为：



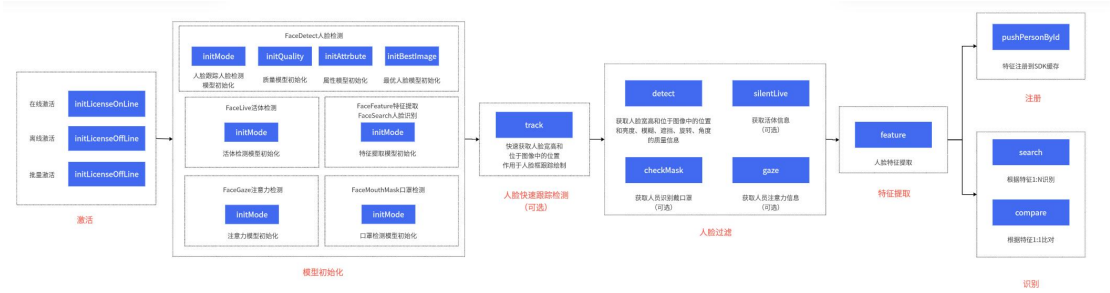
模块以及核心库导入完成后，将 1:N(gateLibrary) 或 1:1(identiflylibrary) 模块中的 FaceUtils 文件放入到项目，该文件对 SDK 初始化以及阈值进行封装，可更好的设置 SDK 参数，具体可参考 [5.3.1 FaceUtils 类方法说明](#)



上述功能导入后，可直接使用 dataLibrary 中封装好的类进行界面展示和识别接口的调用，dataLibrary 功能说明可参考 [5.3 工程核心类和方法说明\(dataLibrary\)](#)

5.2 项目集成流程说明

项目集成中根据自身不同业务场景，可自由定制化开发，其中 SDK 能力层主要流程和接口截图流程如下：



SDK 核心库、FaceUtils、SDK 模型和 dataLibrary 模块集成到项目中后，先调用激活接口 initLicenseOffLine(离线激活)、initLicenseOnLine(在线激活)、initLicenseBatchLine(批量激活)，激活成功(OnResponse 方法 code 参数为 0 则成功)后，才可以进行之后操作，具体代码如下：

```
bdFaceAuth.initLicenseOffLine(getActivity(), new Callback() {
    @Override
    public void onResponse(int code, String response) {
        // 离线激活
        getActivity().runOnUiThread(new Runnable() {
            @SuppressWarnings("SetTextI18n")
```

```
bdFaceAuth.initLicenseOnLine(getActivity(), end, new Callback() {
    @Override
    public void onResponse(final int code, final String response) {
        // 在线激活
        if (code == 0) {
            getActivity().runOnUiThread(new Runnable() {
                @Override
```

```
// todo 提示填写官网申请的批量授权的license ID
faceAuth.initLicenseBatchLine(getActivity(), licenseKey: "", new Callback() {
    @Override
    public void onResponse(final int code, final String response) {
        // 批量激活
        if (code == 0) {
            getActivity().runOnUiThread(new Runnable() {
```

激活成功后，需要进行 SDK 模型初始化和数据库初始化，模型初始化成功后才可以进行后续识别和注册操作，具体代码如下：

```
if (FaceSDKManager.initStatus != FaceSDKManager.SDK_MODEL_LOAD_SUCCESS) {
    FaceSDKManager.getInstance().initModel(mContext,
    FaceUtils.getInstance().getBDFaceSDKConfig(), new SdkInitListener() {
        // 模型初始化接口
        @Override
        public void onStart() {}
        @Override
        public void initLicenseSuccess() {}
        @Override
        public void initLicenseFail(int errorCode, String msg) {}
        @Override
        public void initModelSuccess() {}
        @Override
        public void initModelFail(int errorCode, String msg) {}
    });
    FaceSDKManager.initModelSuccess = true; // 模型初始化成功后，将该参数设置为true，防止反复初始化
    ToastUtils.toast(mContext, "模型加载成功，欢迎使用");
} else {
    initDBApi();
    FaceSDKManager.initModelSuccess = true;
```

其中 FaceSDKManager.getInstance().initModel 接口包含：

FaceDetect 类中人脸检测模型的初始化，可参考 [6.3.1 模型加载接口](#)

FaceDetect 类中质量检测模型的初始化，可参考 [6.3.2 质量检测模型加载接口](#)

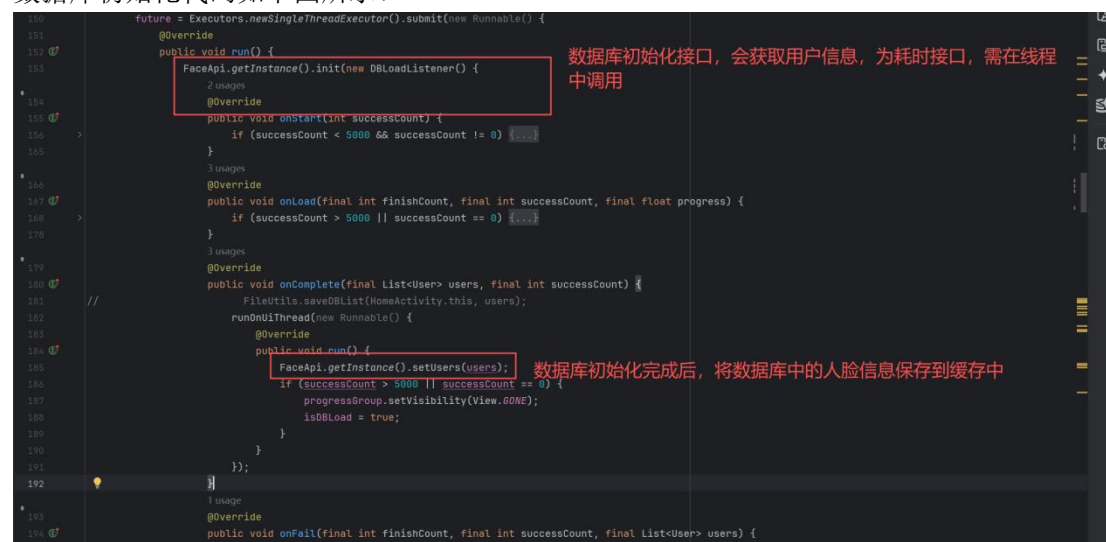
FaceLive 类中活体检测模型的初始化，可参考 [6.4.1 活体模型加载接口](#)

FaceFeature 类中人脸特征提取模型的初始化，可参考 [6.5.1 特征提取模型加载接口](#)

FaceSearch 人脸识别模型的初始化，可参考 [6.6 FaceSearch 识别对象](#) 等，单个模型初始化接口可参考下图：



数据库初始化代码如下图所示：



模型初始化成功后，需要进行人脸注册才可以正常识别到人脸，

通过 FaceSDKManager.getInstance().personDetect 获取图片的人脸特征结果，



其中，FaceSDKManager.getInstance().personDetect 特征提取接口中包含：

FaceDetect 的 detect 人脸检测接口调用，可参考 [6.3.5 人脸检测接口](#)

FaceFeature 的 feature 人脸特征提取接口调用，可参考 [6.5.2 特征提取接口](#)，具体截图如下

```

1153 public float personDetect(
1154     final Bitmap bitmap, final byte[] feature, final BDFaceCheckConfig bdFaceCheckConfig, Context context) {
1155     BDFaceImageInstance rgbInstance = new BDFaceImageInstance(bitmap);
1156     float ret = -1;
1157     FaceInfo[] faceInfos;
1158     BDFaceQualityConfig bdQualityConfig = bdFaceCheckConfig == null ? null : bdFaceCheckConfig.bdQualityConfig;
1159     if (bdFaceCheckConfig != null) {
1160         bdFaceCheckConfig.bdFaceDetectListConfig.usingDetect = true;
1161         faceInfos = faceModel.getFaceDetectPerson().detect(
1162             BDFaceSDKCommon.DetectType.DETECT_VIS,
1163             BDFaceSDKCommon.AlignType.BDFACE_ALIGN_TYPE_RGB_ACCURATE,
1164             rgbInstance,
1165             faceInfos, null,
1166             bdFaceCheckConfig.bdFaceDetectListConfig);
1167     } else {
1168         faceInfos = faceModel.getFaceDetectPerson().detect(BDFaceSDKCommon.DetectType.DETECT_VIS, rgbInstance);
1169     }
1170     if (faceInfos != null && faceInfos.length > 0) {
1171         // 判断质量检测, 针对模糊度、遮挡、角度
1172         if (onPersonQualityCheck(faceInfos[0], bdQualityConfig, new FaceQualityBack(context))) {
1173             ret = faceModel.getFacePersonFeature().feature(
1174                 BDFaceSDKCommon.FeatureType.BDFACE_FEATURE_TYPE_LIVE_PHOTO,
1175                 rgbInstance,
1176                 faceInfos[0].landmarks,
1177                 feature);
1178         } else {
1179             rgbInstance.destory();
1180             return -10;
1181         }
1182     }
1183     rgbInstance.destory();
1184     return ret;

```

人脸检测

特征提取

特征提取成功后，通过 FaceApi 的 registerUserIntoDBmanager 或 userAdd 接口注册到数据库，

```

// 将用户信息保存到数据库中
boolean importDBSuccess = FaceApi.getInstance().registerUserIntoDBmanager( groupName: null,
    userName, picName, userInfo: null, bytes); // 人脸特征

// 保存数据库成功
if (importDBSuccess) {
    // 将用户信息保存到数据库
}

```

数据库保存成功后，在通过 FaceSDKManager.getInstance().initPush，将数据库的人脸信息放到 SDK 缓存中

```

private BDLiveConfig bdLiveConfig;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    FaceSDKManager.getInstance().initPush(context: this);
    initFaceCheck();
}

```

进行识别前，将数据库的人脸特征放入数据库

注册成功后，在识别界面创建识别阈值 BDFaceCheckConfig 和 BDLiveConfig，其中包含识别、活体、质量等阈值的配置信息

```

private BDFaceCheckConfig bdFaceCheckConfig;
// 2 usages
private BDLiveConfig bdLiveConfig;

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    FaceSDKManager.getInstance().initDataBases(context: this);
    initFaceCheck();
}

// 相关配置参数
// 1 usage
private void initFaceCheck() {
    bdFaceCheckConfig = FaceUtils.getInstance().getBDFaceCheckConfig();
    bdLiveConfig = FaceUtils.getInstance().getBDLiveConfig();
}

```

识别阈值参数创建完成后，创建 BDFaceImageConfig 摄像头数据参数类，并使用 CameraPreviewManager 和 GLMantleSurfacView 打开摄像头，并展示摄像头界面

```
3 usages
private BDFaceImageConfig bdFaceImageConfig;
// 初始化rgb数据
1 usage
@Override
protected void initFaceConfig(int height, int width) {
    bdFaceImageConfig = new BDFaceImageConfig(height, width,
        SingleBaseConfig.getBaseConfig().getRgbDetectDirection(),
        SingleBaseConfig.getBaseConfig().getMirrorDetectRGB(),
        BDFaceSDKCommon.BDFaceImageType.BDFACE_IMAGE_TYPE_YUV_NV21);
}
```

RGB摄像头参数, 宽高 角度 视频帧等

```
<com.example.datalibrary.gl.view.GLMantleSurfaceView
    android:id="@+id/camera_textureview"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#121212" />
```

```
27 @Override
28 protected void onCreate(Bundle savedInstanceState) {
29     super.onCreate(savedInstanceState);
30     setContentView(R.layout.activity_camera_face_identify);
31     initView();
32 }
33 1 override
34 protected void initView() {
35     glmantleSurfaceView = findViewById(R.id.camera_textureview);
36     glmantleSurfaceView.initSurface(SingleBaseConfig.getBaseConfig().getRgbRevert(),
37     SingleBaseConfig.getBaseConfig().getMirrorVideoRGB(), SingleBaseConfig.getBaseConfig().isOpenGL());
38     CameraPreviewManager.getInstance().startPreview(glmantleSurfaceView,
39     SingleBaseConfig.getBaseConfig().getRgbVideoDirection(), PREFER_WIDTH, PREFER_HEIGHT);
40 }
41 @Override
42 protected void onResume() {
43     super.onResume();
44     startTestOpenDebugRegisterFunction();
45 }
46 1 usage
47 private void startTestOpenDebugRegisterFunction() {
48     if (SingleBaseConfig.getBaseConfig().getRGBCameraId() != -1) {
49         CameraPreviewManager.getInstance().setCameraFacing(SingleBaseConfig.getBaseConfig().getRGBCameraId());
50     } else {
51         CameraPreviewManager.getInstance().setCameraFacing(CameraPreviewManager.CAMERA_FACING_FRONT);
52     }
53     int[] cameraSize = CameraPreviewManager.getInstance().initCamera();
54     initFaceConfig(cameraSize[1], cameraSize[0]);
55     isPause = true;
56     CameraPreviewManager.getInstance().setmCameraDataCallback(new CameraDataCallback() {
57         2 usages
58         @Override
59         public void onGetCameraData(byte[] data, Camera camera, int width, int height) {
60             onCameraData(data);
61         }
62     });
63 }
```

onCreate中初始化摄像头

开启摄像头

初始化摄像头数据,

获取摄像头每帧参数

摄像头启动成功后, 在 CameraPreviewManager 的 setmCameraDataCallback 接口可获取到摄像头每帧数据, 将每帧数据放到 BDFaceImageConfig 摄像头数据参数类中, 通过 FaceSDKManager.getInstance().onDetectCheck 对该数据进行人脸识别操作

```
56 bdFaceImageConfig.setData(data); 将摄像头每帧数据, 放到摄像头参数类中
57 FaceSDKManager.getInstance().onDetectCheck(bdFaceImageConfig, bdMirFaceImageConfig, bdDepthFaceImageConfig: null,
58 bdFaceCheckConfig, new FaceDetectCallback() {
59     8 usages
60     @Override
61     public void onFaceDetectCallback(final List<LivenessModel> models) {
62         if (isVideo) {
63             upload(models);
64             if (developFragment.isSaveImage) {
65                 SaveImageManager.getInstance().saveImage(models, bdLiveConfig);
66             }
67         }
68     }
69     26 usages
70     @Override
71     public void onTip(int code, final String msg) {
72     }
73     4 usages
74     @Override
75     public void onFaceDetectDarwCallback(List<LivenessModel> models) {
76         // 人脸检测结果
77         showFrame(models);
78     }
79 });
80 }
```

人脸结果回调

人脸快速检测回调, 进行人脸绘制跟踪

其中 FaceSDKManager.getInstance().onDetectCheck 识别接口包含:

FaceDetect 类中的 track 人脸快速检测接口调用, 可参考 [6.3.6 人脸跟踪接口](#)

FaceDetect 类中的 detect 人脸检测接口调用, 可参考 [6.3.5 人脸检测接口](#)

FaceLive 类中的 silentLive 人脸活体检测接口调用, 可参考 [6.4.2 人脸静默活体检测接口](#)

FaceFeature 类中的 feature 人脸特征提取接口调用，可参考 [6.5.2 特征提取接口](#)
FaceSearch 类中的注册接口调用，可参考 [6.6.1.1 单个特征注册接口](#)
FaceSearch 类中的 1:N 识别接口调用，可参考 [6.6.3 1:N 识别接口](#)
FaceSearch 类中的 1:1 识别接口调用，可参考 [6.6.4 1:1 比对接口](#)，具体代码截图如下：

```
// 快速检测获取人脸信息，仅用于绘制人脸框，详细人脸数据后续获取
FaceInfo[] faceInfos = faceModel.getFaceTrack().track(
    BDFaceSDKCommon.DetectType.DETECT_VIS,
    BDFaceSDKCommon.AlignType.BDFACE_ALIGN_TYPE_RGB_FAST,
    rgbInstance);

return faceInfos;
```

快速人脸检测接口，可在10到20毫秒内检测出图像中的人脸信息

```
if (bdFaceCheckConfig != null) {
    bdFaceCheckConfig.bdFaceDetectListConfig.usingQuality = true;
    faceInfos = faceModel.getFaceDetect().detect(
        BDFaceSDKCommon.DetectType.DETECT_VIS,
        BDFaceSDKCommon.AlignType.BDFACE_ALIGN_TYPE_RGB_ACCURATE,
        rgbInstance,
        fastFaceInfos,
        bdFaceCheckConfig.bdFaceDetectListConfig);
} else {
    faceInfos = faceModel.getFaceDetect().detect(BDFaceSDKCommon.DetectType.DETECT_VIS, rgbInstance);
}
```

人脸检测接口，该接口除了返回人脸宽高和位置信息，还会返回角度 遮挡 光照 模糊 年龄 性别等信息

这两个接口均为人脸检测接口，但第一个可以灵活控制需要识别到的功能，且可以使用track的人脸信息，提升检测速度

```
Log.e( tag: "test_camera", msg: rgbInstance.getImage().data.length + "开始");
scores = faceModel.getFaceLive().silentLive(type, rgbInstance, faceInfo.landmarks, liveScore);
return scores;
```

活体检测接口，判断当前图像人脸是否为真人，不是2D图像或动物等人脸

```
List<User> users = FaceApi.getInstance().getAllUserList();
for (int i = 0; i < users.size(); i++) {
    User user = users.get(i);
    faceModel.getFaceSearch().pushPersonById(user.getId(), user.getFeature());
}

if (FaceApi.getInstance().getmUserNum() != 0) {
    ToastUtils.toast(context, text: "人脸库加载成功");
}
```

人脸特征注册接口，将提取的特征注册到缓存

```
List<? extends Feature> featureResult =
    faceModel.getFaceSearch().search(type, threshold: 0, topNum: 1, feature );
// TODO 返回top num = 1 个数据集合，此处可以任意设置，会返回比对从大到小排序的num 个数据集合
if (featureResult != null && featureResult.size() > 0) {

    // 获取第一个数据
    Feature topFeature = featureResult.get(0);
    // 判断第一个人脸特征是否和传入的人脸特征相似
```

人脸识别接口，将视频流中的特征与缓存中的特征进行比对，返回得分最高的信息

```
float score = faceModel.getFaceSearch().compare(
    BDFaceSDKCommon.FeatureType.BDFACE_FEATURE_TYPE_ID_PHOTO,
    livenessModel.getFeature(),
    secondFeature,
    isPercent: true);
```

1:1比对，传入两个人脸特征返回相似度

5.3 工程核心类和方法说明

Demo 工程中的 dataLibrary 模块为 demo 应用层核心模块，封装了数据库、摄像头以及 SDK 识别功能的使用，核心功能类分为：

类名	功能说明
FaceApi	数据库封装类，会在默认路径创建 face.db 人脸库文件、进行对人员信息和人脸特征的增删改查、都会保存在该文件中，具体可参考 5.3.2 FaceApi 类方法说明

FaceSDKManager	SDK 流程封装，通过一个方法可进行人脸检测、活体检测、质量检测、1:1 比对、1:N 识别效果，且封装了人脸模型的初始化，人脸特征注册接口，具体可参考 5.3.3 FaceSDKManager 类方法说明
CameraPreviewManager	摄像头启动类，单例的工具类仅支持一个摄像头的启动，可通过该类进行 RGB 摄像头的展示，对界面预览中出现的变形问题进行优化处理，并获取摄像头逐帧数据，具体可参考 5.3.4 CameraPreviewManager 类方法说明
PreviewTexture	摄像头启动类，相比 CameraPreviewManager 更加轻量级的效果展示，支持创建多个实例进行多个摄像头同时启动，具体可参考 5.3.5 PreviewTexture 类方法说明
FaceModel	SDK 接口封装类，封装了模型初始化(包含检测 活体 质量 识别 特征提取 口罩等模型)和 SDK 原子接口的封装，可进行快速初始化和对原子接口快速调用

5.3.1 FaceUtils 类方法说明

该类对识别、质量(光照、遮挡、模糊、角度)、活体等参数的阈值进行了封装，可通过该类快速获取识别和 SDK 初始化时所需要的阈值信息

方法名	所需参数	返回值	功能说明
GetBDFaceSdkConfig()	无	BDFaceSDKConfig: 人脸检测配置实体类	获取人脸检测实体类，通常在模型初始化时传入
GetBDFaceCheckConfig()	无	BDFaceCheckConfig	获取 1:1 和 1:N 时所需要的阈值信息
GetBDLiveConfig()	无	BDLiveConfig	获取活体阈值信息

5.3.2 FaceApi 类方法说明

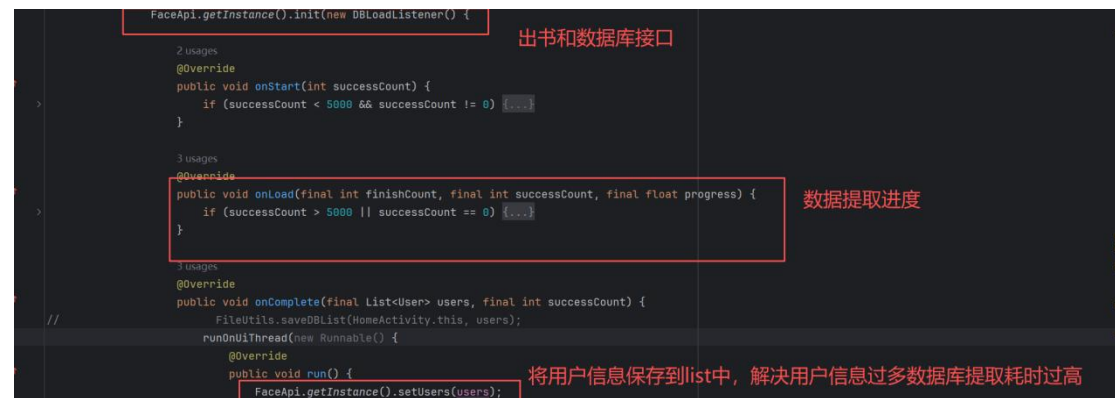
该类为业务层封装类，客户可根据自身业务调整内部逻辑，主要作用为人脸数据库的初始化，和人员信息的增删改查

方法名	所需参数	功能说明
init(DBLoadListener dbLoadListener, Context context)	DBLoadListener: 初始化返回结果 Context: 上下文	初始化数据库
userAdd(User user)	User: 用户信息	添加用户
getAllUserList()	无	查询所有用户
getUserListByUserName(String userName)	String: 用户名	根据用户名查询用户
getUserListById(int _id)	_id: 用户 id	根据用户 id 查找用户
userDeleteByName(String userName)	String: 用户名	根据用户名删除用户

registerUserIntoDBmanager(String groupName, String userName, String picName, String userInfo, byte[] faceFeature)	groupName:群组名 userName:用户名 picName:人脸图片名 userInfo:用户信息 faceFeature:人脸特征	添加用户,同 userAdd
---	---	----------------

faceSDKManager 相关接口截图及说明如下:

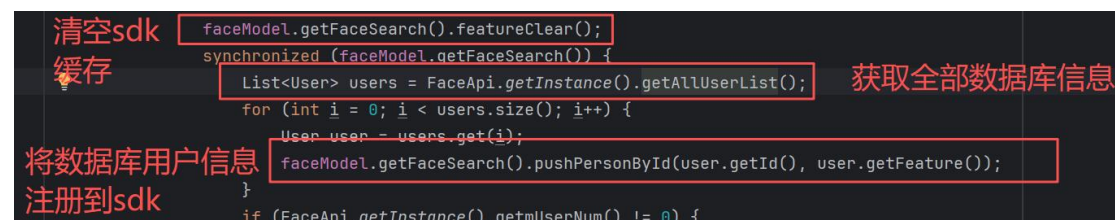
初始化人脸数据库, 会创建数据库并且获取数据库中的所有用户信息(该操作为耗时操作, 需要在线程中进行), 获取到的所有用户信息建议储存在 list (users) 中, 方便后续进行用户信息提取操作, 优化每次获取用户全部信息耗时过高问题



将用户信息注册到数据库, 进行识别前需要提取出该信息放入 SDK 缓存中进行识别, 具体识别接口可参考 [6.6 FaceSearch 识别对象](#)



查询所有用户, demo 在应用层做了初始化后可以将提取到的用户数据作为 list 放到缓存的操作, 所以此接口获取不会出现耗时操作



根据姓名查询用户信息, 该接口可用于在人脸信息特征提取成功准备注册到数据库时判断

数据库中是否包含该用户

```
// 7、根据姓名查询数据库与文件中对应的姓名是否相等，如果相等，则直接过滤
List<User> listUsers = FaceApi.getInstance().getUserListByUserName(userName);
if (listUsers != null && listUsers.size() > 0) {
    Log.i(TAG, "msg: " + "与之前图片名称相同");
    boolean isDelete = FaceApi.getInstance().userDeleteByName(userName);
    // (1) 删除成功
```

根据用户名查询该用户，
如果存在，则删除，在注册到数据库

根据用户 id 查询用户，demo 中将数据库 id 设置为自增的 int 值，以保证每个 id 之间不重复，使用该方法可以在人脸识别成功后，通过识别出的用户 id 在数据库中查询用户信息

```
List<? extends Feature> featureResult =
    faceModel.getFaceSearch().search(type, bdFaceCheckConfig.scoreThreshold, topNum: 1, feature );
// TODO 识别top_num = 1 全部数据集，此处可以任意设置，全部图片对比从小到大排序的num 全部数据集
if (featureResult != null && featureResult.size() > 0) {
    // 获取第一个数据
    Feature topFeature = featureResult.get(0);
    // 判断第一个数据是否大于设定阈值，如果大于，则返回成功
    thresholdScore = bdFaceCheckConfig.scoreThreshold;
    if (topFeature != null && topFeature.getScore() > thresholdScore) {
        // 当前featureEntity 只有id+feature 属性，在数据库中找到完整信息
        Log.e(TAG, "msg: " + topFeature.getScore());
        User user = FaceApi.getInstance().getUserListById(topFeature.getId());
        // (2) 通过用户id获取用户信息
```

识别接口

获取识别到的最相似的人脸

通过用户id获取用户信息

根据姓名在数据库删除用户，若产出成功识别前需要在 SDK 缓存中也删除该接口，或者从新获取一次数据库信息刷新一遍 SDK 缓存，具体识别接口可参考 [6.6 FaceSearch 识别对象](#)

```
Log.i(TAG, "msg: " + "与之前图片名称相同");
boolean isDelete = FaceApi.getInstance().userDeleteByName(userName);
if (!isDelete){
```

5.3.3 FaceSDKManager 类方法说明

方法名	所需参数	功能说明
onDetectCheck(BDFaceImageConfig bdFaceImageConfig, BDFaceImageConfig bdNirFaceImageConfig, BDFaceImageConfig bdDepthFaceImageConfig, BDFaceCheckConfig bdFaceCheckConfig, aceDetectCallBack faceDetectCallBack)	BDFaceImageConfig:RGB 视频流 参数 BDFaceImageConfig:NIR 视频流 参数 BDFaceImageConfig:DEPTH 视频 流参数 BDFaceCheckConfig:识别阈值 FaceDetectCallBack: 识别返回 结果	视频流 1:1 识别和 1:N 识 别的封装接口，可进行人 脸大小位置信息、质量检 测、活体检测、识别、识 别耗时的功能
setActiveLog	boolean:布尔类型	是否开启日志
initModel(Context context, BDFaceSDKConfig config, SdkInitListener listener)	Context:上下文 BDFaceSDKConfig:人脸检测配置 实体类	初始化模型
InitPush	Context:上下文	将人脸数据库中的人脸特 征注册到 sdk 内存

personDetect(Bitmap bitmap, byte[] feature, BDFaceCheckConfig bdFaceCheckConfig, Context context)	Bitmap:人脸图片 BDFaceCheckConfig:人脸检测和识别阈值 Context:上下文	提取图片人脸特征
--	---	----------

faceSDKManager 相关接口截图及说明如下:

模型初始化接口, 针对人脸快速检测、人脸跟踪、活体检测、特征提取、口罩识别、属性检测、注意力检测、等功能进行模型初始化操作 (只有初始化加载了模型、后续人脸检测等能力才能生效)

```
FaceSDKManager.getInstance().initModel(mContext, 模型初始化接口  
FaceUtils.getInstance().getBDFaceSDKConfig(), new SdkInitListener() {  
    2 usages
```

视频流逐帧检测接口, 进行 RGB 和 NIR 的人脸检测、活体检测、质量检测、特征提取、1:N 识别和 1:1 识别

```
FaceSDKManager.getInstance().personDetectCheck(bdFaceImageConfig, bdNirFaceImageConfig, bdDepthFaceImageConfig: null,  
    bdFaceCheckConfig, new FaceDetectCallBack() {  
    8 usages  
    @Override  
    public void onFaceDetectCallBack(List<LivePersonModel> livePersonModel) {  
    视频流检测接口
```

特征提取接口, 对图片进行人脸特征提取

```
final Bitmap bitmap = BitmapFactory.decodeStream(getContentResolver().openInputStream(uri1));  
if (bitmap == null) {  
    bdFaceCheckConfig.setSecondFeature(null);  
    return;  
}  
byte[] secondFeature = new byte[512]; 传入bitmap人脸图片, 返回人脸特征  
// 提取特征值  
float ret = FaceSDKManager.getInstance().personDetect(bitmap, secondFeature,  
    FaceUtils.getInstance().getBDFaceCheckConfig(), context: this);
```

initpush 接口, 主要作用为在识别前将人脸数据库中的信息同步到 SDK 缓存中(在缓存中比对识别速度更快), 在人脸识别时会将视频流中提取的人脸特征和 SDK 缓存中的特征进行比较, 并返回得分最大的人脸

```
/**  
 * 数据库发现变化时候, 重新把数据库中的人脸信息添加到缓存中, id+feature  
 */  
1 usage  
public void initPush(final Context context) {  
  
    if (future3 != null && !future3.isDone()) {  
        future3.cancel( mayInterruptIfRunning: true);  
    }  
  
    future3 = es3.submit(new Runnable() {  
        @Override  
        public void run() {  
            faceModel.getFaceSearch().featureClear(); 清空sdk缓存信息  
            synchronized (faceModel.getFaceSearch()) {  
                List<User> users = FaceApi.getInstance().getAllUserList();  
                for (int i = 0; i < users.size(); i++) {  
                    User user = users.get(i);  
                    faceModel.getFaceSearch().pushPersonById(user.getId(), user.getFeature());  
                }  
                if (FaceApi.getInstance().getUserNum() != 0) {  
                    ToastUtils.toast(context, (text: "人脸库加载成功");  
                }  
            }  
        }  
    });  
}
```

视频流人脸识别接口，可以识别摄像头中人脸信息，返回人脸坐标进行应用层快速人脸绘制跟踪，并对人脸进行质量检测、活体检测和识别

FaceSDKManager.java

```
44 public class FaceSDKManager {
243 /**
244  * 检测-活体-人脸-人脸检测流程
245  *
246  * @param bdFaceImageConfig 可能是YUV 数据流
247  * @param bdNirFaceImageConfig 红外YUV 数据流
248  * @param bdDepthFaceImageConfig 深度depth 数据流
249  * @param bdFaceCheckConfig 检测参数
250  * @param faceDetectCallback
251  */
252 public void onDetectCheck(
253     final BDFaceImageConfig bdFaceImageConfig,
254     final BDFaceImageConfig bdNirFaceImageConfig,
255     final BDFaceImageConfig bdDepthFaceImageConfig,
256     final BDFaceCheckConfig bdFaceCheckConfig,
257     final FaceDetectCallback faceDetectCallback) {
258     if (!FaceSDKManager.initModelSuccess) {
259         return;
260     }
261 }
```

FaceSDKManager.getInstance().onDetectCheck(bdFaceImageConfig, bdNirFaceImageConfig, bdDepthFaceImageConfig: null, bdFaceCheckConfig, new FaceDetectCallback() {

8 usages

```
@Override
public void onFaceDetectCallback(List<LivenessModel> livenessModel) {
    upload(livenessModel);
    if (developFragment.isSaveImage) {
        SaveImageManager.getInstance().saveImage(livenessModel, bdLiveConfig);
    }
}
```

26 usages

```
@Override
public void onTip(int code, String msg) {
}
```

4 usages

```
@Override
public void onFaceDetectDarwCallback(List<LivenessModel> models) {
    // 绘制人脸框
    showFrame(models);
}
```

}}

人脸识别结果，返回人脸坐标，质量检测接口，活体检测结果，识别结果，识别图片，总耗时等

主要作用为快速返回人脸坐标信息，进行人脸框绘制跟踪

5.3.4 CameraPreviewManager 类方法说明

方法名	所需参数	功能说明
startPreview(GIMantleSurfacView textureView, int videoDirection , int width, int height)	GIMantleSurfacView: 摄像头展示 view int(videoDirection): 摄像头角度 int(width):摄像头宽度 Int(height):摄像头高度	初始化摄像头
setCameraFacing(int cameraFacing)	Int:摄像头 id	设置摄像头 id，通常 0 为 RGB，1 为 NIR
InitCamera()	无	启动摄像头，并返回开启摄像头的宽高
setmCameraDataCallback(CameraDataCallback mCameraDataCallback)	CameraDataCallback: 摄像头逐帧数据监听器	监听摄像头每帧数据
StopPreview()	无	关闭摄像头

CameraPreviewManager 相关接口截图及说明如下：

初始化摄像头接口，设置展示画面，设置绘制人脸框镜像信息，设置摄像头角度，摄像头宽高等信息

```
glMantleSurfaceView = findViewById(R.id.camera_textureview);
glMantleSurfaceView.initSurface(SingleBaseConfig.getBaseConfig().getRgbRevert(),
    SingleBaseConfig.getBaseConfig().getMirrorVideoRGB(), SingleBaseConfig.getBaseConfig().isOpenGL());
CameraPreviewManager.getInstance().startPreview(glMantleSurfaceView,
    SingleBaseConfig.getBaseConfig().getRgbVideoDirection(), PREFER_WIDTH, PREFER_HEIGHT);
}
@Override
protected void onResume() {
    super.onResume();
    // 初始化摄像头参数，设置展示view，摄像头角度，摄像头宽高信息
}
```

初始化摄像头
展示view，设置
人脸库镜像信息

设置摄像头 id，部分摄像头为双目摄像头，包含 RGB 和 NIR 两款摄像头，通常 id 为 0 是 rgb，1 为 nir

```
private void startTestOpenDebugRegisterFunction() {
    CameraPreviewManager.getInstance().setCameraFacing(0);
}
```

设置摄像头id

开启摄像头，并且会返回摄像头的宽高（如果初始化摄像头时设置的宽高 摄像头不适配，会返回当前启动摄像头当前适配的宽高）

```
CameraPreviewManager.getInstance().setCameraFacing(0);

int[] cameraSize = CameraPreviewManager.getInstance().initCamera();
initFaceConfig(cameraSize[1], cameraSize[0]);
isPause = true;
```

开启摄像头

摄像头数据监听接口，通过该接口获取摄像头的每帧数据，通常为 nv21 类型，通过获取的摄像头每帧数据送到 SDK 进行人脸检测等工作

```
CameraPreviewManager.getInstance().setmCameraDataCallback(new CameraDataCallback() {
    2 usages
    @Override
    public void onGetCameraData(byte[] data, Camera camera, int width, int height) {
        onCameraData(data);
    }
});
```

摄像头关闭接口，在界面关闭时调用，清空摄像头缓存(退出 SDK 或不使用摄像头的时候注意关闭释放资源)

```
isPause = false;
CameraPreviewManager.getInstance().stopPreview();
```

5.3.5 PreviewTexture 类方法说明

方法名	所需参数	功能说明
-----	------	------

setCamera(Camera camera, int width, int height)	Camera:摄像头实例 int(width):摄像头宽度 Int(height):摄像头高度	开启摄像头
---	---	-------

PreviewTexture 相关接口截图及说明如下：

PreviewTexture 为摄像头展示 view，继承 ViewGroup，可以通过 setCamera 启动摄像头，并将摄像头信息展示到该 view 中

```
nirPreview = new PreviewTexture(context, this, nirView);  
  
if (SingleBaseConfig.getBaseConfig().getRGBCameraId() != -1) {  
    nirCamera = Camera.open(Math.abs(SingleBaseConfig.getBaseConfig().getRGBCameraId() - 1));  
} else {  
    nirCamera = Camera.open(cameraId: 1);  
}  
  
ViewGroup.LayoutParams layoutParams = nirView.getLayoutParams();  
int w = layoutParams.width;  
int h = layoutParams.height;  
int cameraRotation = SingleBaseConfig.getBaseConfig().getNirVideoDirection();  
nirCamera.setDisplayOrientation(cameraRotation);  
if (cameraRotation == 90 || cameraRotation == 270) {  
    layoutParams.height = w;  
    layoutParams.width = h;  
    // 旋转90度或者270，需要调整宽高  
} else {  
    layoutParams.height = h;  
    layoutParams.width = w;  
}  
nirView.setLayoutParams(layoutParams);  
nirPreview.w.setCamera(nirCamera, PREFER_WIDTH, PREFER_HEIGHT);  
initNirFaceConfig(PREFER_HEIGHT, PREFER_WIDTH);  
nirCamera.setPreviewCallback(new Camera.PreviewCallback() {  
    @Override
```

Annotations in the image:

- 创建PreviewTexture实例 (points to `nirPreview = new PreviewTexture(...)`)
- 创建摄像头实例 (points to `nirCamera = Camera.open(...)`)
- 启动摄像头，并展示到preview中 (points to `nirPreview.w.setCamera(...)`)

5.3.6 FaceModel 类方法说明

faceModel 类可以对识别和注册流程中的全部模型进行快速初始化，并获取对应线程(识别和注册通常为双线程，不能使用相同类同时检测)中所需的示例类

方法名	所需参数	功能说明
init(BDFaceSDKConfig config, Context context)	BDFaceSDKConfig :人脸检测 参数 context:上下文	模型快速初始化接口
setListener(SdkInitListener listener)	SdkInitListener :模型初 始化监听回调	模型初始化监听接口
getFacePersonFeature()	无	获取注册线程中的人脸特征 提取示例类

getFacePersonSearch()	无	获取注册线程中的识别示例类
getFaceDetectPerson()	无	获取注册线程中的人脸检测示例类
getFaceSearch()	无	获取人脸识别示例类
getFaceTrack()	无	获取快速检测人脸示例类
getFaceCrop()	无	获取人脸裁剪示例类
getFaceFeature()	无	获取人脸特征提取示例类
getFaceDetect()	无	获取人脸检测示例类
getFaceNirDetect()	无	获取 NIR 图像人脸检测示例类
getFaceLive()	无	获取活体检测示例类
getFaceMoutMask()	无	获取口罩检测示例类
getFaceSafetyHat()	无	获取安全帽检测示例类
getFaceGaze()	无	获取眼部注意力检测示例类

faceModel 相关接口截图及说明如下：

```

42 public class FaceModel implements SdkInitListener {
43
44     public void setListener(SdkInitListener listener) { this.listener = listener; }
45
46     private SdkInitListener listener;
47
48     public FaceModel() {
49         isModelInit = false;
50         cropBuilder = new CropBuilder(listener);
51         trackBuilder = new TrackBuilder(listener);
52         detectBuilder = new DetectBuilder(listener);
53         detectQualityBuilder = new DetectQualityBuilder(listener);
54         detectNirBuilder = new DetectNirBuilder(listener);
55         darkBuilder = new DarkBuilder(listener);
56         liveBuilder = new LiveBuilder(listener);
57         featureBuilder = new FeatureBuilder(listener);
58         featurePersonBuilder = new FeaturePersonBuilder(listener);
59         faceGazeBuilder = new FaceGazeBuilder(listener);
60         mouthMaskBuilder = new MouthMaskBuilder(listener);
61         faceSafetyHatBuilder = new FaceSafetyHatBuilder(listener);
62     }
63
64     public void init(BDFaceSDKConfig config, Context context) {
65         LogUtils.d(TAG, "msg: isModelInit = " + isModelInit);
66         if (!isModelInit) {
67             return;
68         }
69         BDFaceInstance trackInstance = new BDFaceInstance();
70         trackInstance.createInstance();
71         BDFaceInstance detectInstance = new BDFaceInstance();
72     }
73 }
  
```

设置模型初始化监听

调用模型初始化接口

5.4 工程数据库 db 文件路径自定义说明

dataLibrary 中默认会将注册的用户信息默认储存在 data/data/包名/database 数据库的 face.db 默认路径下，可在 DBHelper 文件夹中放开这部分代码，即可修改数据库文件存储路径

5.5 在线特征提取接口调用方式

5.5.1 接口调用

百度 AI 官网提供了可从云端进行人脸特征值提取的接口功能（通过 http 调用传入图片 base64，返回人脸特征值），可以实现人脸特征抽取和人脸库的构建，主要用于在服务端（云端）提取与人脸离线识别 SDK 通用的人脸特征值，通过该接口获取的特征值可直接注册到设备中进行人脸 1:1 或 1:N 识别，接口调用文档如下：<https://ai.baidu.com/ai-doc/FACE/0kg7edktq>

此外，demo 工程中添加了该接口的调用方式，代码可以参考 dataLibrary 模块中的 OnLineFeatureManager 类，位置如下：

注: 该接口需输入对应 SDK 版本参数 (该参数是为了对齐 SDK 的版本和云端提取版本的特征值通用、只有版本对齐了, SDK 的特征值和云端提取的特征值才能通用), 目前常用参数为:

如: 8.0(8.0 到 9.0 范围, 比如 8.1、8.2、8.5) 的 sdk 对齐云端: Android_8001

3568 3.1 专版 sdk: RV1109_3000



5.5.2 特征字符串转换

在线特征提取接口会返回 base64 位的人脸特征字符串, 可使用 `Base64.decode()` 方法转换为 byte 数组, 截图如下:

```
base64String = Uri.decode(base64String);  
byte[] decode = Base64.decode(base64String, Base64.DEFAULT);
```

5.6 摄像头模组集成方法说明

此章节为应用层的摄像头模组说明，会对不同摄像头进行简单的描述和调用方式，其中摄像头通常分为三类，类型如下、实际场景中，可根据自身需要选择使用那种摄像头配置：

摄像头类型	摄像头功能说明
RGB	标准摄像头，通过红、绿、蓝三原色通道捕获彩色图像的摄像头，安卓手机中自带摄像头准为 RGB 摄像头 同时也是必不可少的摄像头，在项目中人脸检测、质量检测、1:1 和 1:N 识别都需要使用该摄像头
RGB+NIR	RGB:同上 NIR:近红外摄像头，主要用于辅助活体检测，更好的防御在活体检测过程中电子产品产生的人脸攻破问题
RGB+DEPTH	RGB:同上 DEPTH:3D 结构光深度摄像头，主要用于辅助活体检测，更好的防御在活体检测成 3D 头模产生的人脸攻破问题

5.6.1 RGB 摄像头模组集成说明

RGB 摄像头为标准摄像头，Android 手机自带摄像头均可理解为 RGB 摄像头，使用 Android 原生自带的 Camera 类进行界面展示和视频流提取，项目工程可参照官网下载的工程包中的 FaceSDKAndroid 进行集成，工程文件可参照:3.2 节 官网工程包结构说明，并且项目中对 Camera 进行了封装，具体可参照:4.3.5 节 CameraPreviewManager 类方法说明

```
1 override
protected void initView() {
    glMantleSurfacView = findViewById(R.id.camera_textureview);
    glMantleSurfacView.initSurface(SingleBaseConfig.getBaseConfig().getRgbRevert(),
        SingleBaseConfig.getBaseConfig().getMirrorVideoRGB(), SingleBaseConfig.getBaseConfig().isOpenGL());
    CameraPreviewManager.getInstance().startPreview(glMantleSurfacView,
        SingleBaseConfig.getBaseConfig().getRgbVideoDirection(), PREFER_WIDTH, PREFER_HEIGHT);
}

@Override
protected void onResume() {
    super.onResume();
    startTestOpenDebugRegisterFunction();
}

1 usage
private void startTestOpenDebugRegisterFunction() {
    CameraPreviewManager.getInstance().setCameraFacing(0);
    int[] cameraSize = CameraPreviewManager.getInstance().initCamera();
    initFaceConfig(cameraSize[1], cameraSize[0]);
    isPause = true;

    CameraPreviewManager.getInstance().setmCameraDataCallback(new CameraDataCallback() {
        2 usages
    });
}
```

onCreate中初始化摄像头

设置摄像头id，通常rgb为0

开启摄像头

设置摄像头监听器，获取摄像头信息

5.6.2 RGB+NIR 双目摄像头模组集成说明

RGB 和 NIR 摄像头均可以使用 android 原生自带的 Camera 类进行界面展示和视频流提取，可以理解为这两种摄像头启动方式完全相同，唯一不同在一启动时传入 id 不同(id 通常由系统决定，未插入其他 usb 设备情况下，通常为 0 是 RGB、1 是 NIR)。项目工程可参照官网下载的工程包中的 FaceSDKAndroid 进行集成，工程文件可参照:3.2 官网工程包结构说明，集成说明如下：

RGB 摄像头：同上，可参照 5.6.1 节 RGB 摄像头模组集成说明

NIR 摄像头：demo 中使用 PreviewTexture 对 NIR 摄像头进行了封装(也可以开启 RGB 摄像头)，具体可参考:[5.3.5 PreviewTexture 类方法说明](#)

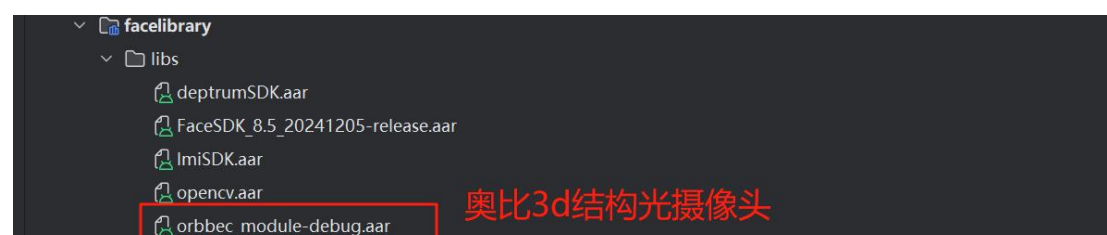


5.6.3 RGB+DEPTH 摄像头模组集成说明

RGB+DEPTH 的 3D 结构光摄像头会根据不同厂家生产的摄像头不同，调用方式各不相同，需要取对应厂家的官网下载摄像头启动的 SDK 集成到项目中，使用厂家规定的启动开启才行，目前官网项目种仅提供了奥比和华捷两款 3D 结构光的启动方式，项目工程可参照官网下载的工程包中的 FaceSDKAndroid_depth 项目进行集成，工程文件可参照:3.2 官网工程包结构说明,集成说明如下：

5.6.3.1 奥比摄像头

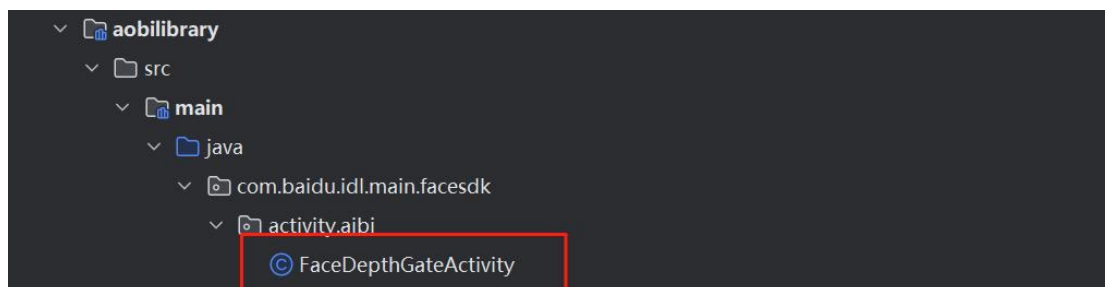
奥比摄像头 SDK 位于工程文件 FaceLibrary/libs/orbbec_module-debug.aar 下，将该 SDK 集成到项目可以调取奥比 3D 结构光摄像头接口



项目代码位于

aobilibrary\src\main\java\com\baidu\idl\main\facesdk\activity\ai\bi\FaceDepthGateActivity.java

中，提供了 RGB 和 DEPTH 的效果展示



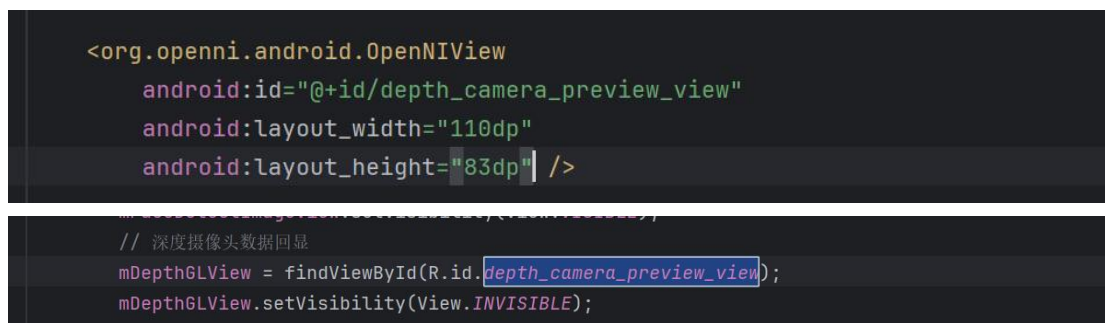
RGB 摄像头:

奥比 RGB 摄像头可以使用原生自带 camera 方式启动，可参照 5.5.1 RGB 摄像头模组集成说明

DEPTH 摄像头:

奥比 3D 结构光摄像头需要用到奥比 SDK 中的 OpenNIHelper(摄像头启动类) VideoStream(摄像头数据拉取类)和 OpenNIView(数据展示类)

首先先在 ui 中创建摄像头展示 view，创建深度摄像头回显

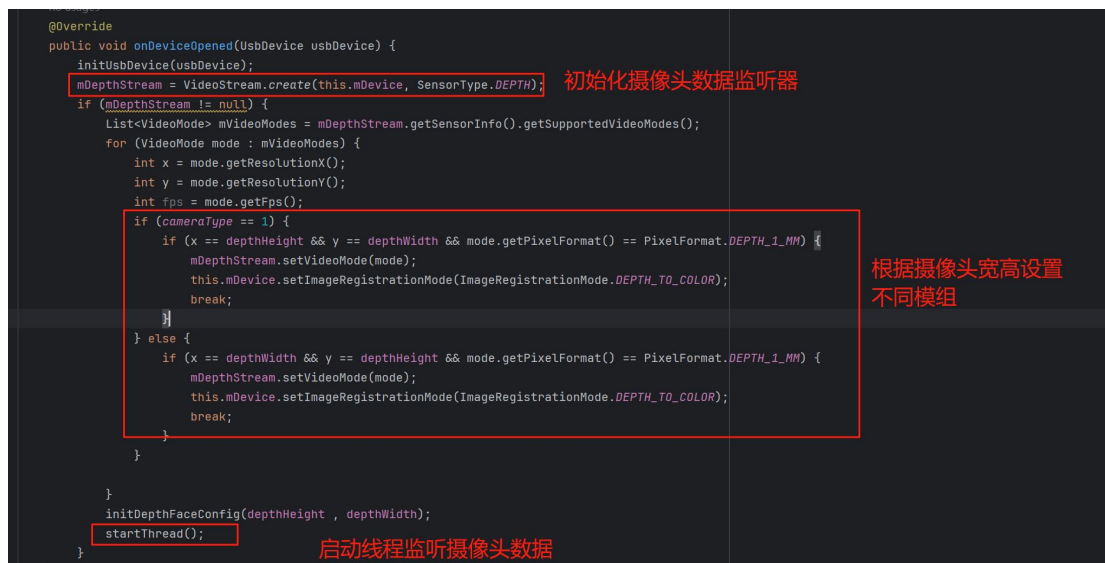


创建完成后，项目中创建 OpenNIHelper 摄像头启动类，并调用 requestDeviceOpen 方法开始，监听摄像头状态，



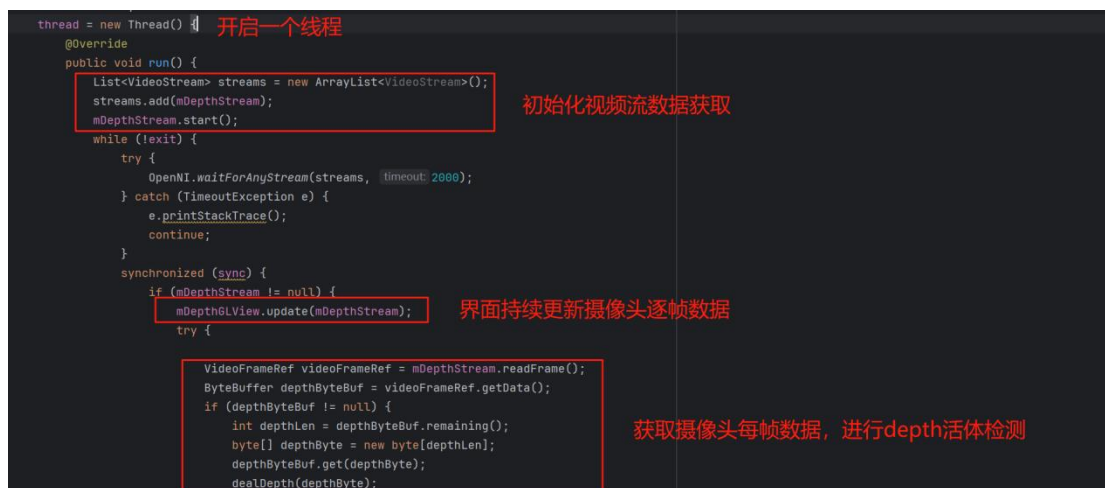
在 requestDeviceOpen 方法启动摄像头后，会拉起 onDeviceOpened，表示摄像头已开启，失败则会拉起 onDeviceOpenFailed(摄像头已经启动)或 onDeviceNotFound(没有找到深度摄像头)

onDeviceOpened 拉起成功后初始化 VideoStream 摄像头数据拉取类，且由于奥比摄像头种类不同，摄像头宽高比也不同，这里需要使用宽高判断摄像头模组，截图和奥比模组种类如下：

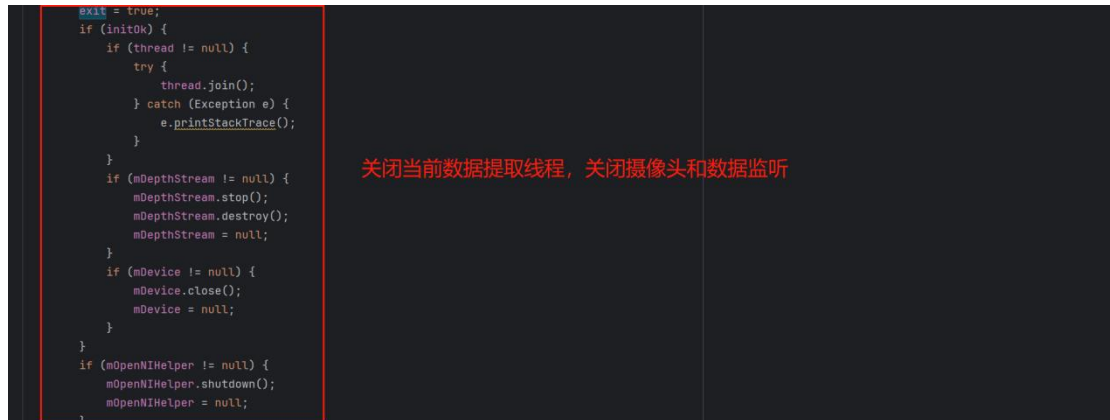


奥比摄像头类型	宽高比
奥比中光海燕、大白	640*400
奥比中光海燕 Pro、Atlas	400*640
奥比中光蝴蝶、Astra Pro\Pro S	640*480

模组宽高设置后，开启一个独立线程，使用 VideoStream 进行摄像头逐帧数据的拉取，并展示到 OpenNIView 中，截图如下：

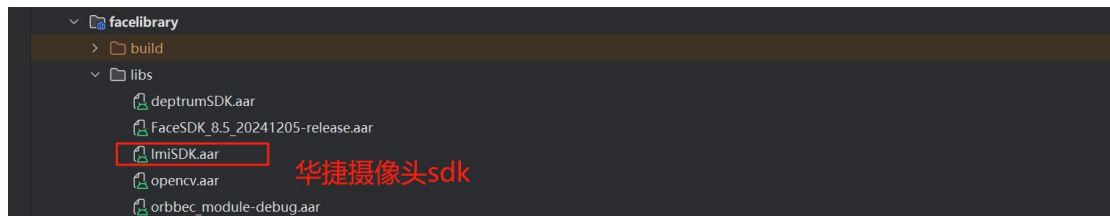


最后，退出界面时 (onDestroy , onPause) 需关闭摄像头，关闭当前监听的线程，确保后续功能正确，截图如下



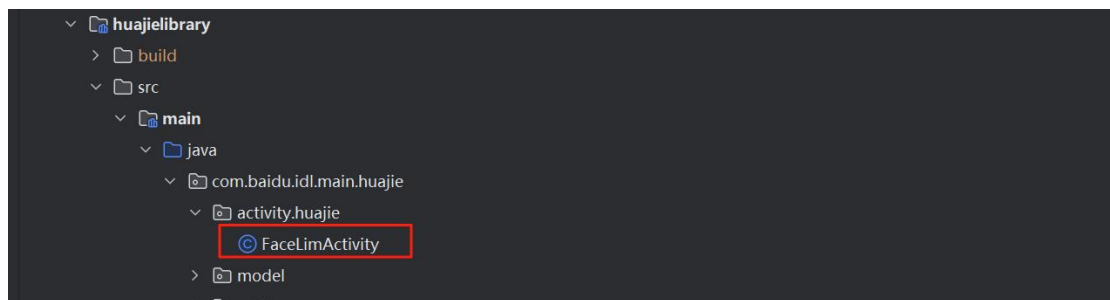
5.6.3.2 华捷摄像头

华捷摄像头 SDK 位于工程文件 FaceLibrary/libs/orbbec_module-debug.aar 下，将该 SDK 集成到项目可以调取奥比 3D 结构光摄像头接口



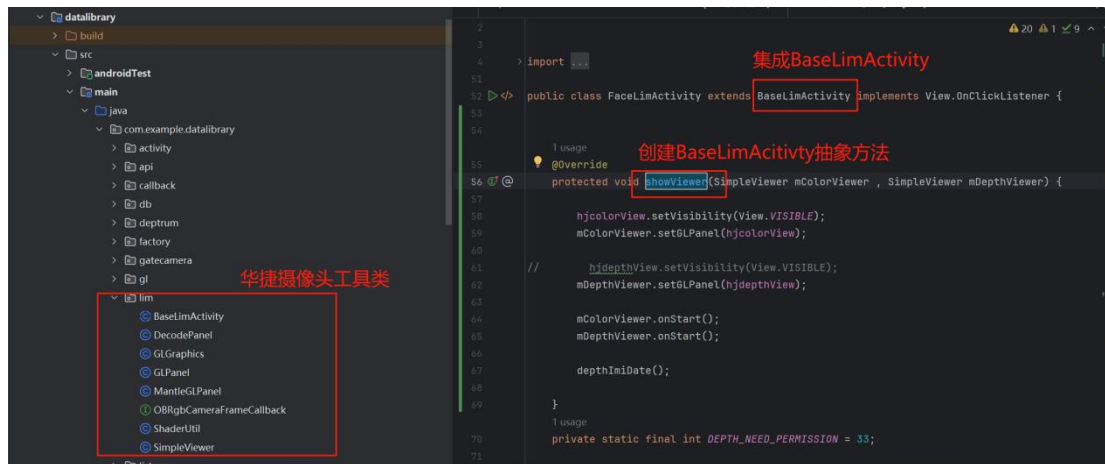
项目代码位于

huajielibrary\src\main\java\com\baidu\idl\main\huajie\activity\huajie\FaceLimActivity.java 中，提供了 RGB 和 DEPTH 的效果展示，截图如下：



并且，需要将 datalibrary\src\main\java\com\example\datalibrary\lim 华捷摄像头工具类

导入到项目中，集成 BaseLimActivity，创建 showViewer 抽象方法 (showViewer 会在摄像头启动后被调用)，具体截图如下

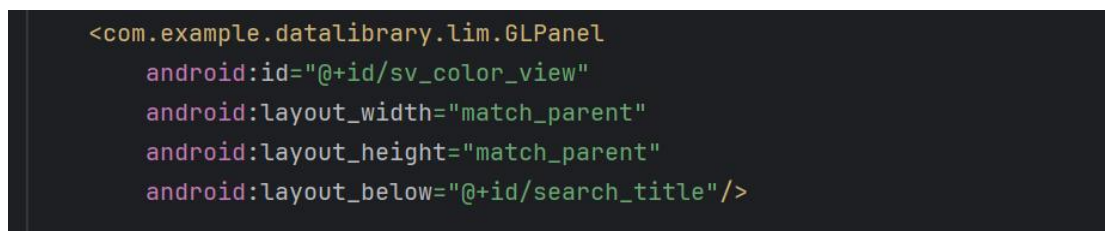


集成 BaseLimActivity 后调用 BaseLimActivity 的 openLim 方法，可以打开 RGB 和 DEPTH 摄像头，启动成功后会调用 showViewer 的方法，并返回 RGB 和 DEPTH 数据



RGB 摄像头:

展示华捷 RGB 摄像头需要用到 GLPanel 摄像头展示 view，在 xml 中创建该 view 实例, 在 activity 中获取彩色图像预览对象，截图如下：

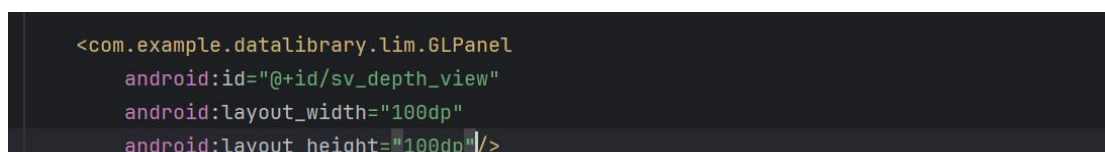


创建 GLPanel 彩色摄像头对象后，在 showViewer 抽象方法中将 GLPanel 赋值给 RGB 展示对象即可完成摄像头效果展示，截图如下：



DEPTH 摄像头:

展示华捷 DEPTH 摄像头需要用到 GLPanel 摄像头展示 view，在 xml 中创建该 view 实例, 在 activity 中获取彩色图像预览对象，截图如下：



```
// 华捷深度图像预览
```

```
hjdepthView = findViewById(R.id.sv_depth_view);
```

创建 GLPanel 深度摄像头对象后，在 showViewer 抽象方法中将 GLPanel 赋值给深度展示对象即可完成摄像头效果展示，截图如下：

```
@Override
protected void showViewer(SimpleViewer mColorViewer , SimpleViewer mDepthViewer) {

    hjcolorView.setVisibility(View.VISIBLE);
    mColorViewer.setGLPanel(hjcolorView);

    //    hjdepthView.setVisibility(View.VISIBLE);
    mDepthViewer.setGLPanel(hjdepthView);

    mColorViewer.onStart();
    mDepthViewer.onStart();

    depthImiDate();
}
```

设置深度摄像头显示图像

显示rgb和深度摄像头

RGB 和 DEPTH 摄像头数据获取：

showViewer 方法被调用可以开启独立线程，获取 RGB 和 DEPTH 视频流数据，使用华捷 SDK 的 ImiDevice.ImiFrame 类获取 RGB 和 DEPTH 的每帧数据，并对每帧数据进行过滤，最终获取正确的视频流数组，具体截图如下

```
@Override
protected void showViewer(SimpleViewer mColorViewer , SimpleViewer mDepthViewer) {

    hjcolorView.setVisibility(View.VISIBLE);
    mColorViewer.setGLPanel(hjcolorView);

    //    hjdepthView.setVisibility(View.VISIBLE);
    mDepthViewer.setGLPanel(hjdepthView);

    mColorViewer.onStart();
    mDepthViewer.onStart();

    depthImiDate();
}
```

摄像头启动成功，开始拉取rgb和depth数据

```
...
private void depthImiDate() {
    new Thread(new Runnable() {
        @Override
        public void run() {
            ImiDevice.ImiFrame colorFrame = null;
            ImiDevice.ImiFrame depthFrame = null;
            while (!mIsExitLoop) {
                try {
                    // 获取rgb数据帧
                    if (colorFrame == null) {
                        if (huajiemDevice != null && !mDestroyFlag && !huajiemDevice.isOpenning()) {
                            colorFrame = huajiemDevice.readNextFrame(ImiDevice.ImiStreamType.COLOR, timeout: 100);
                        }
                    }
                    // 获取depth数据帧
                    if (null == depthFrame) {
                        if (huajiemDevice != null && !mDestroyFlag && !huajiemDevice.isOpenning()) {
                            depthFrame = huajiemDevice.readNextFrame(ImiDevice.ImiStreamType.DEPTH, timeout: 100);
                        }
                    }
                    // color and depth sync. find the right delta value
                    if (colorFrame != null && depthFrame != null) {
                        long ut = colorFrame.getTimestamp() / 1000;
                        long dt = depthFrame.getTimestamp() / 1000;
                        long delta = (dt - ut);
                        if (delta < -25) {
                            depthFrame = null;
                        } else if (delta > 25) {
                            colorFrame = null;
                        }
                    }
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        }
    }).start();
}
```

创建rgb和depth摄像头实例

获取每帧数据

数据过滤

```

334         if (null == depthFrame || null == colorFrame) {
335             continue;
336         }
337
338         ByteBuffer mColorBuffer = colorFrame.getData();
339         ByteBuffer mDepthBuffer = depthFrame.getData();
340
341         if (mColorBuffer != null && mDepthBuffer != null) {
342             // 将rgb、depth ByteBuffer 转换为byte[]
343             byte[] rgbData = decodeValue(mColorBuffer);
344             dealRgb(rgbData);
345             byte[] depthData = decodeValue(mDepthBuffer);
346             dealDepth(depthData);
347         }

```

数据过滤成功后，获取视频流每帧数据，并转换为byte数组

5.7 多人脸识别说明

SDK 支持多个人脸进行人脸面部 ui 绘制和识别，可调整 SDK 的 maxDetectNum 参数调整人脸识别数量(建议单次识别 10 个人脸以内)，并在 FaceSDKManager 类中开启多人脸识别测试识别效果，具体截图如下：

```

public BDFaceSDKConfig getBDFaceSDKConfig() {
    BDFaceSDKConfig config = new BDFaceSDKConfig();
    // TODO: 最小人脸个数检查，默认设置为1, 用户根据自己需求调整
    config.maxDetectNum = 2;
    // config.trackInterval = Integer.MAX_VALUE; 多人脸识别情况下需注释该参数

    // TODO: 默认为80px。可传入大于30px的数值，小于此大小的人脸不予检测，生效时间第一次加载模型
}

```

人脸最多识别数量

Project

- manager
 - FaceModel
 - FaceSDKManager
 - IdentifyResult
 - OnLineFeatureManager
 - SaveImageManager
- model
- utils
- view
- res
- AndroidManifest.xml

```

44 public class FaceSDKManager {
45
46     41 usages
47     private FaceModel faceModel;
48
49     5 usages
50     private boolean checkMouthMask = false;
51
52     2 usages
53     private boolean isMultiIdentify = true;
54
55     1 usage
56     private FaceSDKManager() {
57         faceAuth = new FaceAuth();
58     }
59 }

```

设置开启多人脸识别

src

- main
 - java
 - com.baiduidl.main.facesdk
 - activity.gate
 - FaceOneToNActivity
 - FacePageActivity
 - FaceUIActivity
 - fragment
 - utils
 - FaceUtils
 - res
 - AndroidManifest.xml

```

19 public class FaceOneToNActivity extends FacePageActivity {
20
21     58
22     @Override
23     void onCameraData(byte[] data) {
24         bdFaceImageConfig.setData(data);
25         FaceSDKManager.getInstance().onDetectCheck(bdFaceImageConfig, bdNirFaceImageConfig, bdDepthFaceImageConfig, nu
26         bdFaceCheckConfig, new FaceDetectCallBack() {
27             8 usages
28             @Override
29             public void onFaceDetectCallBack(List<LivenessModel> livenessModel) {
30                 upload(livenessModel);
31                 if (developFragment.isSaveImage) {
32                     SaveImageManager.getInstance().saveImage(livenessModel, bdLiveConfig);
33                 }
34             }
35         }
36     }
37 }

```

list中每个值为一个人脸的识别结果

5.8 Log 日志说明

项目中对激活、模型初始化、识别过程进行了日志打印，可通过 LogUtils 类设置是否打印日志，且可通过 FaceModel(模型初始化)、FaceSDKManager(识别)、BdFaceAuth(激活) 文字过滤日志，截图如下：



6 SDK 功能接口说明

6.1 FaceAuth 鉴权接口

构造方法	参数	类型	说明
FaceAuth()	无		创建一个 FaceAuth

6.1.1 在线授权

方法名	initLicenseOnLine			
说明	用户通过申请授权码，在线授权，激活设备			
方法	initLicenseOnLine(final Context context, final String licenseID, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
licenseKey	官网授权码	是	String	
callback	鉴权结果 0 为成功, 错误码可参考: 9.1 在线 和 离线初始化错误码说明	是	Callback	

6.1.2 离线授权

方法名	initLicenseOffLine
说明	用户申请鉴权文件，放在 SD 卡下，点击按钮直接鉴权
方法	initLicenseOffLine(final Context context, final Callback callback)

请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
callback	鉴权结果 0 为成功, 错误码可参考: 9.1 在线 和 离线初始化错误码说明	是	Callback	

6.1.3 批量授权

方法名	initLicenseOnLine			
说明	用户通过申请在线批量 licenseID 进行激活, 可直接网络请求获取鉴权文件			
方法	initLicenseBatchLine(final Context context, final String licenseKey, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
licenseKey	官网在线批量授权码	是	String	
callback	鉴权结果 0 为成功, 错误码可参考: 9.1 在线 和 离线初始化错误码说明	是	Callback	

6.1.4 底层 log 日志接口

方法名	setActiveLog			
说明	用于 Debug 时候输出 Log 详细信息			
方法	setActiveLog(BDFaceSDKCommon.BDFaceLogInfo logInfo, int isLog)			
请求参数	说明	必须	类型	返回参数
logInfo	底层 log 打印 BDFACE_LOG_ERROR_MESSAGE // 打印输出错误日志 BDFACE_LOG_VALUE_MESSAGE // 打印输出值日志 BDFACE_LOG_TYPE_PERF // 打印性能日志 BDFACE_LOG_TYPE_ALL // 打印全部日志 BDFACE_LOG_TYPE_DEBUG // 打印 debug 日志	是	BDFaceLogInfo	无
isLog	0: 不输出日志; 1: 输出日志	是	int	

6.1.5 设置 cpu 核数

方法名	setCoreConfigure			
说明	根据开发板类型，设置加速对 CPU 核数依赖，调整参数，提高性能			
方法	setCoreConfigure(BDFaceSDKCommon.BDFaceCoreRunMode runMode, int coreNum)			
请求参数	说明	必须	类型	返回参数
runMode	推荐使用 0， 1， 3；如果有需要绑核的话，用 0 和 1；如果不需要绑核，系统自动调度的话，用 3 BDFACE_LITE_POWER_HIGH=0， 绑定大核运行模式。如果 ARM CPU 支持 big.LITTLE，则优先使用并绑定 Big cluster。如果设置的线程数大于大核数量，则会将线程数自动缩放到大核数量。如果系统不存在大核或者在一些手机的低电量情况下会出现绑核失败，如果失败则进入不绑核模式。 BDFACE_LITE_POWER_LOW=1， 绑定小核运行模式。如果 ARM CPU 支持 big.LITTLE，则优先使用并绑定 Little cluster。如果设置的线程数大于小核数量，则会将线程数自动缩放到小核数量。如果找不到小核，则自动进入不绑核模式。 BDFACE_LITE_POWER_FULL=2， 大小核混用模式。线程数可以大于大核数量。当线程数大于核心数量时，则会自动将线程数缩放到核心数量。 BDFACE_LITE_POWER_NO_BIND=3， 不绑核运行模式（推荐）。系统根据负载自动调度任务到空闲的 CPU 核心上。 BDFACE_LITE_POWER_RAND_HIGH=4， 轮流绑定大核模式。如果 Big cluster 有多个核心，则每预测 10 次后切换绑定到下一个核心。 BDFACE_LITE_POWER_RAND_LOW=5， 轮流绑定小核模式。如果 Little cluster 有多个核心，则每预测 10 次后切换绑定到下一个核心。	是	BDFaceCoreRunMode	无
coreNum	cpu 核数，选择线程数，建议不	是	int	

	超过 2			
--	------	--	--	--

6.1.6 获取 deviceId 设备指纹

方法名	getDeviceId			
说明	用于官网离线激活生成授权文件			
方法	getDeviceId(Context context)			
请求参数	说明	必须	类型	返回参数
Context	上下文	是	Context	String 指纹信息

6.2 BdFaceInstance 线程实例

当人脸检测、特征提取需要多个线程使用时，需要用到该对象，需要在每个不同线程中分别创建 人脸检测和特征提取对象，每个线程的检测和提取对象在创建时传入该实例，以保证不同线程中的识别稳定性

构造方法	参数	类型	说明
BdFaceInstance()	无		创建一个 BdFaceInstance

6.2.1 开启 BdFaceInstance 实例

方法名	creatInstance			
说明	开启一个 BdFaceInstance 实例			
方法	creatInstance()			
请求参数	说明	必须	类型	返回参数
无	初始化该实例	是	无	无

6.3 FaceDetect 检测对象

该对象主要用于人脸的快速检测，人脸跟踪，人脸检测，质量检测，模糊检测等功能

构造方法

构造方法	参数	类型	说明
------	----	----	----

FaceDetect()	无		无参构造调用（执行默认创建的 instance）
FaceDetect(BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的 instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.2 BDFaceInstance 线程实例

6.3.1 模型加载接口

方法名	initModel			
说明	模型加载如检测、对齐等模型，支持可见光、近红外,快速检测模型			
方法	initModel(final Context context, final String detectModel, final String alignModel, final BDFaceSDKCommon.DetectType detectType, final BDFaceSDKCommon.AlignType alignType, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
detectModel	检测模型路径	是	String	
alignModel	对齐模型路径	是	String	
detectType	检测类型 DETECT_VIS 为可见光 DETECT_NIR 为近红外	是	DetectType	
alignType	对齐类型 BDFACE_ALIGN_TYPE_RGB_ACCURATE 为可见光对齐类型; BDFACE_ALIGN_TYPE_RGB_FAST 为可见光快速对齐类型; BDFACE_ALIGN_TYPE_NIR_ACCURATE 为近红外对齐类型	是	AlignType	
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.3.2 质量检测模型加载接口

方法名	initQuality
说明	质量检测模型加载，判断人脸遮挡信息，光照信息，模糊信息，模型包含模糊模型，遮挡信息，作用于质量检测接口
方法	initQuality(final Context context, final String blurModel,

	final String occlurModel, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
blurModel	模糊检测模型路径	是	String	
occlurModel	遮挡检测模型路径	是	String	
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.3.3 最优人脸模型加载接口

方法名	initBestImage			
说明	最优人脸模型初始化接口，可对图像质量进行筛选			
方法	initBestImage(final Context context, final String bestModel, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
bestModel	最优人脸模型路径	是	String	
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.3.4 配置信息加载接口

方法名	loadConfig			
说明	检测最小人脸，是否开启内部质量检测，检测或者追踪时间间隔等配置			
请求参数	loadConfig(BDFaceSDKConfig config)			
请求参数	说明	必须	类型	返回参数
config	人脸信息配置实体类	是	BDFaceSDKCon	无

6.3.5 人脸检测接口

6.3.5.1 根据图片和 loadConfig 配置进行人脸检测

方法名	detect			
说明	人脸检测接口，可获取图像中的人脸宽高，中心点位置和人脸关键点信息，根据 6.3.4 loadConfig 中配置，可获取头部姿态角，光照，模糊，属性，模糊等信息			

请求参数	detect (BDFaceSDKCommon.DetectType type, BDFaceImageInstance imageInstance)			
请求参数	说明	必须	类型	返回参数
detectType	检测类型 DETECT_VIS 为可见光 DETECT_NIR 为近红外	是	DetectType	FaceInfo[] : 人脸参数信息
imageInstance	图片数据信息, 可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	

6.3.5.2 根据传入的人脸框信息进行检测

方法名	detect			
说明	可灵活配置的人脸检测接口，使用输入的 faceInfos 通过配置 bdFaceDetectListConfig 可以控制是否进行以下能力的预测：人脸检测，关键点提取，头部姿态角，光照，模糊，属性，情绪，闭眼，闭嘴。输出返回值为预测后的 faceInfos，可作用于后续活体，特征抽取入参。（faceInfos[] 可配置为空，即为第一帧检测）			
方法	detect (BDFaceSDKCommon.DetectType detectType, BDFaceSDKCommon.AlignType alignType, BDFaceImageInstance imageInstance, FaceInfo[] faceInfos, BDFaceDetectListConfig bdFaceDetectListConfig)			
请求参数	说明	必须	类型	返回参数
detectType	检测类型 DETECT_VIS 为可见光 DETECT_NIR 为近红外	是	DetectType	FaceInfo[] : 人脸参数信息
alignType	对齐类型 BDFACE_ALIGN_TYPE_RGB_ACCURATE 为可见光对齐类型； BDFACE_ALIGN_TYPE_RGB_FAST 为可见光快速对齐类型； BDFACE_ALIGN_TYPE_NIR_ACCURATE 为 近红外对齐类型	是	AlignType	
imageInstance	图片数据信息, 可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	
faceInfos[]	人脸框数据 (可以通过人脸追踪能力获取人脸框)	否	FaceInfos[]	
bdFaceDetectListConfig	功能配置	是	BdFaceDetectListConfig	

6.3.6 人脸跟踪接口

方法名	track			
说明	视频人脸跟踪检测，追踪图片中多个人脸信息，通过参数 num 配置，接口包含检测和跟踪功能，返回基本人脸信息和 72 关键点，可以绘制人脸框，描绘眼耳鼻嘴关键点，也可作用于后续活体，特征抽取入参			
方法	track(BDFaceSDKCommon.DetectType detectType, BDFaceSDKCommon.AlignType alignType, BDFaceImageInstance imageInstance)			
请求参数	说明	必须	类型	返回参数
detectType	检测类型 DETECT_VIS 为可见光 DETECT_NIR 为近红外	是	DetectType	FaceInfo[] : 人脸参数信息
alignType	对齐类型 BDFACE_ALIGN_TYPE_RGB_ ACCURATE 为可见光对齐类型; BDFACE_ALIGN_TYPE_RGB_ FAST 为可见光快速对齐类型; BDFACE_ALIGN_TYPE_NIR_ ACCURATE 为 近红外对齐类型	是	AlignType	
imageInstance	图片数据信息,可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImage Instance	

6.3.7 检测模型卸载接口

方法名	uninitModel			
说明	卸载模型			
方法	uninitModel()			
请求参数	说明	必须	类型	返回参数
无	无	是	无	Int

6.4 FaceLive 活体检测对象

该对象主要用于对卡通人物、动物、2D 纸张、电子产品、3D 头模的防御，需要与 faceDetect 人脸检测对象同步使用

构造方法	参数	类型	说明
FaceLive()	无		无参构造调用（执行默认创建的 instance）
FaceLive(BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的 instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.7 BDFaceImageInstance 图片对象

6.4.1 活体模型加载接口

方法名	initModel			
说明	静默活体检测模型初始化，可见光活体模型，深度活体，近红外活体模型初始化			
方法	initModel(final Context context, final String visModel, final String vis2dmaskModel, final String visHandModel, final String visReflectionModel, final String nirModel, final String depthModel, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
visModel	可见光图片活体模型	是	String	
vis2dmaskModel	优化 RGB 活体攻破模型，可以为空	否	String	
visHandModel	优化 RGB 活体攻破模型，可以为空	否	String	
visReflectionModel	优化 RGB 活体攻破模型，可以为空	否	String	
nirModel	红外图片活体模型	否	String	
depthModel	深度图片活体模型	否	String	
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.4.2 人脸静默活体检测接口

方法名	silentLive
-----	------------

说明	静默活体分值检测，返回 0-1 结果，demo 默认值超过 0.8 为活体			
方法	silentLive(BDFaceSDKCommon.LiveType type, BDFaceImageInstance bdFaceImageInstance, float[] landmarks, float liveThreshold)			
请求参数	说明	必须	类型	返回参数
type	BDFACE_SILENT_LIVE_TYPE_RGB 可见光图像静默活体检测 BDFACE_SILENT_LIVE_TYPE_NIR 红外图像静默活体检测 BDFACE_SILENT_LIVE_TYPE_DEPTH 深度图像静默活体检测	是	LiveType	float
imageInstance	图片数据信息，可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	
landmarks	FaceDetect 接口 detect 或 track 方法检测到的 FaceInfo 中的人脸关键点信息	是	float[]	
liveThreshold	RGB 活体攻破模型开启的话需要添加，设置活体攻破模型检测阈值	否	float	

6.4.3 活体模型卸载接口

方法名	uninitModel			
说明	卸载模型			
方法	uninitModel()			
请求参数	说明	必须	类型	返回参数
无	无	是	无	Int

6.5 FaceFeature 特征提取对象

该对象主要用于对人脸信息的特征抽取，需要与 faceDetect 人脸检测对象同步使用

构造方法	参数	类型	说明
FaceFeature()	无		无参构造调用（执行默认创建的 instance）
FaceFeature(BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的 instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.7 BDFaceImageInstance 图片对象

6.5.1 特征提取模型加载接口

方法名	initModel			
说明	离线特征获取模型加载，目前支持可见光模型，近红外检测模型（非必要参数，可以为空），证件照模型；用户根据自己场景，选择相应场景模型			
方法	initModel(final Context context, final String idPhotoModel, final String visModel, final String nirModel, final String rgbModel, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
idPhotoModel	证件照图片模型	是	String	
visModel	可见光图片模型	是	String	
nirModel	红外图片模型	否	String	
rgbModel	RGBD 图片模型	否	String	
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.5.2 特征提取接口

方法名	feature			
说明	支持 RGB 证件照、RGB 生活照、NIR 图片的识别，仅对一张图片进行特征提取			
方法	feature(BDFaceSDKCommon.FeatureType featureType, BDFaceImageInstance imageInstance, float[] landmarks, byte[] feature)			
请求参数	说明	必须	类型	返回参数
featureType	BDFAACE_FEATURE_TYPE_LIVE_PHOTO 生活照 BDFAACE_FEATURE_TYPE_ID_PHOTO 证件照 BDFAACE_FEATURE_TYPE_NIR 红外	是	FeatureType	float 128 为提取成功
imageInstance	图片数据信息，可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	
landmarks	FaceDetect 接口 detect 或 track FaceInfo 中的人脸关键点信息	是	float[]	
feature	人脸特征 feature 数组，默认初始化 512 空字节 (3568 专版 SDK 为 1024 字节)	是	Byte[]	

6.5.3 特征提取卸载接口

方法名	uninitModel			
说明	卸载模型			
方法	uninitModel()			
请求参数	说明	必须	类型	返回参数
无	无	是	无	Int

6.6 FaceSearch 识别对象

该对象主要用于对人脸特征的注册、1:1 识别、1:N 识别，需要与 faceFeature 人脸特征提取功能同步使用

构造方法	参数	类型	说明
FaceSearch()	无		无参构造调用（执行默认创建的 instance）
FaceSearch(BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的 instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.2 BDFaceInstance 线程实例

6.6.1 SDK 注册接口

该接口可以将数据库存储的人脸特征信息注册到 SDK 内存中，通过识别接口返回注册到的最相似的人员信息和识别得分

6.6.1.1 单个特征注册接口

方法名	pushPersonById			
说明	单张特征注册			
方法	pushPersonById(int pointID, byte[] feature)			
请求参数	说明	必须	类型	返回参数
pointID	人员 id，不可重复	是	Int	Int

feature	人脸特征 feature 数组，默认初始化 512 空字节 (3568 专版 SDK 为 1024 字节)	是	Byte[]	0 为成功
---------	---	---	--------	-------

6.6.1.2 批量特征注册接口

方法名	pushPersonFeatureList			
说明	多张特征注册			
方法	pushPersonFeatureList(List<? extends Feature> features)			
请求参数	说明	必须	类型	返回参数
features	多张人脸特征信息	是	List<Feature>	Int 0 为成功

6.6.1.3 单个特征删除接口

方法名	delPersonById			
说明	删除一张注册的特征			
方法	delPersonById(int pointID)			
请求参数	说明	必须	类型	返回参数
pointID	注册时传入的 id	是	Int	Int 0 为成功

6.6.1.4 全部特征删除接口

方法名	featureClear			
说明	删除所有注册的特征			
请求参数	featureClear()			
请求参数	说明	必须	类型	返回参数
无		无	无	Int 0 为成功

6.6.2 识别多帧检测接口

该接口会筛选识别过程中的识别成功且满足条件优质人脸最为底图重新入库，且可自由设置每个人员的优质人脸数量和条件阈值，建议 8.5 以上的通用版 SDK 中使用

6.6.2.1 多帧检测入库数量接口

方法名	setMaxUpdateSize
-----	------------------

说明	该接口可设置识别多帧的人脸入库数量			
方法	setMaxUpdateSize(int maxUpdateSize)			
请求参数	说明	必须	类型	返回参数
maxUpdateSize	入库数量, 0 为不使用多帧	是	Int	无

6.6.2.2 多帧检测入库时间间隔接口

方法名	setInputDBIntervalTime			
说明	该接口可设置多帧入库后多久可在进行多帧检测入库			
方法	setInputDBIntervalTime(long intervalTime)			
请求参数	说明	必须	类型	返回参数
intervalTime	间隔多久可从继续让识别人脸冲新入库	是	Long	无

6.6.2.3 多帧检测识别阈值设置接口

方法名	setRegisterCompareThreshold			
说明	多帧识别阈值设置，内部会与识别到的人员得分进行比较，满足该阈值可以进行下一步判断，全部通过方可入库			
方法	setRegisterCompareThreshold(float registerCompareThreshold)			
请求参数	说明	必须	类型	返回参数
registerCompareThreshold	多帧检测识别入库阈值	是	Long	无

6.6.2.4 多帧检测更新照阈值设置接口

方法名	setUpdateCompareThreshold			
说明	多帧识别更新照阈值设置，会与内部其他该人员的多帧特征进行比较，满足该阈值可以进行下一步判断，全部通过方可入库			
方法	setUpdateCompareThreshold(float updateCompareThreshold)			
请求参数	说明	必须	类型	返回参数
updateCompareThreshold	多帧检测识别入库阈值	是	Long	无

6.6.2.5 多帧检测入库阈值设置

方法名	setInputDBThreshold			
说明	满足上述阈值判断后，SDK 会判断该识别结果得分是否满足该阈值条件，通过方可入库			
方法	setInputDBThreshold(float inputDBThreshold)			

请求参数	说明	必须	类型	返回参数
inputDBThreshold	多帧入库阈值	是	Long	无

6.6.2.6 多帧检测人员入库监听

方法名	setInputDBListener			
说明	对识别过程中识别图像满足多帧条件后，识别图片入库的监听			
方法	setInputDBListener(InputDBListener inputDBListener)			
请求参数	说明	必须	类型	返回参数
inputDBListener	onInputDB(int id1, int id2) 多帧检测监听，返回多帧入库后的人员 id	是	InputDBListener	无

6.6.3 1:N 识别接口

方法名	search			
说明	当前 feature 和注册的 Feature 集合比对，返回预加载 Feature 集合中命中的 id，feature 字段和比对分值 score；用户可以通过 id 在数据库中查找全量信息。			
方法	search(BDFaceSDKCommon.FeatureType featureType, float threshold, int topNum, byte[] feature)			
请求参数	说明	必须	类型	返回参数
featureType	FeatureType.FEATURE_VIS 生活照 FeatureType.FEATURE_ID_PHOTO 证件照	是	FeatureType	List<Feature> 识别到的人员结果，可通过 id 在数据库中查找
threshold	比对阈值	是	float	
topNum	获取前 num 个 feature+id 映射数组	是	Int	
feature	当前检查人脸特征值	是	Byte[]	

6.6.4 1:1 比对接口

方法名	compare
-----	---------

说明	两张 feature 人脸特征进行比对，返回 float 得分			
方法	compare(BDFaceSDKCommon.FeatureType featureType, byte[] feature1, byte[] feature2, boolean isPercent)			
请求参数	说明	必须	类型	返回参数
featureType	FeatureType.FEATURE_VIS 生活照 FeatureType.FEATURE_ID_PHOTO 证件照	是	FeatureType	float 比对得分 通常 isPercent 建议 设为 true，满分 100 分，推荐阈值 80 分为识别通过
feature	当前检查人脸特征值	是	Byte[]	
feature	比对的人脸特征	是	Byte[]	
isPercent	是否需要阈值映射，建议为 true	是	Boolean	

6.7 BDFaceImageInstance 图片对象

该对象可以将视频流数据或图片(bitmap)数据转换为 SDK 可用图像

构造方法	参数	类型	说明
BDFaceImageInstance(byte[] data, int height, int width, BDFaceSDKCommon.BDFaceImageType imageType, float angle, int isMbyteArrayror)	data	byte[]	视频流数据
	height	Int	高
	width	Int	宽
	imageType	BDFaceImageTyp	图片类型
	angle	Int	旋转角度
	isMbyteArrayror	Int	是否镜像
BDFaceImageInstance(Bitmap bitmap)	bitmap	Bitmap	图片对象

6.7.1 获取图片信息接口

方法名	getImage			
说明	如果需要校验底层图片，通过该接口获取底层送检图片信息数据			
方法	getImage()			
请求参数	说明	必须	类型	返回参数
无	无			无

6.7.2 图片销毁接口

方法名	destory			
说明	销毁图片对象，每一帧图像数据使用完之后，必须调用，否着会出现内存溢出			
方法	destory()			
请求参数	说明	必须	类型	返回参数
无	无			无

6.8 FaceGaze 注意力检测对象

该对象主要用于眼部状态检测

构造方法	参数	类型	说明
FaceGaze()	无		无参构造调用（执行默认创建的 instance）
FaceGaze(BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的 instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.2 BDFaceInstance 线程实例

6.8.1 注意力模型加载接口

方法名	initModel			
说明	眼睛状态检测，同时判断出左眼，右眼 6 种状态，分别为向上，向下，向左，向右，向前，闭合			
方法	initModel(final Context context, final String gazeModel, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
GazeModel	注意力模型	是	String	
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.8.2 注意力信息获取接口

方法名	gaze
-----	------

说明	眼睛状态检测，同时判断出左眼，右眼 6 种状态，分别为向上，向下，向左，向右，向前，闭合			
方法	gaze(BDFaceImageInstance imageInstance, float[] landmarks)			
请求参数	说明	必须	类型	返回参数
imageInstance	图片数据信息，可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	BDFaceGazeInfo 注意力信息 实体类，具体可参考 7.3 BDFaceGazeInfo 实体类
landmarks	FaceDetect 检测后 landmark	是	float[]	

6.8.3 注意力卸载接口

方法名	uninitModel			
说明	卸载模型			
方法	uninitModel()			
请求参数	说明	必须	类型	返回参数
无	无	是	无	Int

6.9 FaceCrop 抠图对象

该对象可以根据人脸框或者人脸关键点进行抠图

构造方法	参数	类型	说明
FaceCrop()	无		无参构造调用（执行默认创建的 instance）
FaceCrop(BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的 instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.2 BDFaceInstance 线程实例

6.9.1 抠图能力加载接口

方法名	initFaceCrop
说明	初始化抠图能力
方法	initFaceCrop(final Callback callback)

请求参数	说明	必须	类型	返回参数
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.9.2 人脸关键点抠图接口

方法名	cropFaceByLandmark			
说明	眼睛状态检测，同时判断出左眼，右眼 6 种状态，分别为向上，向下，向左，向右，向前，闭合			
方法	cropFaceByLandmark(BDFaceImageInstance imageInstance, float[] landmark, float enlargeRatio, boolean correction, AtomicInteger isOutofBoundary)			
请求参数	说明	必须	类型	返回参数
imageInstance	图片数据信息，可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	BDFaceImageInstance
landmarks	FaceDetect 检测后 landmark	是	float[]	
enlargeRatio	抠图放大倍数	是	float	
correction	是否进行人脸矫正	是	Boolean	
isOutofBoundary	抠图是否：是否超出图像范围（是否有黑边） 0 为未超出，1 为超出	是	Int	

6.10 FaceMouthMask 口罩检测接口

构造方法	参数	类型	说明
FaceMouthMask()	无		无参构造调用（执行默认创建的 instance）
FaceMouthMask(BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的 instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.2 BdFaceInstance 线程实例

6.10.1 口罩检测模型加载接口

方法名	initModel
-----	-----------

说明	加载口罩检测模型			
方法	initModel(final Context context, final String mouthMaskModel, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
mouthMaskModel	口罩检测模型路径	是	String	
Callback	模型加载结果, 请参考 9.2 模型初始化错误码说明	是	Callback	

6.10.2 口罩检测结果获取接口

方法名	checkMask			
说明	通过图片和人脸框数据获取口罩检测置信度数据, 0 代表不戴口罩, 1 代表戴口罩			
方法	checkMask(BDFaceImageInstance bDFaceImageInstance, FaceInfo[] faceInfos)			
请求参数	说明	必须	类型	返回参数
imageInstance	图片数据信息, 可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	float[] 口罩检测结果
faceInfos	人脸数据	是	FaceInfo[] 具体可参考 7.1 FaceInfo 实体类	

6.10.3 口罩检测卸载接口

方法名	uninitModel			
说明	卸载模型			
方法	uninitModel()			
请求参数	说明	必须	类型	返回参数
无	无	是	无	Int

6.11 FaceSafetyHat 安全帽检测

构造方法	参数	类型	说明
FaceSafetyHat()	无		无参构造调用 (执行默认创建的 instance)

FaceSafetyHat (BDFaceInstance thisBdFaceInstance)	thisBdFaceInstance	BDFaceInstance	执行自己创建的instance，独立运行在一个线程，多线程调用情况下需要传，具体可参考 6.2 BDFaceInstance 线程实例
---	--------------------	----------------	---

6.11.1 安全帽模型加载接口

方法名	initModel			
说明	加载安全帽检测模型			
方法	initModel(final Context context, final String safetyHatModel, final Callback callback)			
请求参数	说明	必须	类型	返回参数
context	上下文	是	Context	无
safetyHatModel	安全帽检测模型路径	是	String	
Callback	模型加载结果，请参考 9.2 模型初始化错误码说明	是	Callback	

6.11.2 安全帽检测结果获取接口

方法名	checkHat			
说明	检测是否带安全帽			
方法	checkHat (BDFaceImageInstance bdFaceImageInstance, FaceInfo[] faceInfos)			
请求参数	说明	必须	类型	返回参数
imageInstance	图片数据信息，可参考 6.7 BDFaceImageInstance 图片对象	是	BDFaceImageInstance	float[] 口罩检测结果
faceInfos	人脸数据	是	FaceInfo[] 具体可参考 7.1 FaceInfo 实体类	

6.11.3 安全帽检测卸载接口

方法名	uninitModel
说明	卸载模型

方法	uninitModel()			
请求参数	说明	必须	类型	返回参数
无	无	是	无	Int

7 SDK 实体类说明

7.1 FaceInfo 实体类

变量	类型	说明
faceID	Int	人脸索引值，标记连续视频帧追踪中人脸 ID
centerX	float	人脸中心点 x 坐标
centerY	float	人脸中心点 y 坐标
width	float	人脸宽度
height	float	人脸高度
angle	float	人脸角度
score	float	人脸置信度
landmarks	float[]	人脸 72 个关键点数据（鼻子，眼镜，嘴巴，眉毛）
yaw	float	人脸左右偏转角
roll	float	人脸平行平面内的头部旋转角
pitch	float	人脸上下偏转角
bluriness	float	人脸模糊度信息
illum	int	人脸光照信息
occlusion	BDFaceOcclusion	人脸遮挡信息，可参考 7.2 BDFaceOcclusion 实体类
age	Int	人脸年龄
gender	BDFaceGender	性别 BDFACE_GENDER_FEMALE, // 女性 BDFACE_GENDER_MALE, // 男性

bestImageScore	float	最优人脸得分

7.2 BDFaceOcclusion 实体类

变量	类型	说明
leftEye	float	左眼遮挡置信度
rightEye	float	右眼遮挡置信度
nose	float	鼻子遮挡置信度
mouth	float	嘴巴遮挡置信度
leftCheek	float	左脸遮挡置信度
rightCheek	float	右脸遮挡置信度
chin	float	下巴遮挡置信度

7.3 BDFaceGazeInfo 实体类

变量	类型	说明
leftEyeConf	float	左眼的置信度
rightEyeConf	float	右眼的置信度
leftEyeGaze	BDFaceGazeDirection	左眼的注意力信息 BDFACE_GAZE_DIRECTION_UP, // 向上看 BDFACE_GAZE_DIRECTION_DOWN, // 向下看 BDFACE_GAZE_DIRECTION_RIGHT, // 向右看 BDFACE_GAZE_DIRECTION_LEFT, // 向左看 BDFACE_GAZE_DIRECTION_FRONT, // 向前看 BDFACE_GAZE_DIRECTION_EYE_CLOSE, // 闭眼
rightEyeGaze	BDFaceGazeDirection	右眼的注意力信息 同上

7.4 Feature 实体类

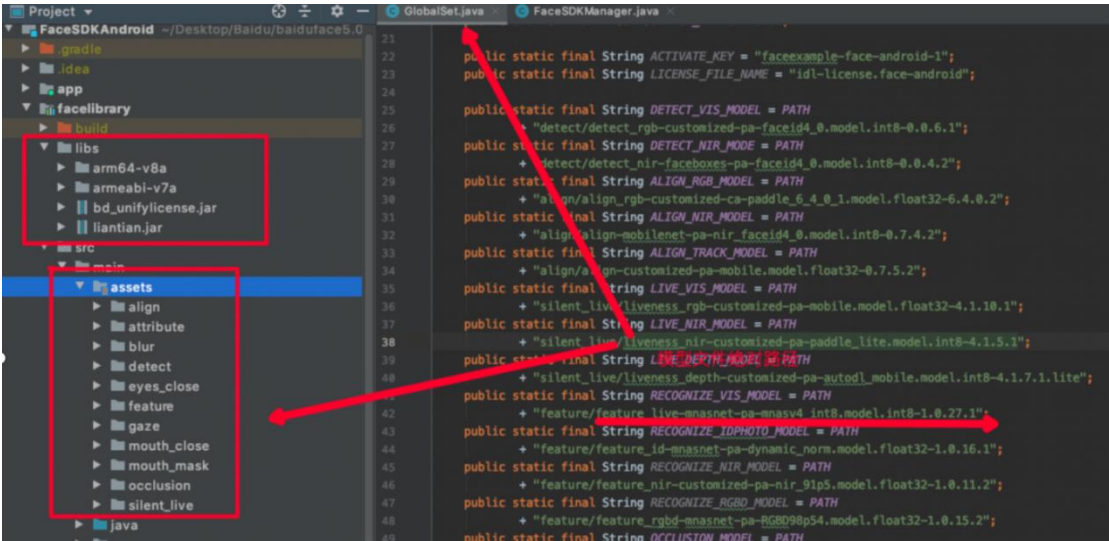
变量	类型	说明
id	int	人员 id
feature	byte[]	人员特征，通常为 512 字节组数 (3568 专版 SDK 为 1024 字节)
score	float	识别得分

8 升级指导

注意:不同版本的人脸特征模型不能同步，大版本更新（小版本不需要，如 8.1 更新到 8.2 则不需要, 7.0 到 8.0 需要）需要删除人脸库，重新导入人脸

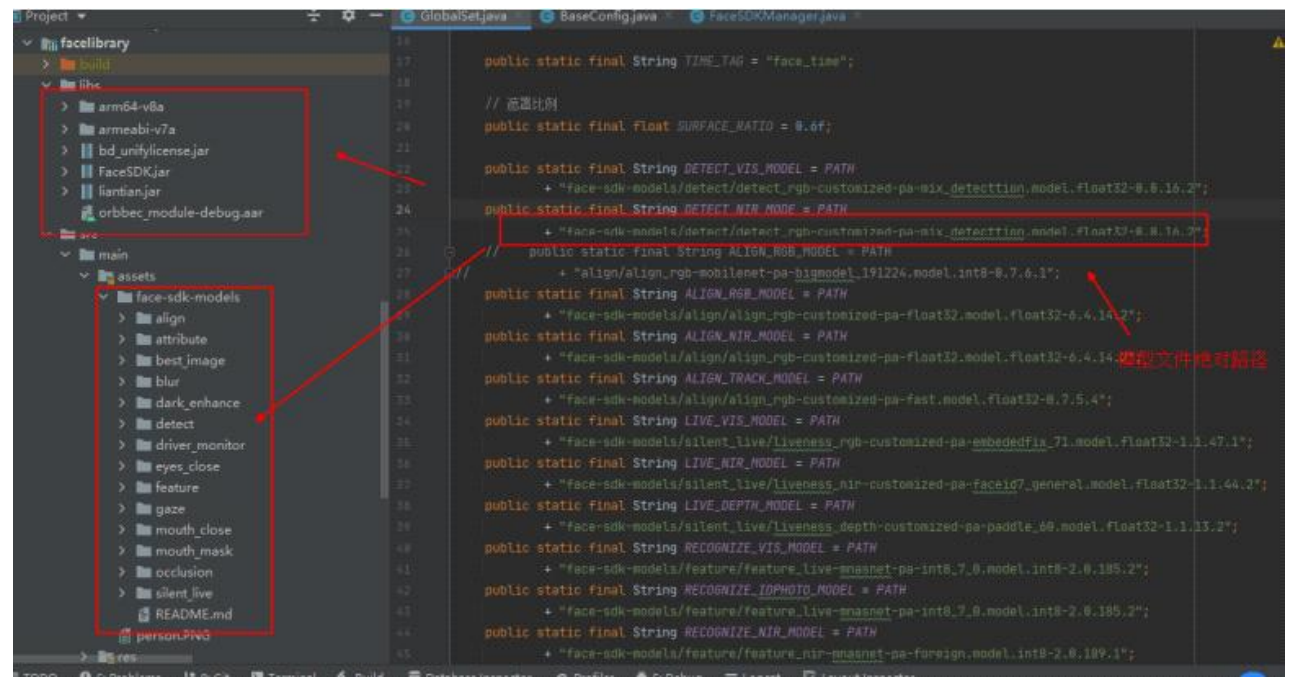
8.1 3.v - 4.v

libs 下所有 jar 和文件替换成 4.0 的 2.assets 下所有模型文件替换成 4.0 的，GlobalSet 模型路径要对应到 assets 下相应的文件夹



8.2 4.v - 7.v

版本更新后, SDK 对应的识别 活体 人脸检测等模型, 均会更新, 不同版本模型无法同步, 所有需要更新最新模型路径, 模型路径通常在每个模块下的 GlobaSet 文件夹中



8.3 7.v - 8.v

(更新版本要刷新人脸底库, 重新导入人脸)

同上所述, 版本更新后, SDK 对应的识别 活体 人脸检测等模型, 需更新 GlobaSet 文件下的所有模型路径



8.0 开始, 识别对象从 FaceFeature 改为了 FaceSearch, 添加了单个人脸特征注册功能, 请添加最新识别对象

2 usages

```
private FaceSearch faceSearch;
```

添加最新识别对象

```
if (bdFaceInstance == null){
    faceFeature = new FaceFeature();
    faceSearch = new FaceSearch();
}else {
    faceFeature = new FaceFeature(bdFaceInstance);
    faceSearch = new FaceSearch(bdFaceInstance);
}
faceSearch.setInputDBListener(new FaceSearch.InputDBListener() {
    no usages
    @Override
    public void onInputDB(int i, int ii){ Log.e( tag: "face_feature_db_add" , msg: i + " " + ii); }
});
faceSearch.setMaxUpdateSize(10);
faceSearch.setInputDBIntervalTime(0);
faceSearch.setRegisterCompareThreshold(0.8f);
faceSearch.setUpdateCompareThreshold(0.9f);
faceSearch.setInputDBThreshold(0.92f);
```

faceSearch初始化

```
public void run() {
    synchronized (FaceApi.getInstance().getAllUserList()) {
        FaceSDKManager.getInstance().getFaceFeature().featurePush(new ArrayList<Feature>());
        FaceSDKManager.getInstance().getFaceSearch().pushPersonFeatureList(
            FaceApi.getInstance().getAllUserList());
    }
    if (FaceApi.getInstance().getmUserNum() != 0) {
```

更新最新push注册接口

```
List<? extends Feature> featureResult = FaceSDKManager.getInstance()
    .getFaceSearch().search(type, topNum: 1, feature, isPercent: true);
```

识别接口调整

// TODO 返回top num = 1 个数据集, 此处可以任意设置, 会返回比从大到小排序的num 个数据集

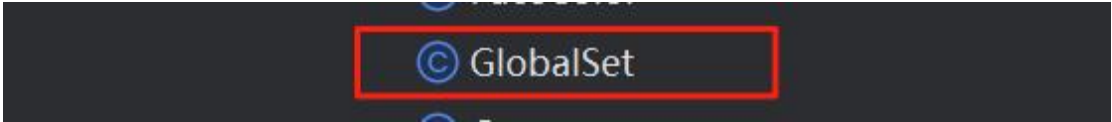
其他版本 SDK 升级 8.1+, 需替换模型以及 SDK

替换为

更新模型

The screenshot shows the 'facelibrary' project structure in Android Studio. On the left, the 'libs' directory contains 'FaceSDK.jar', which is highlighted with a red box. A red arrow points from this box to the right, where the 'face-sdk-library' project structure is shown. In the 'libs' directory of 'face-sdk-library', 'FaceSDK_8.1_20230216-release.aar' is highlighted with a red box. Below this, the 'src/main/assets' directory is shown, with 'face-sdk-models' highlighted with a red box. A red arrow points from the 'FaceSDK.jar' box to the 'FaceSDK_8.1_20230216-release.aar' box, and another red arrow points from the 'face-sdk-models' box to the 'FaceSDK_8.1_20230216-release.aar' box.

替换后更新模型路径文件



9 激活和模型初始化错误码说明

9.1 在线和离线初始化错误码说明

错误码	描述
0	激活成功
1	license 未初始化
2	license 数据解密失败
3	license 格式错误
4	key 校验错误
5	算法 id 校验错误
6	MD5 校验失败
7	设备 id 校验失败
8	包名校验错误
9	过期时间不正确
10	功能未授权
11	授权过期
12	本地文件读取失败
13	远程数据拉取失败
14	本地时间校验失败
1001	文件读取失败
1002	缓存清理失败
1003	zip 解析失败
1004	zip 文件读取失败
1005	没找到授权文件
1006	context 为空
1007	devicesId 为空
1008	json 异常
1009	http 请求异常

1010	http 数据异常
1011	网络链接异常
28200	未知错误
216100	非法参数
216102	不支持的类型
216101	请求参数不足
290000	无操作权限
290001	序列号生成错误
290002	序列号错误
290003	设备标识错误
290004	序列号被激活
290005	license 生成失败
290006	license 过期时间设置错误
290007	license 更新失败
290008	license 下载次数超过上限
290009	license 被其他设备绑定
290010	license 设备绑定失败
290011	license 查询失败

9.2 模型初始化错误码说明

错误码	描述
0	激活成功
1	context 为 null
-1	非法的参数
-2	内存分配失败
-3	实例对象为空
-4	模型内容为空
-5	不支持的能力类型
-6	不支持预测类型
-7	预测库对象创建失败
-8	预测库初始化失败
-9	图像数据为空
-10	人脸能力初始化失败
-11	能力未加载
-12	人脸能力已加载
-13	未授权

-14	人脸能力运行异常
-15	不支持的图像类型
-16	图像转换失败

9.3 图片特征批量提取错误码说明

错误码	描述
-1	特征提取失败
1	图片格式不符合要求
2	bitmap 为空
4	人脸角度不通过
5	图像模糊
6	人脸遮挡
7	人脸曝光
8	未检测到人脸
9	检测到多个人脸
10	抠图失败
128	特征提取成功

10 常见问题

10.1 误识别问题

10.1.1 误识底图与识别图片得分过高导致误识别

通常为底图与识别图五官相似度较高导致，请保留误识别时的识别图片与误识别底图，以便方便排查，保存方式请参考 SaveImageManager 文件，图片会存储在 sdcard/Save-Image/文件夹下：

```
14 public class SaveImageManager {
25     public void saveImage(final LivenessModel livenessModel, final BDLiveConfig bdLiveConfig){
30
31         future3 = es3.submit(new Runnable() {
32             @Override
33             public void run() {
34                 String currentTime = System.currentTimeMillis() + "";
35                 BDFaceImageInstance rgbImage = livenessModel.getBdFaceImageInstance();
36                 BDFaceImageInstance nirImage = livenessModel.getBdNirFaceImageInstance();
37                 BDFaceImageInstance depthImage = livenessModel.getBdDepthFaceImageInstance();
38                 if (rgbImage != null){
39                     Bitmap bitmap = BitmapUtils.getInstanceBmp(rgbImage);
40                     if (livenessModel.getRgbLivenessScore() > bdLiveConfig.rgbLiveScore){
41                         saveImage(bitmap, url: "Save-Image" + "/" + currentTime,
42                             name: currentTime + "_RGB_Feature");
43                     }else {
44
45                         saveImage(bitmap, url: "Save-Image" + "/" + currentTime,
46                             name: currentTime + "_RGB_Live");
47                     }
48                 }
49             }
50         });
51     }
52 }
```

需通过该方法保存rgb识别图片

误识别底图和视频流图片获取成功后，可使用 demo 中的人证核验模块，对比误识别图片得分是否过高



如果得分相似都超过阈值（默认 0.8）则说明误识别人员与识别人员五官相似度过高，可修改人脸注册底图解决该问题

如果相似度不高，说明 SDK 内存混乱导致，这种情况大概率为多线程边导入边识别造成的，则在人脸注册和识别接口加锁：

```

// TODO 函数可以根据不同模型调整
if (featureCheckMode == 3) {
    synchronized (faceModel.getFaceSearch()) {
        List<? extends Feature> featureResult =
            faceModel.getFaceSearch().search(type, bdFaceCheckConfig.scoreThreshold, topNum, 1, feature);
    }
    // TODO 返回top num = 1 个数据集, 此处可以任意设置, 会返回比从大到小排序的num 个数据集
    if (featureResult != null && featureResult.size() > 0) {

```

注册接口前进行加锁

```

future3 = ps3.submit(
    new Runnable() {
        @Override
        public void run() {
            faceModel.getFaceSearch().featureClear();
            synchronized (faceModel.getFaceSearch()) {
                List<User> users = FaceApi.getInstance().getAllUserList();
                for (int i = 0; i < users.size(); i++) {
                    User user = users.get(i);
                    faceModel.getFaceSearch().pushPersonById(user.getId(), user.getFeature());
                }
            }
            if (FaceApi.getInstance().getUserNum() != 0) {
                ToastUtils.toast(context, "人脸库加载成功");
            }
        }
    }
);

```

人脸注册过程中添加同步锁, 以免误识别

10.1.2 多人脸情况下误识别到后方人脸

多人脸情况下 SDK 的人脸检测接口会返回多个人脸信息, 可通过每个人脸的宽高进行判断, 获取最大的人脸送入后续识别, 具体代码如下

```

public class FaceSDKManager {
    // usage
    private FaceInfo[] getTrackCheck(BDFaceImageInstance rgbInstance) {
        // 快速检测获取人脸信息, 仅用于控制人脸框, 详细人脸数据后连接获取
        FaceInfo[] faceInfos = faceModel.getFaceTrack().track(
            BDFaceSDKCommon.DetectType.DETECT_VIS,
            BDFaceSDKCommon.AlignType.BDFACE_ALIGN_TYPE_RGB_FAST,
            rgbInstance);
        return faceInfos;
    }

    // usage
    private void onDetect(
        BDFaceCheckConfig bdFaceCheckConfig,
        BDFaceImageInstance rgbInstance,
        FaceInfo[] fastFaceInfos,
        LivenessModel livenessModel,
        int index,
        DetectListener detectListener) {
        long accurateTime = System.currentTimeMillis();
        FaceInfo[] faceInfos;

        if (bdFaceCheckConfig != null) {
            bdFaceCheckConfig.bdFaceDetectListConfig.usingQuality = true;
            faceInfos = faceModel.getFaceDetect().detect(
                BDFaceSDKCommon.DetectType.DETECT_VIS,
                BDFaceSDKCommon.AlignType.BDFACE_ALIGN_TYPE_RGB_ACCURATE,
                rgbInstance,
                fastFaceInfos,
                bdFaceCheckConfig.bdFaceDetectListConfig);
        }
    }
}

```

人脸快速检测接口

这两个接口均为人脸检测接口, 都会返回一个人脸信息数组

人脸检测接口

```

public class FaceInfo {
    public int faceID;
    public float centerX;
    public float centerY;
    public float width;
    public float height;
    public float angle;
    public float score;
    public float[] landmarks;
    public float yaw;
    public float roll;
    public float pitch;
    public float blurriness;
    public float illum;
    public BDFaceOcclusion occlusion;
}

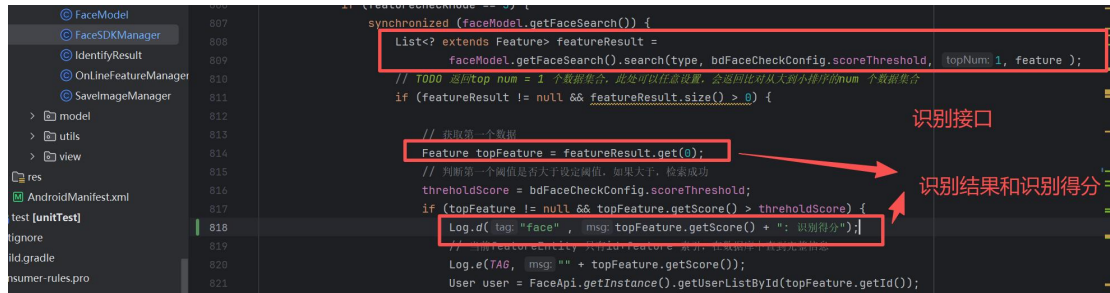
```

通过人脸检测接口可获取到每个人脸的宽高, 可判断每个宽高大小, 从而获取最大的人脸进行送检识别

10.1.3 没有注册底图的人被误识别到

该问题有可能为识别人员五官相似度与库中某一个特征五官相似度相似导致，目前 SDK 会有万分之一概率出现五官相似度过高问题。

可以先获取误识别时的识别得分，如果得分高于阈值可以对阈值进行调整或更新一张误识别人员的底图解决该问题，识别得分获取方式如下：



10.1.4 全量下发人脸(特征注册)同时进行识别，出现误识别如何解决

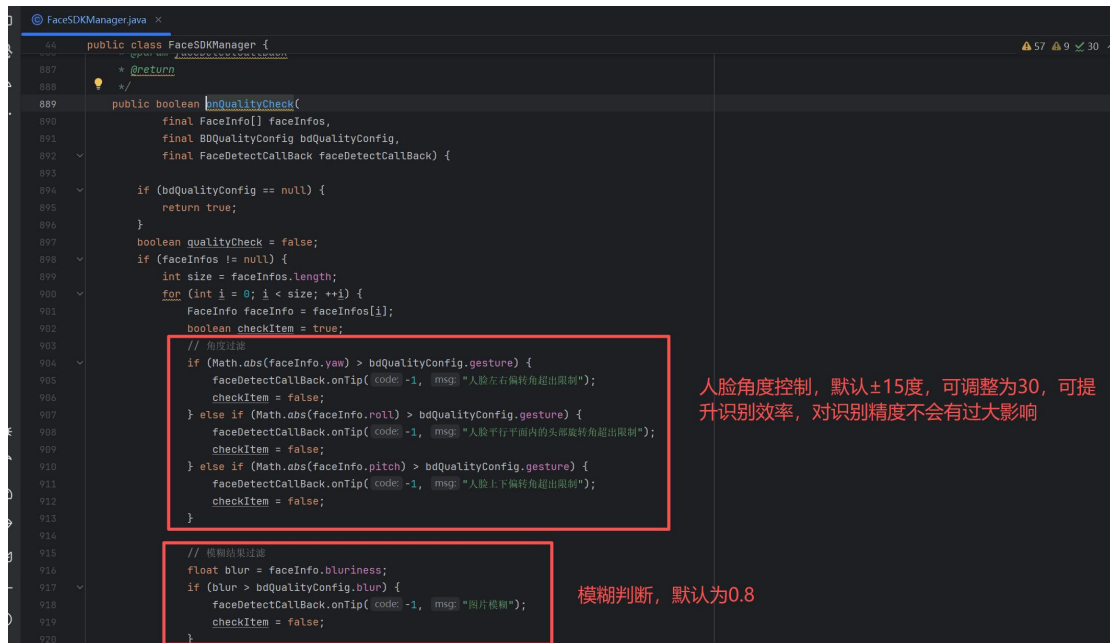
特征注册接口(pushPersonById 或 pushPersonFeatureList)将特征注册到 SDK 缓存时，不能同时调用识别接口(search)，因为注册过程中会打乱底层特征列表顺序，导致识别到的人脸与实际返回的人脸数据不同造成误识别，建议在识别和注册接口加锁保证线程同步，操作如下：



10.2 识别效果问题

10.2.1 识别速度慢，识别率低，识别得分低如何优化？

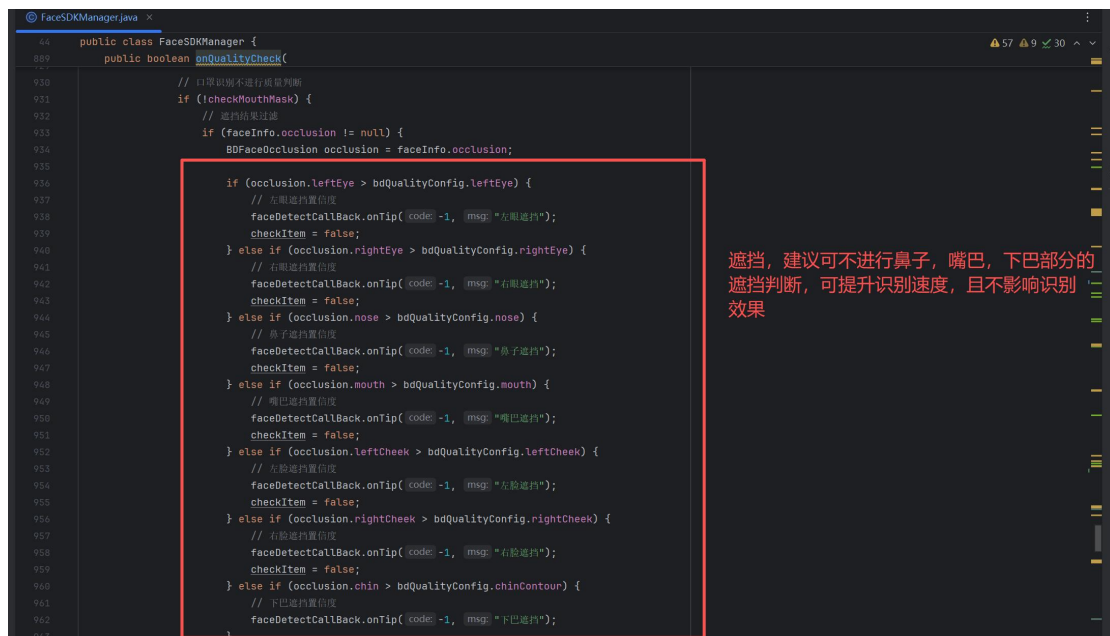
识别速度慢，识别率低，识别率低和速度慢大概率为质量检测未通过导致，可调整质量检测阈值进行优化，质量检测阈值和得分判断如下图：



```
public class FaceSDKManager {  
    // ...  
    public boolean onQualityCheck(  
        final FaceInfo[] faceInfos,  
        final BDQualityConfig bdQualityConfig,  
        final FaceDetectCallBack faceDetectCallBack) {  
        if (bdQualityConfig == null) {  
            return true;  
        }  
        boolean qualityCheck = false;  
        if (faceInfos != null) {  
            int size = faceInfos.length;  
            for (int i = 0; i < size; ++i) {  
                FaceInfo faceInfo = faceInfos[i];  
                boolean checkItem = true;  
                // 角度过滤  
                if (Math.abs(faceInfo.yaw) > bdQualityConfig.gesture) {  
                    faceDetectCallBack.onTip( code: -1, msg: "人脸左右偏转角超出限制");  
                    checkItem = false;  
                } else if (Math.abs(faceInfo.roll) > bdQualityConfig.gesture) {  
                    faceDetectCallBack.onTip( code: -1, msg: "人脸平行平面内的头部偏转角超出限制");  
                    checkItem = false;  
                } else if (Math.abs(faceInfo.pitch) > bdQualityConfig.gesture) {  
                    faceDetectCallBack.onTip( code: -1, msg: "人脸上下偏转角超出限制");  
                    checkItem = false;  
                }  
                // 模糊结果过滤  
                float blur = faceInfo.bluriness;  
                if (blur > bdQualityConfig.blur) {  
                    faceDetectCallBack.onTip( code: -1, msg: "图片模糊");  
                    checkItem = false;  
                }  
            }  
        }  
    }  
}
```

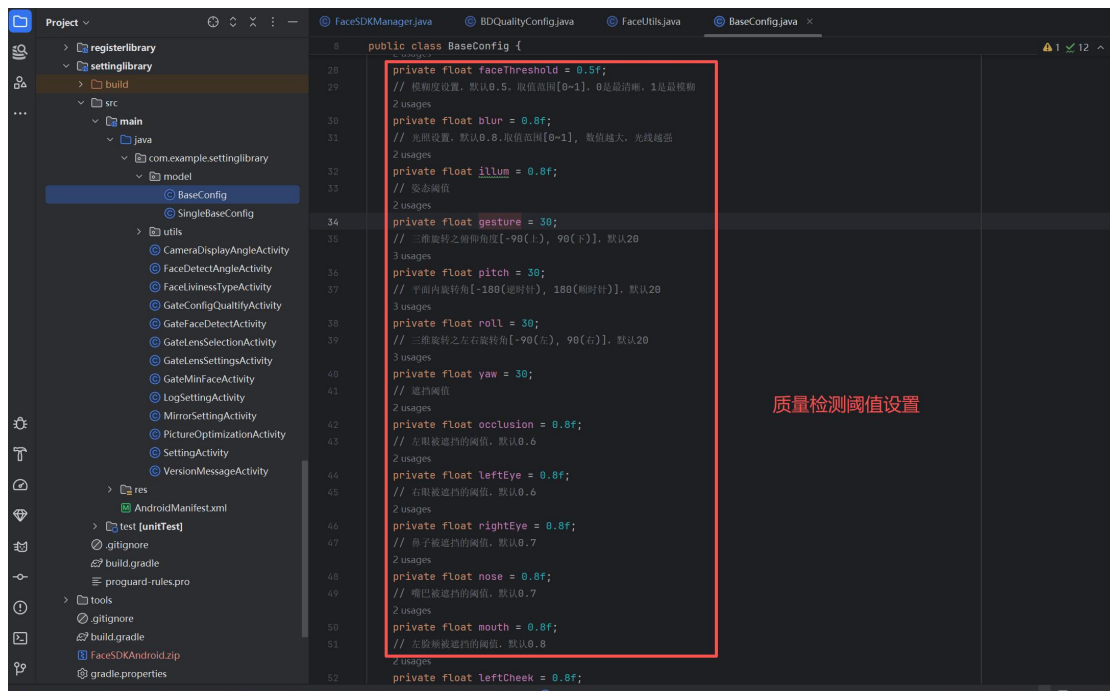
人脸角度控制，默认 ± 15 度，可调整为30，可提升识别效率，对识别精度不会有过大影响

模糊判断，默认为0.8



```
public class FaceSDKManager {  
    // ...  
    public boolean onQualityCheck(  
        // ...  
        if (!checkMouthMask) {  
            // 遮挡结果过滤  
            if (faceInfo.occlusion != null) {  
                BDFaceOcclusion occlusion = faceInfo.occlusion;  
                if (occlusion.leftEye > bdQualityConfig.leftEye) {  
                    // 左眼遮挡位置  
                    faceDetectCallBack.onTip( code: -1, msg: "左眼遮挡");  
                    checkItem = false;  
                } else if (occlusion.rightEye > bdQualityConfig.rightEye) {  
                    // 右眼遮挡位置  
                    faceDetectCallBack.onTip( code: -1, msg: "右眼遮挡");  
                    checkItem = false;  
                } else if (occlusion.nose > bdQualityConfig.nose) {  
                    // 鼻子遮挡位置  
                    faceDetectCallBack.onTip( code: -1, msg: "鼻子遮挡");  
                    checkItem = false;  
                } else if (occlusion.mouth > bdQualityConfig.mouth) {  
                    // 嘴巴遮挡位置  
                    faceDetectCallBack.onTip( code: -1, msg: "嘴巴遮挡");  
                    checkItem = false;  
                } else if (occlusion.leftCheek > bdQualityConfig.leftCheek) {  
                    // 左脸遮挡位置  
                    faceDetectCallBack.onTip( code: -1, msg: "左脸遮挡");  
                    checkItem = false;  
                } else if (occlusion.rightCheek > bdQualityConfig.rightCheek) {  
                    // 右脸遮挡位置  
                    faceDetectCallBack.onTip( code: -1, msg: "右脸遮挡");  
                    checkItem = false;  
                } else if (occlusion.chin > bdQualityConfig.chinContour) {  
                    // 下巴遮挡位置  
                    faceDetectCallBack.onTip( code: -1, msg: "下巴遮挡");  
                    checkItem = false;  
                }  
            }  
        }  
    }  
}
```

遮挡，建议可不进行鼻子，嘴巴，下巴部分的遮挡判断，可提升识别速度，且不影响识别效果



若质量检测通过排查识别得分是否过低，得分过低需排查识别接口的 isPrcent 入参是否为 true，false 情况下需改为 true 在测试，若没问题可更新一张近期的人脸底图测试是否能够正常识别，isPercent 参数和识得分见下图：



10.2.2 注册成功后无法正常识别到

该问题需检查是否通过 pushPersonById 接口将注册人员的特征放入 SDK 缓存

注册分为 ‘特征提取 - 特征注册到本地数据库 - 将数据库人员 id 和特征注册到 SDK 缓存’ 三步操作

可能是特征提取成功并注册到本地数据库后未将人员特征放入缓存导致，可检查是否通过 pushPersonById 接口将特征注册到 SDK 缓存，接口如下：

```

172     future3 = es3.submit(new Runnable() {
173         @Override
174         public void run() {
175             faceModel.getFaceSearch().featureClear();
176             synchronized (faceModel.getFaceSearch()) {
177                 List<User> users = FaceApi.getInstance().getAllUserList();
178                 for (int i = 0; i < users.size(); i++) {
179                     User user = users.get(i);
180                     faceModel.getFaceSearch().pushPersonById(user.getId(), user.getFeature());
181                 }
182                 if (FaceApi.getInstance().getmUserNum() != 0) {
183                     ToastUtils.toast(context, text: "人脸库加载成功");
184                 }
185             }
186         }
187     });

```

注册到数据库的人脸特征需通过该接口放到SDK缓存

10.2.3 数据库查询不到识别到的人员 id

该问题通常为识别流程正常通过，识别得分满足阈值筛选条件，但无法从数据库中查询到该人员信息。原因为 SDK 缓存中的 id 和人脸特征与数据库中的人员信息不匹配导致(可能是数据库更新特征时 SDK 缓存特征为清理导致)，可通过 FaceSearch.clear() 方法清空 SDK 缓存, 从新同步数据库人脸特征，代码截图如下：

```

}

future3 = es3.submit(new Runnable() {
    @Override
    public void run() {
        faceModel.getFaceSearch().featureClear();
        synchronized (faceModel.getFaceSearch()) {
            List<User> users = FaceApi.getInstance().getAllUserList();
            for (int i = 0; i < users.size(); i++) {
                User user = users.get(i);
                faceModel.getFaceSearch().pushPersonById(user.getId(), user.getFeature());
            }
            if (FaceApi.getInstance().getmUserNum() != 0) {
                ToastUtils.toast(context, text: "人脸库加载成功");
            }
        }
    }
});

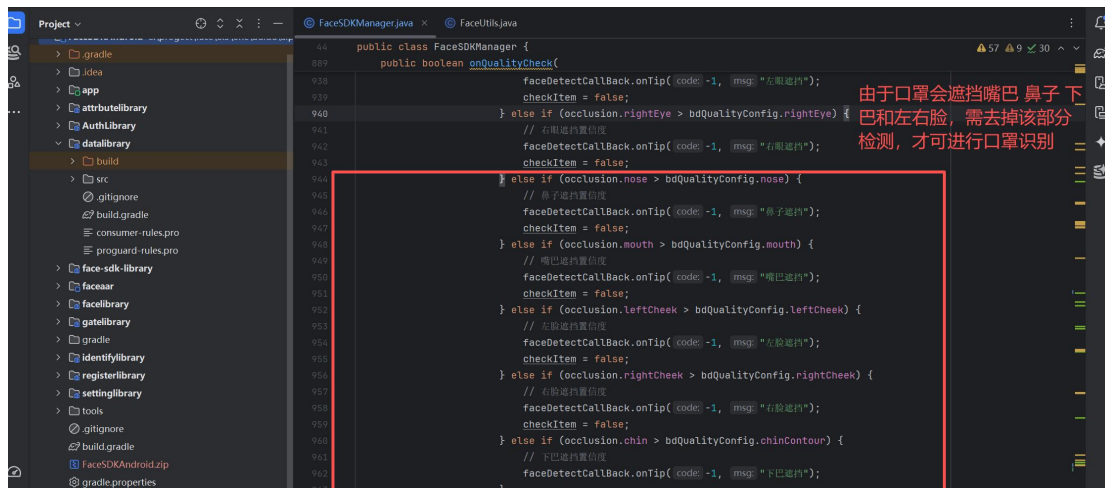
```

清空SDK缓存

将数据库人脸特征从新注册到SDK缓存

10.2.4 戴口罩如何识别？

首先 SDK 支持戴口罩识别，但由于嘴巴鼻子和部分面部被遮挡原因，识别精度会相比不带口罩情况下会降低，需要项目中将质量检测中遮挡部分的判断去掉可进行戴口罩识别，具体代码如下



10.2.5 是否支持远距离如何识别和人脸检测

SDK 建议范围在 0.5~1.5 米(部分比如广角摄像头建议在 1 米内)，超过 1.5 米以上由于面部不清晰会导致识别精度下降或检测不到人脸问题，8.5 版本 SDK 可通过 BDFaceSDKConfig.scaleRatio 参数提升人脸检测的距离，参数越大检测到的距离越远，具体代码如下：

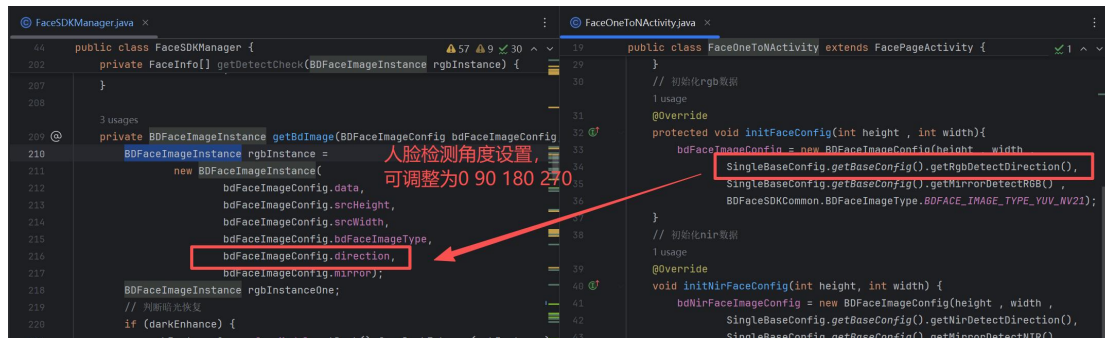


10.2.6 老版特征 SDK 更新后识别不到人脸

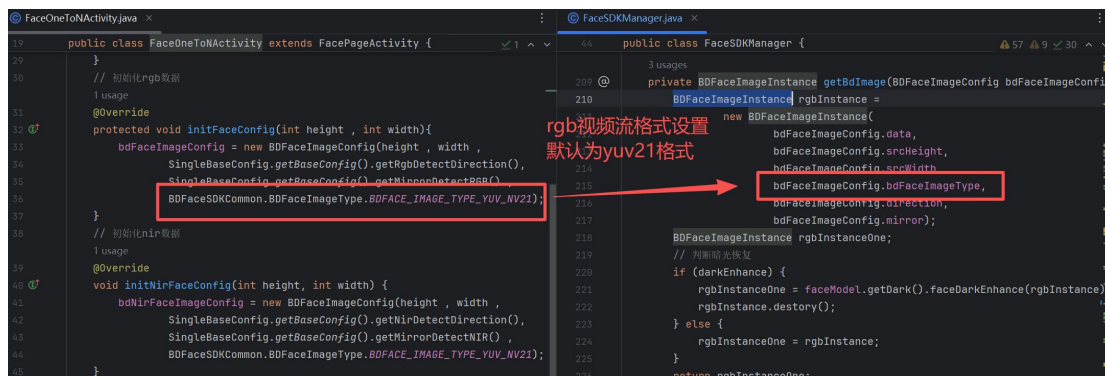
大版本不同情况下 SDK 特征不可公用，比如 6.0、7.0、8.0 此类大版本不同 SDK，需从新提取人脸特征，复用老版特征情况下会出现识别得分普遍偏低、误识别等问题。8.0、8.2、8.5 此类大版本相同 SDK 人脸特征可以公用

10.2.7 检测不到人脸，结果返回为空

该问题通常为 track 接口返回为 null 检测不到人脸导致，通常原因为人脸检测角度调整错误或者摄像头数据不匹配导致，可以先调整 BDFaceImageInstance 中的人脸角度参数(0 90 180 270)测试能否检测到人脸，代码如下：

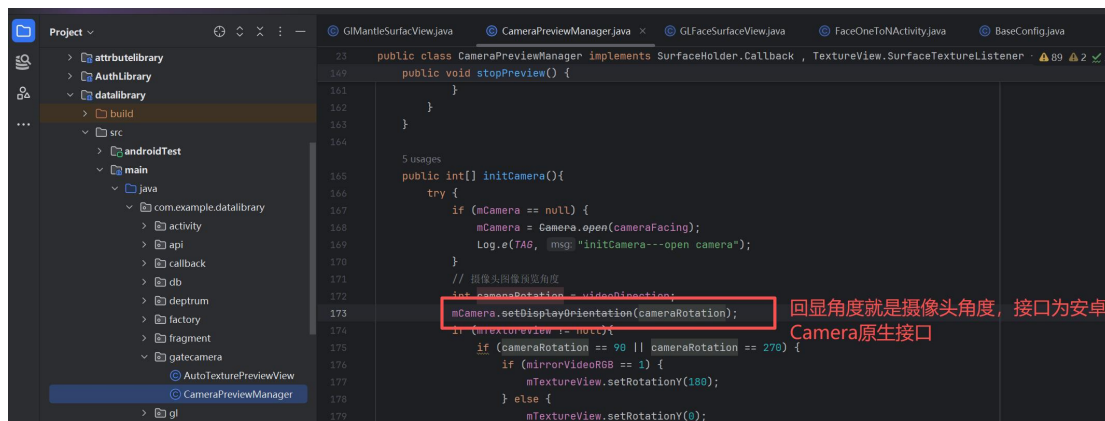


如果 4 个角度都检测不到人脸，可先参考 SaveImageManager.java 类保存人脸检测图片，如果图像成像错误（花屏），为视频流格式错误，可与摄像头厂获取摄像头数据格式，调整识别参数解决，具体截图如下：

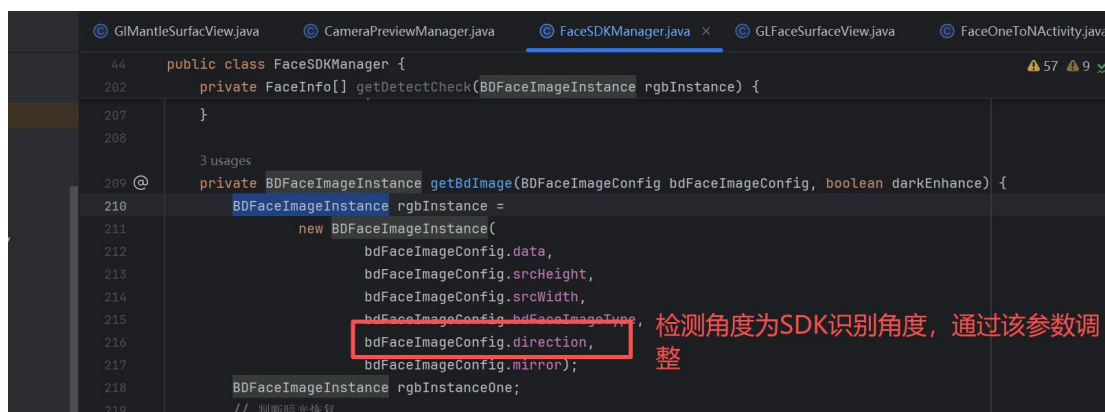


10.2.8 检测角度和回显角度是歪的如何解决

该问题可参考 demo 项目设置中的镜头设置，可调整回显角度和检测角度，具体代码如下：
回显角度接口：

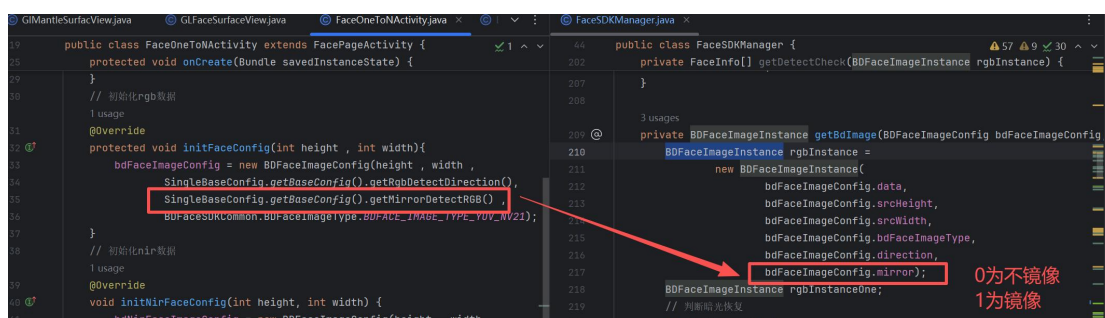


检测角度接口：

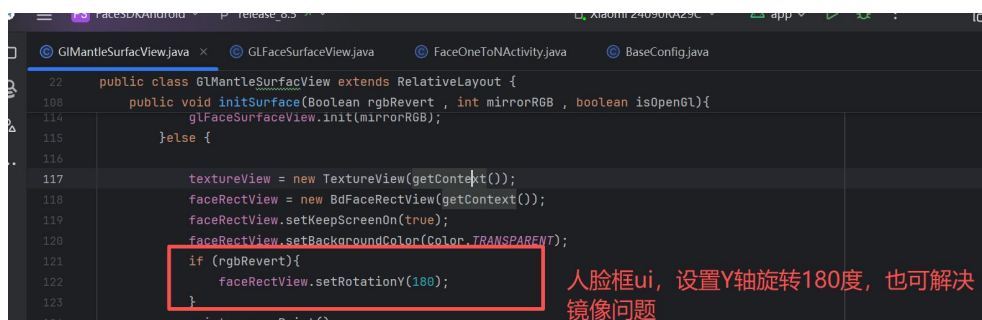


10.2.9 人脸框左右人脸镜像

通常为检测图片与摄像头成像相反(镜像)导致，该问题可以通过调整送检的人脸镜像参数或调整人脸框 view 的旋转值控制(可参考 demo 设置中-镜头设置)，具体代码截图如下：
通过人脸镜像参数调整



通过人脸框 view 旋转参数调整



10.2.10 RGB 活体检测被 2D 图片攻破

目前项目中只使用 RGB 单目活体会存在被 2D 图片或电子图片攻破的情况，采用双目摄像头 RGB+NIR 检测为最优，只有单目情况下可以开启多音子模型优化单目活体效果(开启后活体接口耗时会上升，且通过率会下降)，活体多因子模型如下：

```
36 public void initModel(context context) {
37     faceLiveness.initModel(context,
38         GlobalSet.LIVE_VIS_MODEL,
39         GlobalSet.LIVE_VIS_2DMASK_MODEL,
40         GlobalSet.LIVE_VIS_HAND_MODEL,
41         GlobalSet.LIVE_VIS_REFLECTION_MODEL,
42         "", visHandModel: "", visReflectionModel: "",
43         GlobalSet.LIVE_NIR_MODEL,
44         GlobalSet.LIVE_DEPTH_MODEL,
45         new Callback() {
46             @Override
47             public void onResponse(int code, String response) {
48                 Log.e("face_model", response);
49                 // ToastUtils.toast(context, code + " " + response);
50                 if (code != 0 && listener != null) {
51                     listener.initModelFail(code, response);
52                 }
53             }
54         }
55     );
56 }
```

10.2.11 CPU 占用高 使用卡顿 如何解决

识别速度慢(总耗时 1s 以上)或 CPU 占用一直在 80~90%，可能原因为系统占用过高，SDK 与系统抢占 CPU 资源导致，可通过 adb 或 Profiler 查看系统占有率(部分设备启动摄像头占用会比较高)排查是否为 cpu 占用过高问题，cpu 占有率查看如下：

```
tasks: 239 total, 2 running, 237 sleeping, 0 stopped, 0 zombie
Mem: 2005396K total, 1931672K used, 75493376 free, 109308K buffers
Swap: 1002692K total, 156160K used, 846532K free, 613308K cached
400%cpu 192%user 22%nice 59%sys 113%idle 0%iow 12%irq 2%irq 0%host
```

PID	USER	PR	NI	VIRT	RES	SHR	S[%CPU]	%MEM	TIME+	ARGS
23606	u0_a366	10	-10	14G	391M	151M	R 206	19.9	3:12.58	com.baidu.idl.+
274	cameraserver	20	0	151M	56M	43M	S 27.3	2.8	827:22.99	android.hardwa+
20922	cameraserver	20	0	88M	23M	19M	S 11.3	1.1	0:33.89	cameraserver
178	system	-3	-20	11G	30M	24M	S 10.0	1.5	390:36.13	surfaceflinger
223	system	-3	-8	10G	6.9M	6.3M	S 6.6	0.3	251:11.91	android.hardwa+
25102	shell	20	0	10G	6.4M	4.8M	S 4.6	0.3	0:03.21	transport -cont
141	logd	30	10	10G	6.1M	2.6M	S 3.3	0.3	94:17.52	logd
148	root	20	0	10G	2.0M	2.0M	S 2.3	0.1	87:24.60	rknn_server
28450	shell	20	0	10G	4.3M	3.3M	R 1.6	0.2	0:00.14	top
23415	root	0	-20	0	0	0	I 0.6	0.0	0:00.24	[kworker/u9:2-+
88	root	RT	0	0	0	0	S 0.6	0.0	13:28.59	[irq/23-rga]
23500	shell	20	0	10G	2.1M	1.6M	S 0.3	0.1	0:00.46	process-tracke+
23413	root	20	0	0	0	0	I 0.3	0.0	0:00.52	[kworker/0:2-e+
23355	root	20	0	0	0	0	I 0.3	0.0	0:00.29	[kworker/3:1-e+

解决方案：

可通过 setCoreConfigure 接口降低 SDK cpu 使用线程下降 cpu 占有率，截图如下

```
1 usage
private FaceSDKManager() {
    faceAuth = new FaceAuth();
    faceAuth.setCoreConfigure(BDFaceSDKCommon.BDFaceCoreRunMode.BDFACE_LITE_POWER_NO_BIND, coreNum: 2);
}
```

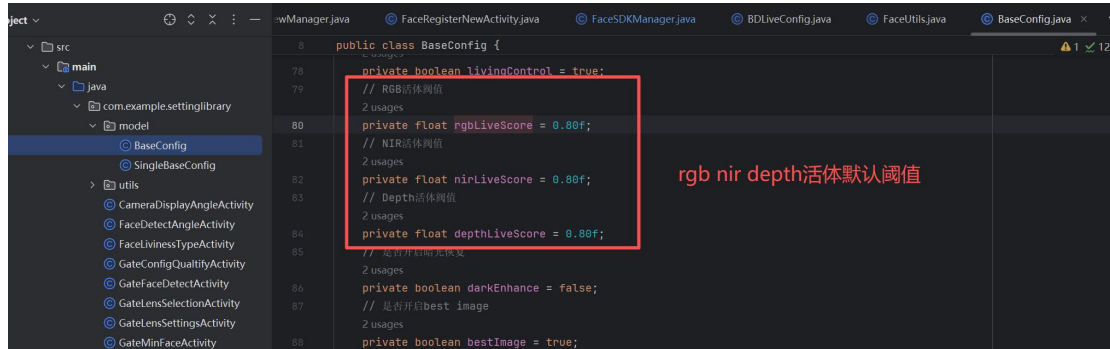
3568 专版 SDK 可通过 powerMode 参数，降低 cpu 占用，阈值建议为 0 高档 1 中档 2 抵挡：

```
BDFaceSDKConfig config = new BDFaceSDKConfig();
// TODO: 最小人脸个数检查，默认设置为1,用户根据自己需求调整
config.maxDetectNum = 2;
// TODO: cpu占用设置，默认为0
config.powerMode = SingleBaseConfig.getBaseConfig().getCpuUp();

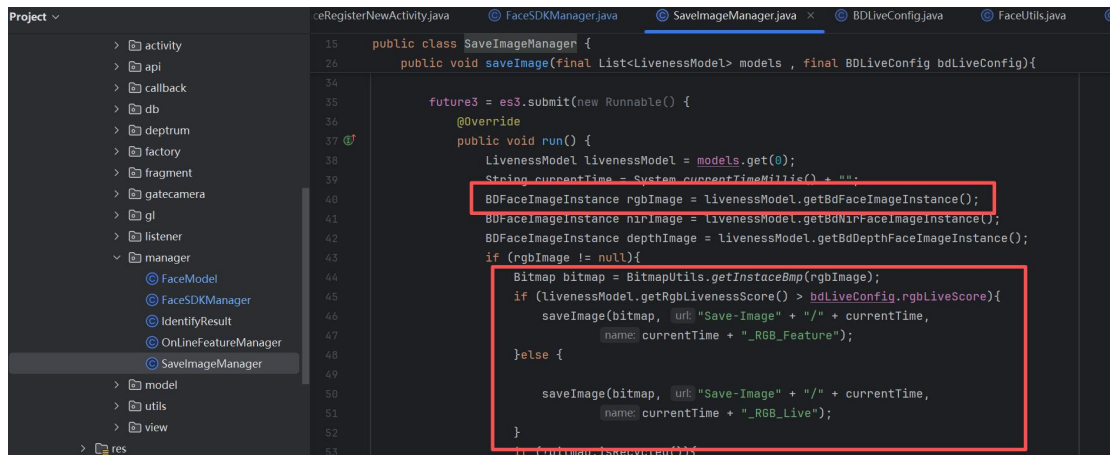
// TODO: 默认为80px。可传入大于30px的数值，小于此大小的人脸不予检测，生效时间第一次加载模型
config.minFaceSize = SingleBaseConfig.getBaseConfig().getMinimumFace();
```

10.2.12 活体得分低如何排查

如果活体得分长时间普遍偏低(人脸检测角度正常)情况下,可能为摄像头成像和摄像头适配问题,可以先确保摄像头镜面无模糊、水汽、贴有半透明保护膜等情况。若摄像头镜面和成像显示正常,可降低活体得分或保存活体不通过图片提交工单,活体得分设置如下图:



活体不通过图片保存方法见下图:



10.2.13 图片批量注册不成功 特征提取失败如何解决?

图片注册失败可先排查中注册图片分辨率,目前建议分辨率不超过 2000*3000,不要小于 640*480,过大会导致无法检测到人脸,建议送检前进行图像分辨率压缩。图片过小则会导致面部大小过小导致检测不到人脸 或 面部质量差出现质量检测不通过情况,可通过根据项目需求调整人脸最小值和质量部分代码来控制,截图如下:

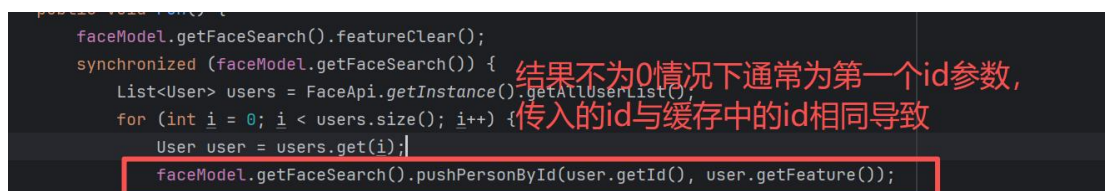




若分辨率正常，可以获取失败错误码，参考 [9.3 图片特征批量提取错误码说明](#) 排查错误原因。

10.2.14 特征注册接口 pushPersonById 注册失败如何解决？

该问题通常为 pushPersonById 注册人脸特征时报错 1003 导致，通常为注册 id 冲突导致，需确保传入 id 没有重复(全量导入情况下线调用 clear 接口情况 SDK 缓存特征)，具体截图如下：



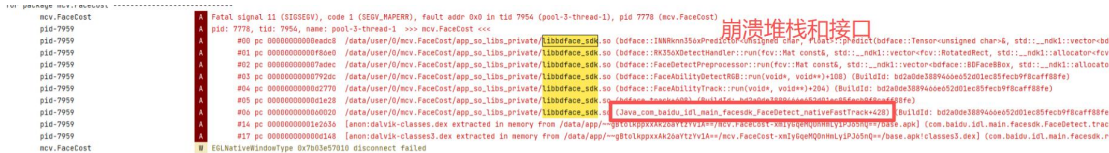
10.2.15 同一时间反复初始化数据库卡死如何解决？

项目中引用的数据库为安卓自带的 sqlite 数据库，初始化数据为耗时操作不建议同一时间反复初始化，通常 5w 底库初始化需要 5 分钟左右，建议放入子线程并进行线程同步，截图如下：

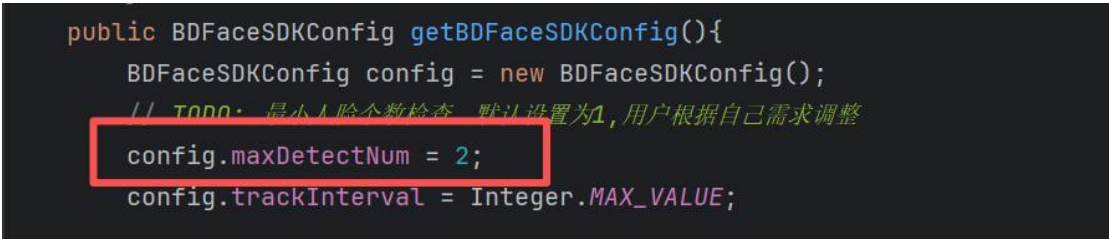


10.2.16 接口崩溃如何解决？

接口崩溃 SDK 会打印接口的堆栈崩溃信息，通常为 track 或 detect 接口的崩溃，崩溃截图如下：

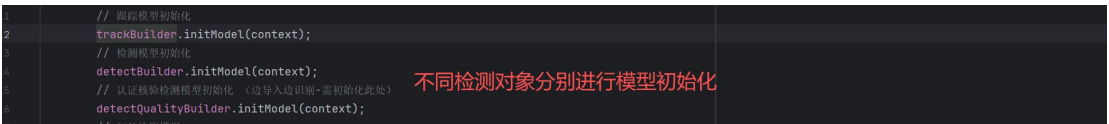
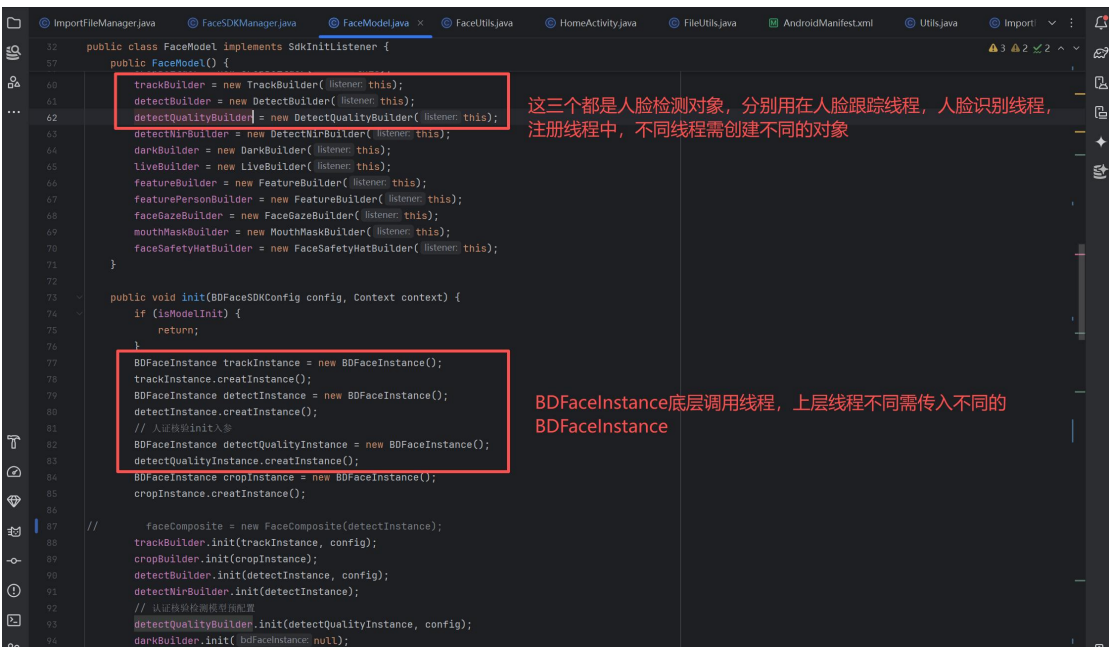


部分老版本 SDK (8.2、8.0、3568_3.8 专版) SDK 有可能在部分设备上出现该问题，可以调整人脸检测数量为 2，解决该问题，调整接口如下：



10.2.17 多线程人脸检测 (图片注册+识别) 崩溃如果解决

该问题通常为同一个检测对象在多个线程同时被调用导致，SDK 不支持单个对象多线程下调用，不同线程需创建不同的检测和识别对象，传入不同的 BDFaceInstance 区分调用，具体截图如下：



10.2.18 如何判断图片中是否存在多人脸？

需要先将人脸检测数量接口调整为 >2 ，后判断人脸检测接口返回的数组长度即可，截图如下：



10.2.19 如何只检测画面中指定区域的人脸？

人脸检测接口中会返回人脸的宽高和人脸中心点，可以通过中心点和宽高判断人脸是否在指定区域，宽高与中心点获取方式如下：



10.2.20 SDK 支持多远距离下识别？能否设置远距离人脸识别？

目前市面上不同摄像头相同距离下拍摄出的人脸大小距离有区别，测试环境下 SDK 支持 0.5~1.5 米内人脸距离识别，如果是广角或超广角摄像头识别距离会缩短。

可以通过 SDK 的 `BDFaceSDKConfig.scaleRatio` 参数提升人脸检测距离，目前建议最远可设置为 2，支持 2 米左右的人脸检测识别（太远情况下可检测到人脸但识别精度对下降），具体代码如下：

```
BDFaceSDKConfig config = new BDFaceSDKConfig();
config.scaleRatio = 2;

// 设置人脸检测距离，默认设置为 1，用户可根据自己摄像头调整
} else {
    faceDetect = new FaceDetect(bdFaceInstance);
}
}
@Override
public void init(BDFaceInstance bdFaceInstance, BDFaceSDKConfig config) {
    if (bdFaceInstance == null) {
        faceDetect = new FaceDetect();
    } else {
        faceDetect = new FaceDetect(bdFaceInstance);
    }
}
```

将BDFaceSDKConfig设置到检测对象中

10.2.21 人脸特征是否支持保存到后台下发到其他设备？

支持，人脸特征为 512 位的 byte 数组，可通过 Base64 转换为字符串保存至后台服务器（服务器需自行部署）后下发到其他设备在解密为数组注册识别，具体代码如下：

```
byte[] bytes = new byte[512];
ImportFeatureResult result;
// 10、走人脸SDK接口，通过人脸检测、特征提取拿到人脸特征值
result = getFeature(bitmap1, bytes,
    BDFaceSDKCommon.FeatureType.BDFACE_FEATURE_TYPE_LIVE_PHOTO);

// 11、判断是否提取成功：128为成功，-1为参数为空，-2表示未检测到人脸
Log.i(TAG, "live_photo = " + result.getResult());
if (result.getResult() == 128) {
    // 将用户信息保存到数据库中
    boolean importDBSuccess = FaceApi.getInstance().registerUserIntoDBmanager(
        groupName: null,
        userName, picName, userInfo: null, bytes);

    // Rename usages
    String featureStr = Base64.encodeToString(bytes, Base64.DEFAULT);
    // 保存数据库成功
}
```

人脸特征

特征转换为String

```
faceModel.getFaceSearch().featureClear();
synchronized (faceModel.getFaceSearch()) {

    String base64String = "";

    byte[] feature = Base64.decode(base64String, Base64.DEFAULT);
    faceModel.getFaceSearch().pushPersonById(id, feature);
}
```

服务器下发的人脸特征，解码为byte数组

特征id

10.2.22 通用版 SDK 与 3568 专版 SDK 有什么区别？SDK 支持哪个版本的 npu？

通用版 SDK：为 cpu 算力没有使用 npu 算力加持，适用于市面上 5.0 以上大部分设备。

3568 专版 SDK：仅支持在 3568 和 3566 主板上使用，有 npu 加持，识别速度相比通用版快 1~2 倍数。支持 1.1.0 到 1.4.0 的 npu 使用

10.2.23 SDK 是否支持外国人识别？有没有专版 SDK

目前最新 8.5 版本 SDK 识别模型融合了外国人和国人的识别功能，支持白人、黑人、国人的人脸识别。

10.3 激活问题

10.3.1 设备指纹获取失败，在线激活报错 no enough params 如何解决

该问题通常出现在 android 10 以上的设备中，由于部分系统差异问题存在该情况，可以关闭电话权限，在项目中搜索 android.permission.READ_PHONE_STATE 权限并全部注释后，卸载 app 重装可以正常获取设备指纹



10.3.2 每次启动都需要激活吗？

根据目前 SDK 策略，app 启动并激活只限于本次启动时使用，杀死程序、设备重启等需要重新打开 app 的操作需要重新进行 SDK 激活和模型初始化。且 SDK 支持静默激活，可在 app 启动时进行调用 SDK 激活接口，无需添加 UI 界面手动点击激活

10.3.3 相同设备不同应用能否使用相同激活码?

可以，SDK 是通过设备指纹进行激活，相同设备不进行恢复出厂设置、系统更新、更换硬件情况下不同应用获取的指纹不变，可以使用相同激活码，并且可通过 SDK 的 `getDeviceId` 接口获取设备指纹，截图如下

```
String deviceId = new FaceAuth().getDeviceId(getAppActivity());
```

10.3.4 激活时填写了不属于硬件指纹的号码或设备指纹改变如何解决?

在离线激活设备指纹填错，恢复出厂设置、系统更新、更换硬件等操作原因导致设备指纹改变，情况下当前激活码无法从新绑定新的指纹，且不可回收，需更换其他激活码从新激活。

10.3.5 在线或批量激活可以不联网激活吗?

在线和批量激活，首次激活需要进行联网，激活成功后会把授权信息保存至 `/data/data/包名 缓存目录` 下,之后第二次开始激活无需进行联网。

但 APP 卸载、清空缓存等操作会清空缓存目录，需从新联网激活，在线、批量激活授权信息缓存位置如下

> com.baidu.mi	drwxrwx--x	2025-12-18 10:39
> com.baidu.idl.face.demo	drwxrwx--x	2025-12-18 10:39
> cache	drwxrws--x	2025-11-11 11:32
> code_cache	drwxrws--x	2025-12-19 11:38
> databases	drwxrwx--x	2025-11-11 11:33
> files	drwxrwx--x	2025-11-11 11:33
> shared_prefs	drwxrwx--x	2025-11-11 11:33
≡ app_idl-license.face-android	-rw-----	2025-11-11 11:33
> com.baidu.input.mi	drwxrwx--x	2025-12-18 10:39

10.3.6 激活失败怎么办?

激活过程中可能会出现激活失败问题，并打印 `license expired error` 等日志，该问题需要手动打印激活的错误码和错误提示来进一步判断错误问题，错误码可参考 [9.1 在线和离线初始化错误码说明](#)，拿离线激活举例，错误码和错误提示可参考如下截图：

```

bdFaceAuth.initLicenseOffline(getActivity(), new Callback() {
    @Override
    public void onResponse(int code, String response) {
        Log.e("face", "错误码"+code + "提示: " + response);
    }
});

```

10.3.7 设备指纹绑定那些信息？

目前安卓离线人脸识别 SDK 获取的设备指纹与 cpu、硬盘、网卡，手机系统等软硬件信息有关，没特殊情况不建议刷机、更改硬件设备

10.4 摄像头问题

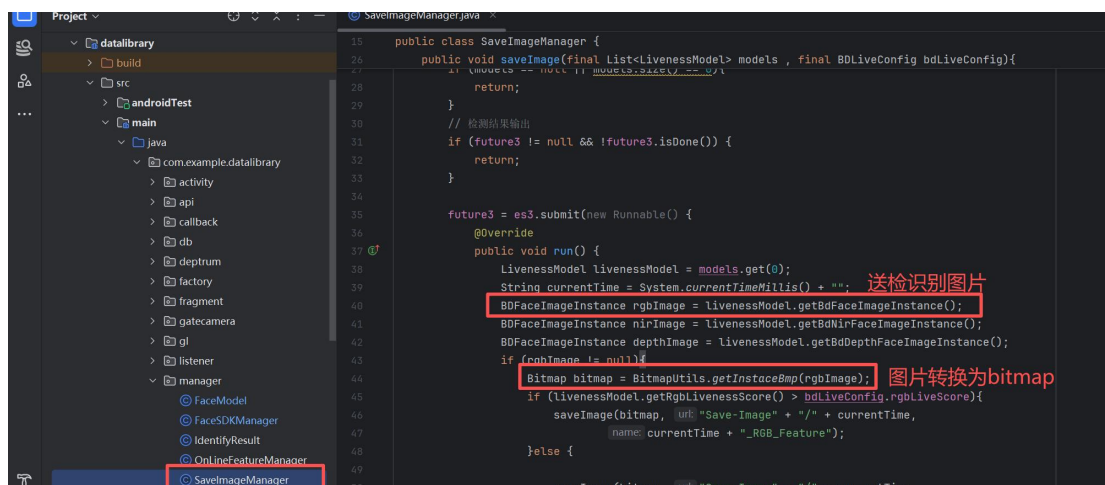
10.4.1 SDK 支持哪些摄像头？

目前市场上安卓自带 Camera 可正常启动的 RGB 和 NIR 摄像头均可使用(包括外接 usb 摄像头)，Depth 摄像头目前 SDK 支持奥比、华捷摄像头

10.4.2 相机采集到的图像如果转成图片？

通常 SDK 与相机部分没有直接关系，相机的每帧图像、ui 展示、启动都是使用的安卓自带的接口完成，通常 RGB 相机会返回 YUV21 格式的视频流数据，可转换为 RGBA 格式后转换为 Bitmap 图片。

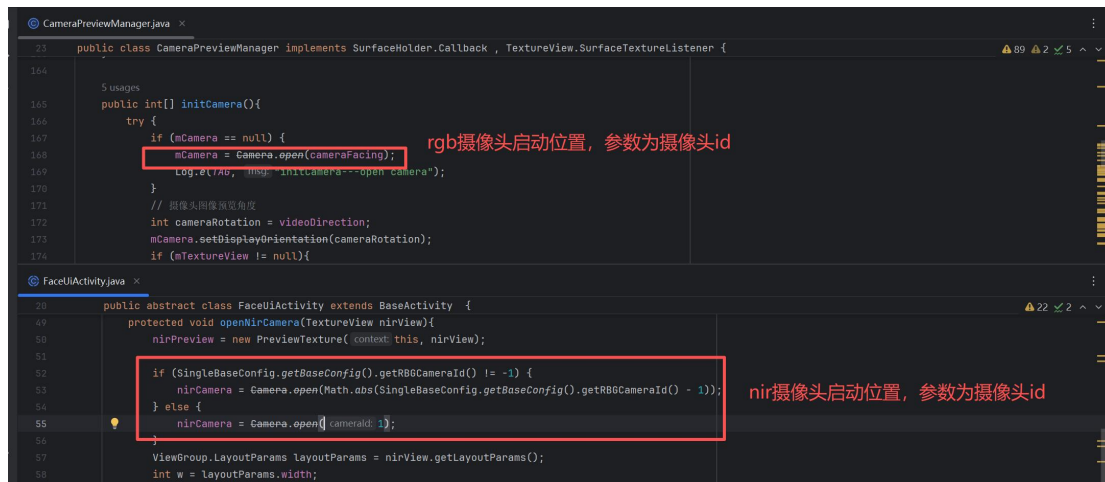
送检的视频流数据会转换为 SDK 需要的 BDFaceImageInstance 图片对象，可参考工程中 SaveImageManager 类转换为 Bitmap 保存，具体代码如下：



10.4.3 摄像头是黑白的, 摄像头想调整为前(后)置摄像头如何调整

摄像头黑白通常为 rgb+nir 双目摄像头导致。

原因为摄像头 id 错误导致, 可调整传入 Camera.open() 接口参数解决该问题(通常为 0 为 rgb 1 为 nir, 部分设备在硬件组装时会搞反), 截图如下:

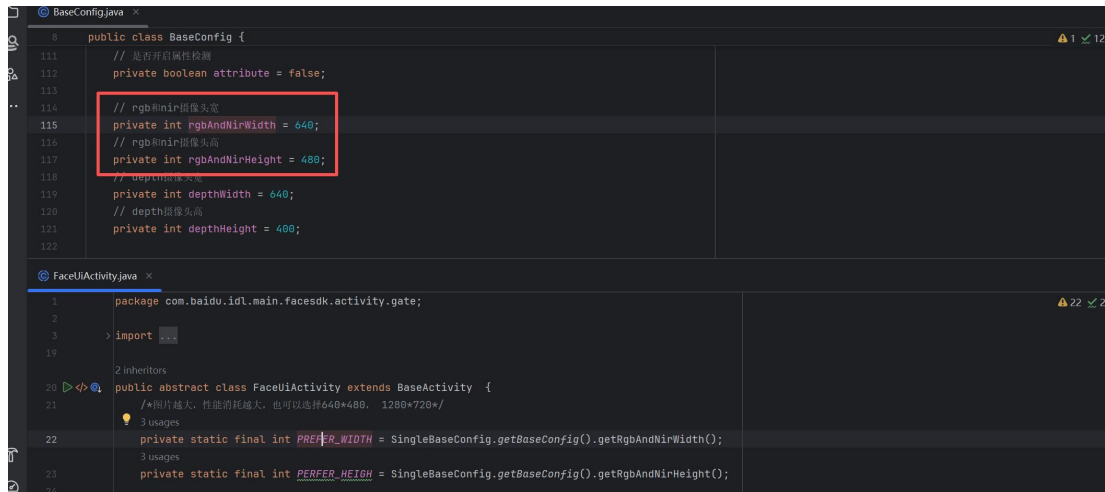


10.4.4 单目 NIR 能否进行人脸识别

目前 SDK 不支持单目 NIR 的人脸识别, 会影响识别和活体精度, 建议使用 RGB 或 RGB+NIR 双目摄像头进行人脸识别

10.4.5 更改分辨率后预览崩溃如何解决

该问题可能与摄像头不支持该分辨率有关, 通常摄像头支持的分辨率为 640*480、1280*720, demo 默认为 640*480, 可先跟厂家沟通是否支持修改的分辨率, demo 工程的分辨率调整接口如下:



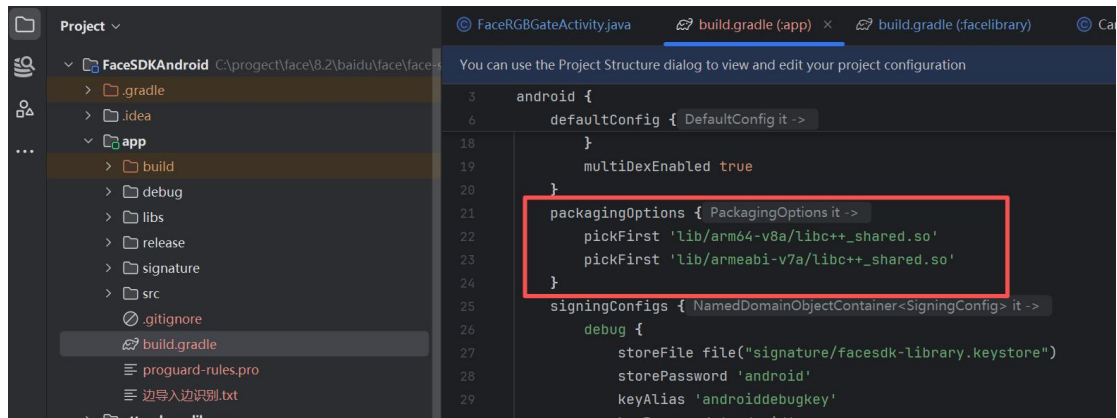
10.5 demo 工程相关问题

10.5.1 人脸注册的图片存在 SD 卡 Success-Import 文件下，能否删除？

可以，但 demo 工程进入数据库界面时会由于找不到图片显示不出人脸图片，不影响特征识别和注册

10.5.2 人脸 SDK 与 openCV 库冲突如何解决

该问题通常为 libc++_shared.so 库冲突导致，可在 gradle 中添加过滤解决该问题，截图如下：



10.5.2 demo 是否支持保存相机注册时的图片？

Demo 支持保存识别时的图片，目前不支持保存相机注册图片，可在工程中调整注册部分代码，使用 SaveImageManage 类进行存图，图片会存在 SD 卡 SaveImage 文件夹下，具体截图如下：



10.6 其他问题

其他问题可关注官网 Android-sdk-常见问题，调用文档如下：

<https://cloud.baidu.com/doc/FACE/s/Zk37clnnn>